POWER APPS CANVAS · SHAREPOINT

Office Supplies Canvas Rebuild Guide

Rebuilding the Office Supplies app in Power Apps Canvas on the existing SharePoint lists

For	A Power Apps maker who will build the app in Studio
Rebuilds	The Office Supplies SPFx web part, build 1.0.0.20 (31 of 31 UAT cases passing)
Data	The 12 existing lists on the Office Supplies SharePoint site
Date	19 October 2026

Contents

1. Start here

- What you are building
- The screens at a glance
- Where everything comes from
- How to use this guide
- What is not in this guide

2. Before you start

- What you need
- The twelve lists
- Create the app
- Connect the lists
- Set the data row limit to 1 for now
- Check the Yes/No filter first

3. The app shell

- Paste App.Formulas
- Paste App.OnStart
- Create the screens
- Set the start screen
- The frame every screen shares
- Pieces every screen reuses

4. Home — scrHome

- Two ways to build it
- Load the data
- The tiles
- Awaiting my action
- My requests
- Fix the pasted screen

5. Work queue — scrWorkQueue

- Build it

6. New request — scrNewRequest

- Load the screen
- Request details
- The items table
- Top bar, callouts and buttons

7. Request detail — scrRequestDetail

- Load the request
- Header and summary
- The journey rail
- Requested items
- The tabs

8. Review, approve and confirm — scrReview

- Load the request
- The top of the screen
- Specify distribution (Review stage)
- Approved items (Approval and Confirm stages)
- The reason dialog

9. Distribute — scrDistribute

- Load the request
- Who and how
- Items to distribute

10. Confirm receipt — scrReceipt

- Load the request
- The screen

11. Catalogue & matrix — scrAdmin

- Load the screen
- Tab 1 — Approval matrix
- Tab 2 — Catalogue & stock
- Tab 3 — Offices & sharing

12. Reports — scrReports

- How Reports gets its numbers
- The tiles
- The charts
- Top consumed items

13. The buttons that save

- Section 7 — Shared recipes
- Section 8 — Save draft and Submit (a new request)
- Section 9 — Resubmit (a Draft or Returned request)
- Section 10 — Proceed (Review, Approval and Confirm)
- Section 11 — Return and Reject
- Section 12 — Record distribution
- Section 13 — Confirm receipt
- Section 14 — Comments and documents
- Section 15 — Administration writes
- Column-reference audit
- Open uncertainties, collected

14. The document flow

- Build it
- Connect it to the app
- Notes

15. Test, run UAT and release

- Step 1 — the delegation check, at a row limit of 1
- Step 2 — raise the limit and test the buttons

Step 3 — user acceptance testing
Step 4 — release
Moving between environments
Keeping this repository in step with the app

Appendix A. The lists and their columns

Appendix B. Defect checklist

Defect checklist

Appendix C. Decisions to confirm with the product owner

Small differences from the current app
Not reproduced

Appendix D. Glossary

1. Start here

This guide walks you through building the **Office Supplies** app again, this time as a **Power Apps Canvas app**. It runs on the same twelve SharePoint lists the current app uses. Follow it from front to back and you end up with an app that behaves the same way as the current one, including every fix found in user acceptance testing (UAT).

What you are building

Office Supplies lets staff in a field office ask for supplies, and lets the right people check, approve and hand them out. Every request follows the same journey:

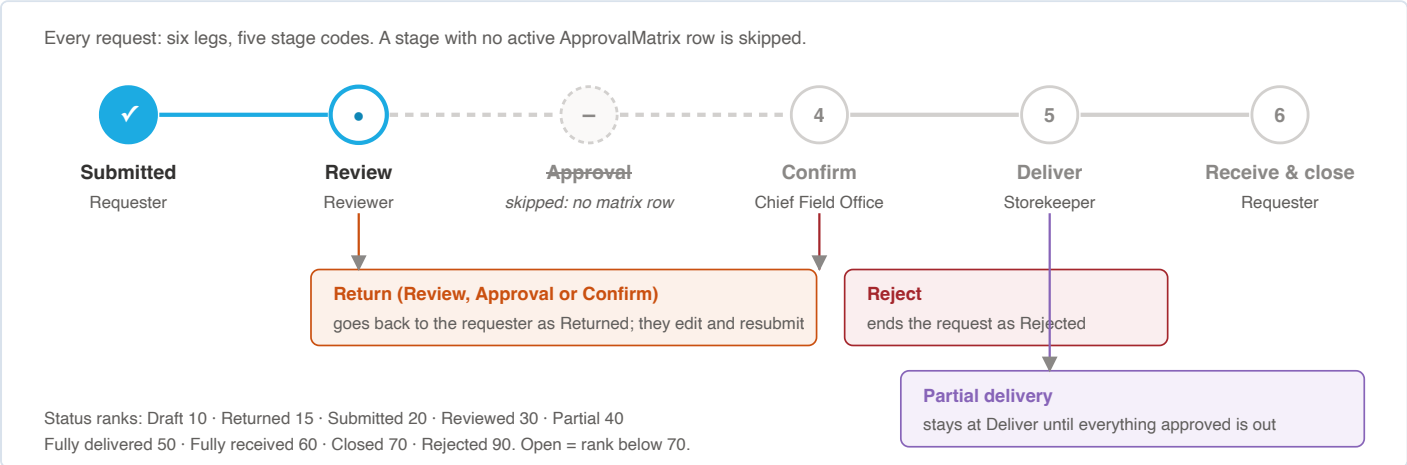


Figure 1. The request journey. A stage with no approval-matrix row for the office is skipped.

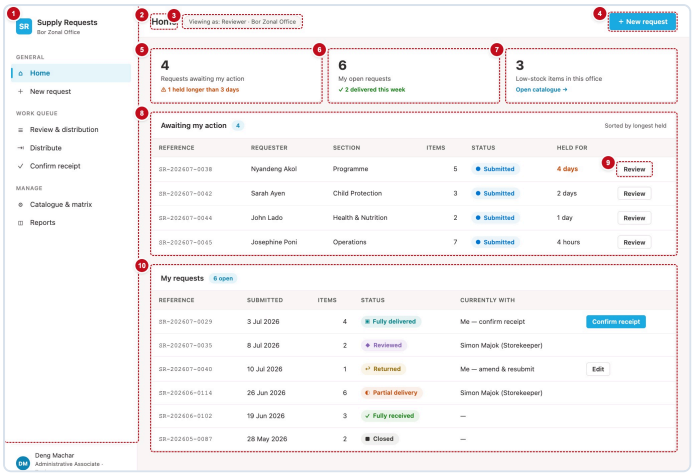
Five roles take part. One person can hold several roles.

Role	What they do in the app
Requester	Any member of staff. Raises a request, follows it, confirms receipt when it arrives.
Reviewer	The office's first approver. Sets how much of each item to give out, and can return or reject the request.
Chief Field Office	Confirms a reviewed request before anything is handed out.
Storekeeper	Hands out the stock, all at once or in parts.
Administrator	Sets up offices, the approval chain, the catalogue and stock, and which offices lend to each other.

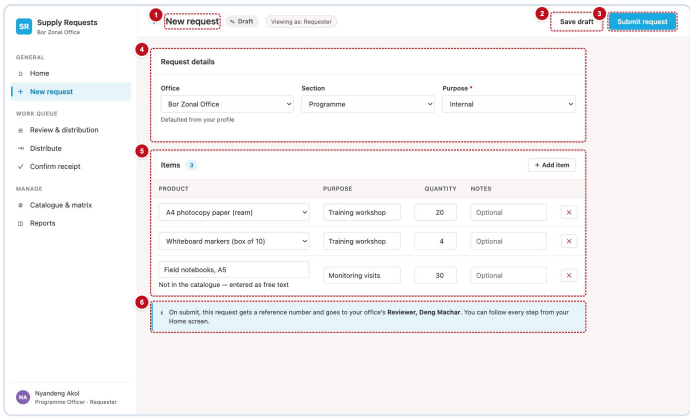
Who acts at each stage comes from the **approval matrix**, one row per office and stage. An office that has no row for a stage skips it. That is why Bor has no finance Approval step.

The screens at a glance

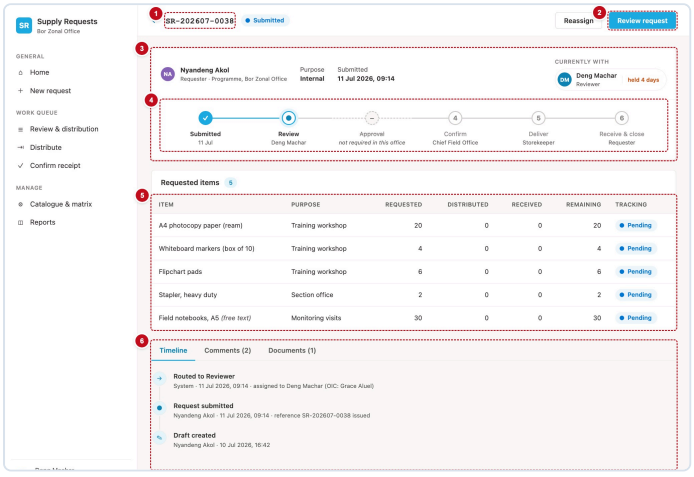
The app has nine screens. Eight match the eight HTML mock-ups in this repository. The ninth, the work queue, lists what is waiting for you at one stage.



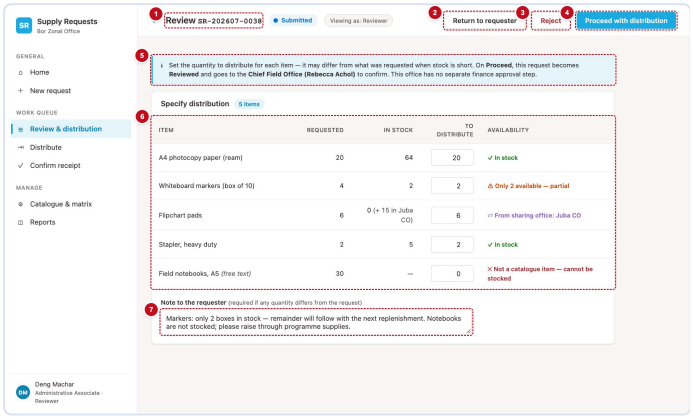
Home (scrHome)



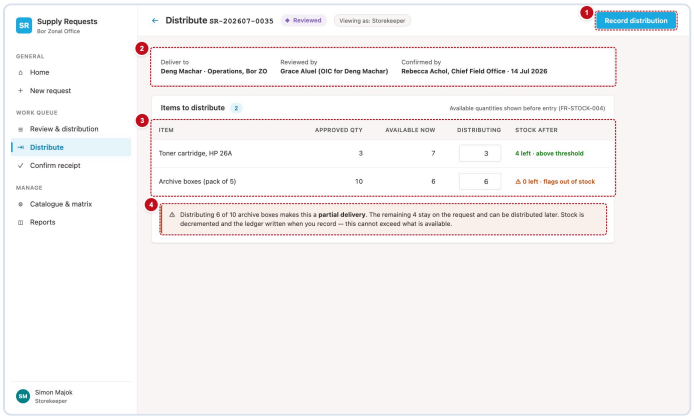
New request (scrNewRequest)



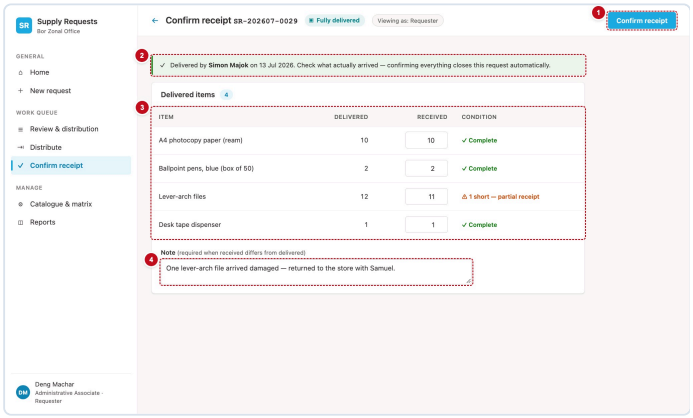
Request detail (scrRequestDetail)



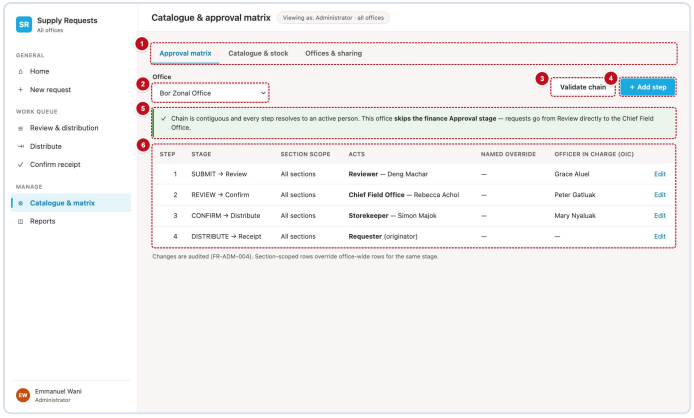
Review, approve, confirm (scrReview)



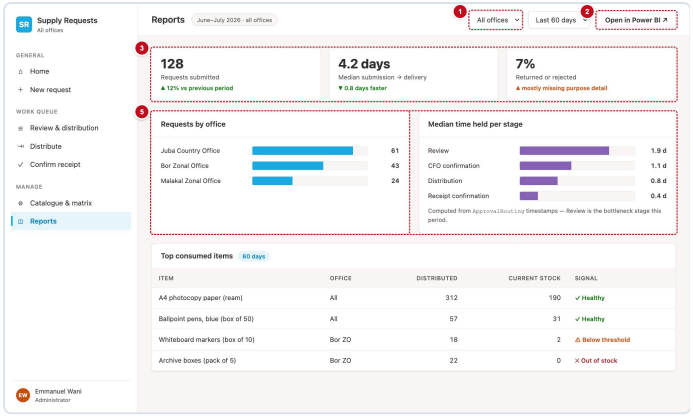
Distribute (scrDistribute)



Confirm receipt (scrReceipt)



Catalogue & matrix (scrAdmin)



Reports (scrReports)

Where everything comes from

You are not designing anything from scratch. Each part of the app already exists somewhere, and this guide tells you where.

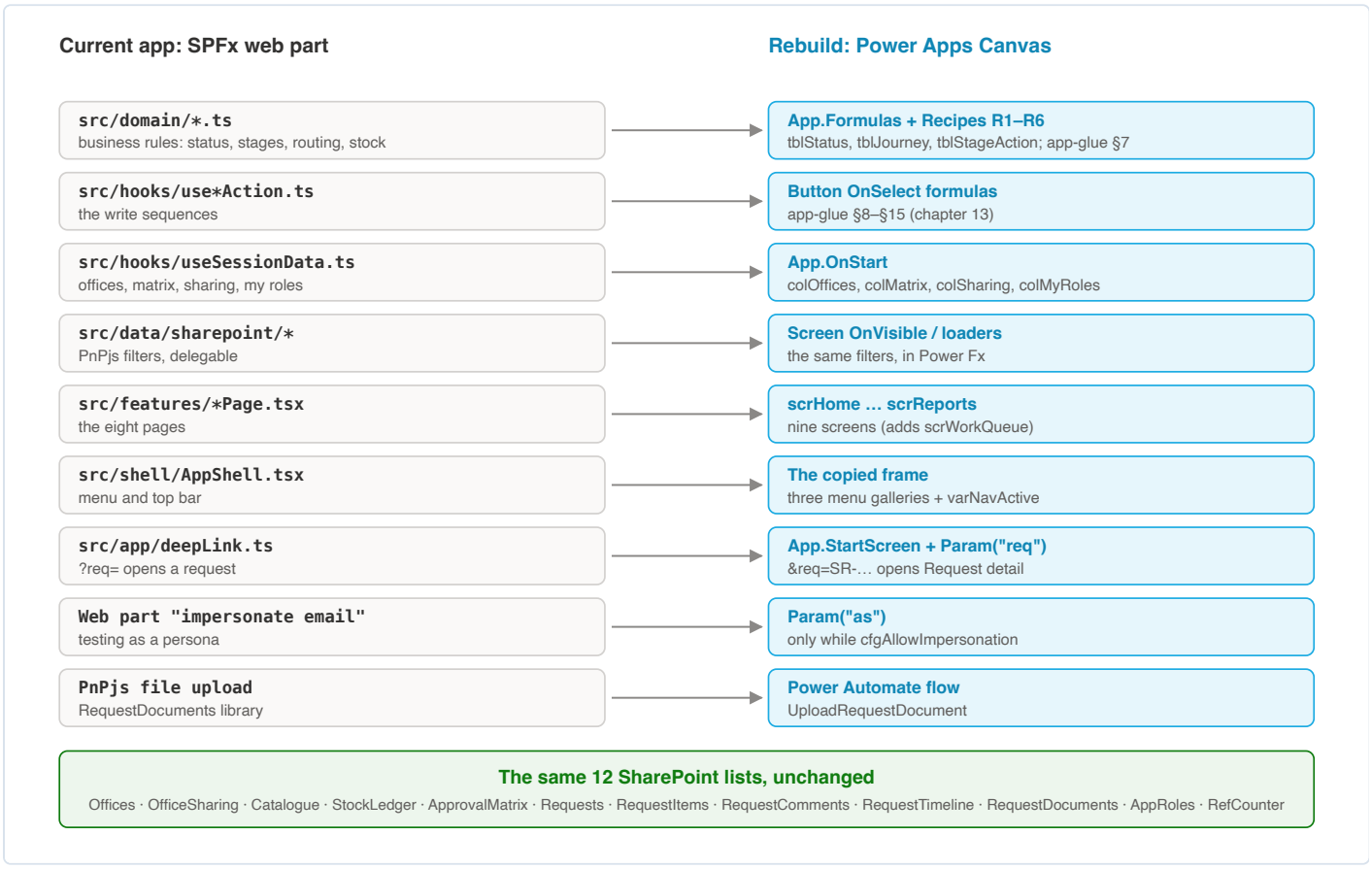


Figure 2. How the parts of the current app map onto the canvas app

You need	It is in	Notes
What each screen looks like	The HTML mock-ups at the root of this repository (`index.html`, `review.html`, …)	Open them in a browser next to Studio.
The lists and their columns	`docs/schema.md` (Appendix A)	The definitive contract. If anything disagrees with it, it wins.

You need	It is in	Notes
Colours, sizes, fonts, status table	`powerapps/App.Formulas.fx`	Paste into the app once (chapter 3).
Start-up data and the user's roles	`powerapps/App.OnStart.fx`	Paste into the app once (chapter 3).
Every button that writes to SharePoint	`powerapps/app-glue.md`, sections 7–15 (chapter 13)	Full Power Fx, ready to paste.
The Home screen, already built	`powerapps/screens/scrHome.pa.yaml`	Paste with Cmd+V, then apply the fixes in chapter 4.
What Studio accepts when you paste code	`powerapps/oracle/NOTES.md`	Read it before pasting any YAML.
How the current app behaves	`pa.init.officesupplies/src/`	Each formula in this guide names the file it was ported from.

How to use this guide

- **Work in order.** Chapters 2 and 3 set up the data and the app shell. Each screen chapter assumes the earlier ones are done.
- **Build every screen read-only first,** then wire up its buttons from chapter 13. A screen that shows the right data is much easier to check than one that also writes.
- **Steps are numbered.** Each starts with what to do. Studio click paths are written like this: **Tree view › scrHome › Insert › Gallery**.
- **Copy formulas from the files, not from the PDF.** A PDF wraps long lines and may change quote marks. Every formula in this guide is in a text file in `powerapps/`: `App.Formulas.fx`, `App.OnStart.fx`, `app-glue.md` §7–§15, and the chapter files in `powerapps/guide/` (`30-home.md`, `34-review.md`, …).
- **Formulas name their home.** `btnSubmit.OnSelect` means: select the control `btnSubmit`, choose `OnSelect` in the property list, and paste the formula into the formula bar.
- **Boxes tell you when to stop and check:**

> ✓ **You should now see**
>
> What the screen should show at this point. If it doesn't, fix it before you go on.

> ⚠ **Watch out**
>
> A mistake that is easy to make, usually one the first canvas build actually hit.

> **Defects this screen must get right**
>
> Behaviours that UAT round 2 checked (DEF numbers from the UAT defect log). Appendix B lists them all.

What is not in this guide

- **Paste-ready YAML for every screen.** Only Home has it. The other screens are described control by control, with every formula, and you build them in Studio.
- **Email or Teams notifications.** The current app sends none, so this guide adds none.
- **Dataverse.** The data stays in SharePoint.
- **Role gates on the Catalogue & matrix and Reports screens.** The current app leaves both open to every user (DEF-135, by design). Chapter 11 shows how to add a gate if the product owner wants one.

2. Before you start

What you need

Item	Detail
Licence	Each user needs a Microsoft 365 licence that includes Power Apps and Power Automate (most E3/E5 and Business plans do). The app uses only *standard* connectors (SharePoint, and Power Apps itself for the flow), so no Power Apps Premium licence is needed.
Environment	A Power Platform environment you can make apps in. The default environment works; a dedicated one is better for moving between test and production later (chapter 15).
Data loss prevention (DLP)	If your tenant has DLP policies, the SharePoint connector must be allowed in that environment. Ask your Power Platform admin if you're not sure.
The SharePoint site	`https://trailanalyticslimited602.sharepoint.com/sites/OfficeSupplies`, with its 12 lists already created. The scripts in `provision/` made them; `provision/README.md` says the site is already provisioned, so you don't need to run them again.
Site permissions	Every user needs **Contribute** on that site, because the app writes to the lists as the person using it. Administrators of the app need the same.
A browser	Microsoft Edge or Google Chrome, signed in to the tenant.
The mock-ups	The eight HTML files at the root of this repository. Open `index.html` in the browser to click through them.

The twelve lists

The app reads and writes these lists and the one library. You never create a column: every column already exists with the exact internal name the formulas use.

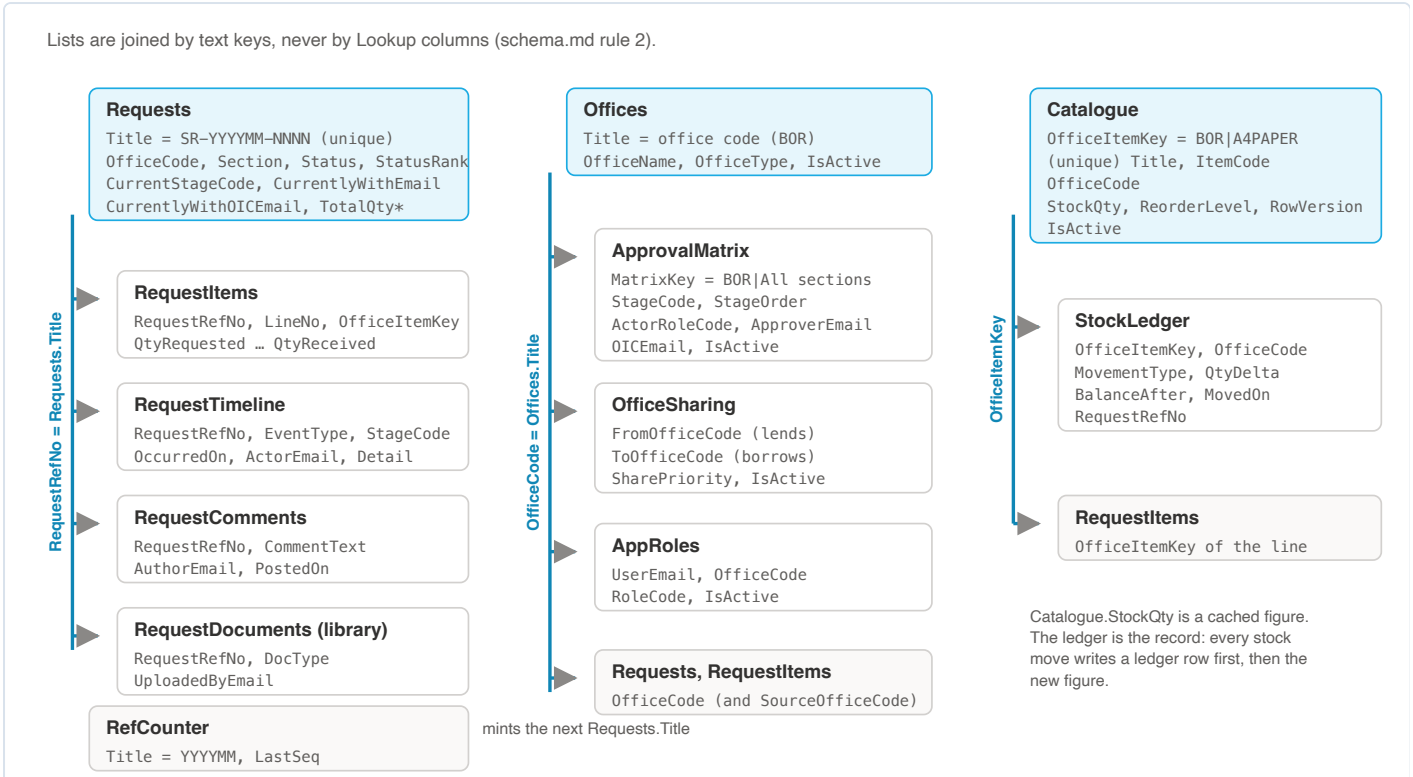


Figure 3. The twelve SharePoint lists and the text keys that join them. There are no Lookup columns.

> **No Lookup columns**
>
> Lists are joined by text keys: `RequestRefNo` (the request reference), `OfficeCode`, `OfficeItemKey` (`BOR|A4PAPER`) and `MatrixKey` (`BOR|All sections`). This keeps every filter fast on large lists. Appendix A lists every column.

Create the app

1. Go to **make.powerapps.com** and check the environment picker at the top right shows the right environment.
2. Select **Create › Blank app › Blank canvas app**.
3. Name it `Office Supplies`, choose **Tablet** format, and select **Create**. Tablet, because the mock-ups are a 264-pixel menu beside a content area up to 1,180 pixels wide.
4. In Studio, select **Settings › Display**. Turn **Scale to fit** off and **Lock aspect ratio** off, so the screen fills the browser window the way the mock-ups do.
5. Select **Settings › Updates** and check that **Formula-level error management** is on (in recent versions it is always on and not shown). The formulas rely on `IfError` and on `Blank()` clearing a column.
6. Select **File › Save**.

Connect the lists

1. In the left rail, select **Data › Add data**, search for **SharePoint**, and pick your SharePoint connection (create one if asked).
2. Enter the site URL above and select **Connect**.

3. Tick all twelve and select **Connect**: `Offices`, `OfficeSharing`, `Catalogue`, `StockLedger`, `ApprovalMatrix`, `Requests`, `RequestItems`, `RequestComments`, `RequestTimeline`, `RequestDocuments`, `AppRoles`, `RefCounter`.

> ✓ **You should now see**
>
> Twelve data sources in the Data pane. `RequestDocuments` is listed as a **document library**. Expand `Requests`: the columns show clean names such as `CurrentlyWithEmail` and `StatusRank`.

> ⚠ **Watch out**
>
> If any column shows as `Office_x0020_Code` or similar, that column was not created by the provisioning script. Recreate it with the right internal name before you build anything on it (schema.md, rule 1).

Set the data row limit to 1 for now

Power Apps fetches at most a set number of rows (the *data row limit*) whenever it cannot ask SharePoint to filter for it. Filtering that SharePoint does itself is called *delegation*. A formula that does not delegate does not fail: it quietly works on the first rows only and gives a wrong answer that looks right.

1. Select **Settings** › **General** › **Data row limit** and set it to **1**.

While the limit is 1, a formula that delegates still returns all its rows (SharePoint filtered them), and one that doesn't returns one row at most. That makes every problem obvious. You raise the limit to 2,000 in chapter 15, once every screen has passed.

> ⚠ **Look, don't click, while the limit is 1**
>
> With the limit at 1, any total the app adds up comes from a single row, and a write button would save that wrong total to SharePoint. Open screens and check the lists they show, but don't press Submit, Proceed, Record distribution or any other button that saves, until the limit is back at 2,000.

Check the Yes/No filter first

`app-glue.md` §3 records that the SharePoint REST API, queried by hand on this site, returned the *wrong* rows for `IsActive eq true`. The Power Apps connector builds its own query and is expected to get it right, but three of the app's start-up queries depend on it, so check it once:

1. Insert a temporary label on any screen and set its `Text` to:

```
CountRows(Filter(Catalogue, OfficeCode = "BOR", IsActive = true)) & " / " &
CountRows(Filter(ApprovalMatrix, IsActive = true))
```

2. Temporarily set the data row limit to 500 and look at the label.

> **✓ You should now see**
>
> `9 / 4` with the seeded data: nine active Bor items and four matrix rows. Delete the label and set the limit back to 1.

If you see `1 / …` or `0 / …`, the connector has the same defect. Replace `IsActive = true` with `IsActive <> false` everywhere this guide uses it, and tell whoever owns the SharePoint site.

3. The app shell

The shell is everything that isn't one screen's content: the named colours and tables, the start-up data, the left menu, the top bar, and the screen names. You build it once.

Paste App.Formulas

`App.Formulas` holds *named formulas*: values the whole app can use by name, like `clrBrand` for the brand colour or `tblStatus` for the status table. Every colour, size and status lookup in this guide uses these names, never a raw value.

1. In the **Tree view**, select **App**.
2. In the property list at the top left, choose **Formulas**.
3. Open `powerapps/App.Formulas.fx`, copy all of it, and paste it into the formula bar.

> **✓ You should now see**
>
> No red underline in the formula bar. Type `clrBrand` into any label's `Color` and the text turns cyan.

The file has two parts. The first, from the original canvas plan, holds the design tokens from `styles.css` and the status and journey tables. The second, added for this rebuild, is shown below. It holds the stage-to-screen table, the choice lists, the timeline glyphs, the Reports stage table and a few numbers.

```
// == Added for the full rebuild (Canvas Rebuild Guide, chapter 3) ============
// Everything below is a constant, like everything above. Formulas that read a
// collection or a variable live in App.OnStart or on the screens instead.

// What each stage's work looks like: the screen that does it and its button
// labels. Ported from pa.init.officesupplies src/domain/stageAction.ts.
// DEF-03 / DEF-120: a row button names the stage it acts on, not always "Review".
tblStageAction =
Table(
    { StageCode: "Review",   Screen: "review",     DetailLabel: "Review request",  RowLabel: "Review",
ScreenTitle: "Review"  },
    { StageCode: "Approval", Screen: "review",     DetailLabel: "Approve request", RowLabel: "Approve",
ScreenTitle: "Approve" },
    { StageCode: "Confirm",  Screen: "review",     DetailLabel: "Confirm request", RowLabel: "Confirm",
ScreenTitle: "Confirm" },
    { StageCode: "Deliver",  Screen: "distribute", DetailLabel: "Distribute",      RowLabel: "Distribute",
ScreenTitle: "Distribute" },
    { StageCode: "Receive",  Screen: "receipt",    DetailLabel: "Confirm receipt", RowLabel: "Confirm receipt",
ScreenTitle: "Confirm receipt" }
);

// Choice values, verbatim from docs/schema.md. Single-column tables (column "Value").
tblSections    = ["Programme", "Child Protection", "Health & Nutrition", "Operations", "Supply & Logistics"];
tblPurposes    = ["Internal", "External"];
tblRoles       = ["Requester", "Reviewer", "Chief Field Office", "Storekeeper", "Administrator"];
tblOfficeTypes = ["Headquarters", "Zonal office", "Field office"];
tblDocTypes    = ["Supporting document", "Request PDF", "Delivery note", "Receipt note"];
tblConditions  = ["Complete", "Short", "Damaged", "Not received"];

// Timeline glyph per RequestTimeline.EventType (schema.md §9).
tblEventGlyph =
```

```
Table(
    { EventType: "Draft created",  Glyph: "✎" },
    { EventType: "Submitted",      Glyph: "●" },
    { EventType: "Routed",         Glyph: "→" },
    { EventType: "Reviewed",       Glyph: "◆" },
    { EventType: "Returned",       Glyph: "↩" },
    { EventType: "Rejected",       Glyph: "×" },
    { EventType: "Confirmed",      Glyph: "✓" },
    { EventType: "Distributed",    Glyph: "▣" },
    { EventType: "Received",       Glyph: "✓" },
    { EventType: "Closed",         Glyph: "■" },
    { EventType: "Comment added",  Glyph: "💬" },
    { EventType: "Reassigned",     Glyph: "⇄" },
    { EventType: "Stock adjusted", Glyph: "±" }
);

// Reports: the events that END each stage, and the stage's chart label.
// Ported from src/domain/timeline.ts (STAGE_EXIT_EVENTS, STAGE_CHART_LABELS).
// Exits is a "|"-delimited list so a test reads: "|" & ev & "|" in Exits.
tblStageExit =
Table(
    { Leg: 2, StageCode: "Review",   ChartLabel: "Review",              Exits: "|Reviewed|Returned|Rejected|"
},
    { Leg: 3, StageCode: "Approval", ChartLabel: "Approval",            Exits: "|Confirmed|Returned|Rejected|"
},
    { Leg: 4, StageCode: "Confirm",  ChartLabel: "CFO confirmation",    Exits: "|Confirmed|Returned|Rejected|"
},
    { Leg: 5, StageCode: "Deliver",  ChartLabel: "Distribution",        Exits: "|Distributed|" },
    { Leg: 6, StageCode: "Receive",  ChartLabel: "Receipt confirmation", Exits: "|Received|Closed|" }
);

numOverdueHours = 72;    // "held longer than 3 days" on Home (HomePage.tsx OVERDUE_DAYS = 3)
numReportCap    = 500;   // Reports read at most this many rows per list (ReportsPage.tsx)
numMintAttempts = 5;     // reference minting retries (src/domain/reference.ts MAX_MINT_ATTEMPTS)
txtNoHolder     = "—";   // DEF-111: nobody holds the request
txtSystemActor  = "System";

// Testing switch. true lets the app run as a seeded persona via the "as" link
// parameter (App.OnStart). The seeded people have no real accounts, so UAT needs
// it. SET IT TO false BEFORE YOU PUBLISH THE APP TO REAL USERS.
cfgAllowImpersonation = true;
```

> ⚠ **Watch out**
>
> `cfgAllowImpersonation = true` lets anyone open the app as someone else by adding `&as=<email>` to the link. UAT needs it, because the seeded people have no real accounts. Set it to `false` before real users get the app (chapter 15).

Paste App.OnStart

`App.OnStart` runs once when the app opens. It loads the small reference lists, works out who the user is and which roles they hold, and sets the working variables.

1. Select **App**, choose **OnStart**, and paste all of `powerapps/App.OnStart.fx`.
2. Right-click **App** in the Tree view and select **Run OnStart**.

```
// ── 1. Reference data (small lists, one round trip each) ──────────────

// All offices, including inactive ones — scrAdmin tab 3 renders the full table
// with an Active column, so it must not be pre-filtered. Pickers filter it
// locally (DEF-14: an inactive office is hidden from pickers, still listed here).
```

```
ClearCollect(colOffices, Offices);

// The approval matrix. IsActive = true on a Yes/No column IS delegable to
// SharePoint. Small list (max 5 rows per office+section).
ClearCollect(colMatrix, Filter(ApprovalMatrix, IsActive = true));

// Office sharing links — needed by the Review screen's ceiling calculation.
ClearCollect(colSharing, Filter(OfficeSharing, IsActive = true));

// ── 2. Who is using the app ──────────────────────────────────────────
// Emails are compared lower-case everywhere (DEF-09: "Email casing is compared
// exactly"). SharePoint's own filters ignore case, but a comparison the app
// makes itself, such as Lower(varReq.CurrentlyWithEmail) = varUserEmail, does not.
Set(varRealEmail, Lower(User().Email));
Set(varUserName,  User().FullName);

// Testing only: open the app with &as=<email> to behave as a seeded persona,
// the canvas equivalent of the SPFx "impersonate email" setting. Ignored unless
// cfgAllowImpersonation (App.Formulas) is true.
Set(
    varUserEmail,
    If(
        cfgAllowImpersonation && !IsBlank(Param("as")),
        Lower(Trim(Param("as"))),
        varRealEmail
    )
);
Set(varImpersonating, varUserEmail <> varRealEmail);

// ── 3. The single role query ─────────────────────────────────────────
// ONE delegable query (UserEmail is indexed text, IsActive is Yes/No). Every
// role check after this is local. Never re-query AppRoles per screen.
ClearCollect(
    colMyRoles,
    Filter(AppRoles, UserEmail = varUserEmail, IsActive = true)
);

// Default office = the office on the user's Requester role if they have one,
// else whatever their first role names, else blank. Coalesce over blank text.
Set(
    varMyOffice,
    Coalesce(
        LookUp(colMyRoles, RoleCode.Value = "Requester").OfficeCode,
        First(colMyRoles).OfficeCode,
        ""
    )
);

// varOffice is the office the screens are CURRENTLY looking at. It starts at the
// user's own office; scrAdmin and scrReports let the user repoint it.
Set(varOffice, varMyOffice);

// ── 4. Role booleans ─────────────────────────────────────────────────
// Derived from colMyRoles, so they are variables, not named formulas. If
// AppRoles changes mid-session these are stale until the app is reopened.
Set(varIsRequester,   !IsEmpty(Filter(colMyRoles, RoleCode.Value = "Requester")));
Set(varIsReviewer,    !IsEmpty(Filter(colMyRoles, RoleCode.Value = "Reviewer")));
Set(varIsCFO,         !IsEmpty(Filter(colMyRoles, RoleCode.Value = "Chief Field Office")));
Set(varIsStorekeeper, !IsEmpty(Filter(colMyRoles, RoleCode.Value = "Storekeeper")));
Set(varIsAdmin,       !IsEmpty(Filter(colMyRoles, RoleCode.Value = "Administrator")));

// The persona line ("Viewing as: Reviewer · Bor Zonal Office"): the highest
// role wins, in the order HomePage.tsx uses.
Set(
    varPersonaRole,
    If(
        varIsAdmin,       "Administrator",
        varIsCFO,         "Chief Field Office",
        varIsReviewer,    "Reviewer",
        varIsStorekeeper, "Storekeeper",
        varIsRequester,   "Requester",
```

```
        "No role"
    )
);
Set(
    varMyOfficeName,
    Coalesce(LookUp(colOffices, Title = varMyOffice).OfficeName, varMyOffice, "No default office")
);

// ── 5. Working state ──────────────────────────────────────────────
Set(varBusy, false);           // button guard: every multi-write sets it first
Set(varErrorMsg, "");          // last user-facing error, "" when clean

// The request currently open. A link with &req=SR-202607-0038 opens it
// directly (App.StartScreen sends the user to scrRequestDetail); every screen
// that shows a request reads it again with LookUp(Requests, Title = varRef).
Set(varRef, Coalesce(Param("req"), ""));
Set(varEditRef, "");           // scrNewRequest: "" = new request, else the ref being edited
Set(varQueue, "review");       // scrWorkQueue: "review" | "distribute" | "receipt"
```

✓ **You should now see**

Select **Variables** (the *(x)* icon in the left rail). `varUserEmail` is your email in lower case, and `colMyRoles` holds your rows from `AppRoles`. If `colMyRoles` is empty, your account has no role yet: add yourself to `AppRoles` in SharePoint (a row with your email, an office code, a role, Active = Yes), or test as a seeded person with the `as` link parameter.

Create the screens

Screen names matter: formulas use them in `Navigate(scrReview)`, so `Screen2` won't do.

1. Select **Insert** › **New screen** › **Blank** nine times, renaming each one in the Tree view straight away:

#	Screen	Mock-up	Chapter
1	`scrHome`	`index.html`	4
2	`scrWorkQueue`	(none: a list like Home's "Awaiting my action")	5
3	`scrNewRequest`	`new-request.html`	6
4	`scrRequestDetail`	`request-detail.html`	7
5	`scrReview`	`review.html`	8
6	`scrDistribute`	`distribute.html`	9
7	`scrReceipt`	`receipt.html`	10
8	`scrAdmin`	`admin.html`	11
9	`scrReports`	`reports.html`	12

1. Delete the original `Screen1`. If Home will be pasted from YAML (chapter 4), don't create `scrHome` here: pasting creates it.
2. Set every screen's `Fill` to `clrCanvas`.

Set the start screen

A link with `&req=SR-202607-0038` should open that request directly (the canvas equivalent of the current app's `?req=` link).

1. Select **App**, choose **StartScreen**, and enter:

```
If(IsBlank(Param("req")), scrHome, scrRequestDetail)
```

`App.OnStart` has already put the reference into `varRef`, so Request detail opens on it.

> ⚠ **Watch out**
>
> This relies on `App.OnStart` finishing before the first screen loads. Leave the setting **Non-blocking OnStart** (under **Settings › Updates**) turned **off**. Don't call `Navigate` in `OnStart`; `StartScreen` replaces it.

The frame every screen shares

Every screen has the same frame: a 264-pixel menu on the left, a 56-pixel top bar, and the content beneath, at most 1,180 pixels wide.

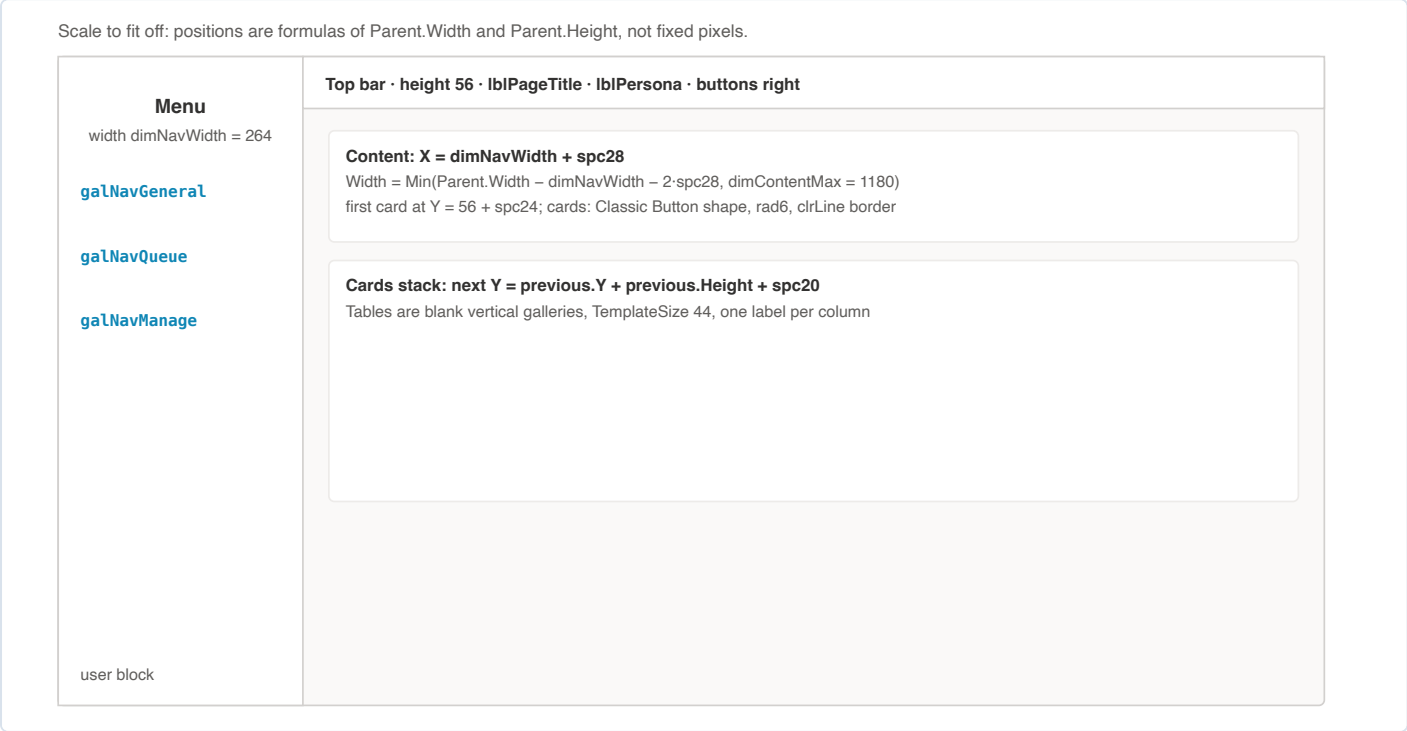


Figure 4. The frame every screen shares, with the named sizes from App.Formulas

Part	Controls	Key properties
Menu background	`recNavBg` (Rectangle)	`X: 0` `Y: 0` `Width: =dimNavWidth` `Height: =Parent.Height` `Fill: =clrSurface`
Menu edge	`recNavEdge` (Rectangle)	`X: =dimNavWidth – 1` `Width: 1` `Height: =Parent.Height` `Fill: =clrLine`

Part	Controls	Key properties
Brand	`recBrandMark` (Classic Button as a shape), `lblBrandMark` "SR", `lblBrandName` "Supply Requests", `lblBrandOffice`	`lblBrandOffice.Text: =varMyOfficeName` (DEF-116, DEF-131)
Menu items	`galNavGeneral`, `galNavQueue`, `galNavManage` (blank vertical galleries) with a group label above each	Below
User block	`recAvatar` shape, `lblAvatar`, `lblUserName`, `lblUserRole`	`lblUserName.Text: =varUserName` `lblUserRole.Text: =varPersonaRole` `lblAvatar.Text: =Upper(Left(varUserName, 1) & Mid(varUserName, Find(" ", varUserName & " ") + 1, 1))`
Top bar	`recTopBg` (Rectangle), `recTopEdge` (1-pixel Rectangle), `lblPageTitle`, `lblPersona`, the screen's buttons on the right	`recTopBg`: `X: =dimNavWidth` `Height: 56` `Width: =Parent.Width - dimNavWidth` `Fill: =clrSurface`
Content	Everything else	`X: =dimNavWidth + spc28` `Width: =Min(Parent.Width - dimNavWidth - 2 * spc28, dimContentMax)` and the first card at `Y: =56 + spc24`

> ⚠ **Watch out**
>
> - Set `Font: =fntUI` on **every** label and button. Studio gives new controls Open Sans, which looks visibly different.
> - A **Rectangle has no rounded corners** in this tenant. For anything rounded (cards, status pills, the avatar) insert a **Classic Button**, set `Text: =""`, set `HoverFill`, `PressedFill` to `=Self.Fill` and `HoverBorderColor`, `PressedBorderColor` to `=Self.BorderColor`, then set the four `Radius…` properties. Inside a gallery give it `OnSelect: =Select(Parent)`; elsewhere `OnSelect: =false`.

The menu

The menu is three small galleries. The highlighted item follows the variable `varNavActive`, which every screen sets in its `OnVisible`. Because nothing in the menu names its own screen, the same menu can be copied onto every screen unchanged.

1. On `scrHome`, set `galNavGeneral.Items` to:

```
Table(
    { Glyph: "⌂", Caption: "Home",        Target: "home" },
    { Glyph: "+", Caption: "New request", Target: "new" }
)
```

`galNavQueue.Items`:

```
Table(
    { Glyph: "≡", Caption: "Review & distribution", Target: "review" },
    { Glyph: "⇥", Caption: "Distribute",            Target: "distribute" },
    { Glyph: "✓", Caption: "Confirm receipt",       Target: "receipt" }
)
```

```
)
```

`galNavManage.Items`:

```
Table(
    { Glyph: "⚙", Caption: "Catalogue & matrix", Target: "admin" },
    { Glyph: "▥", Caption: "Reports",            Target: "reports" }
)
```

2. In each gallery's template, the active bar, glyph and caption use `ThisItem.Target = varNavActive`:

```
// recNavActive.Visible   (the 3-pixel cyan bar at the left of the item)
ThisItem.Target = varNavActive
// lblNavGlyph.Color
If(ThisItem.Target = varNavActive, clrBrandDark, clrInk2)
// lblNavCaption.Color and .FontWeight
If(ThisItem.Target = varNavActive, clrBrandDark, clrInk)
If(ThisItem.Target = varNavActive, FontWeight.Semibold, FontWeight.Normal)
// the item background (a Classic Button shape): .Fill
If(ThisItem.Target = varNavActive, clrBrandTint, Color.Transparent)
```

3. Set the `OnSelect` of **each of the three galleries** (the gallery itself, not a control inside it) to the same formula:

```
Switch(
    ThisItem.Target,
    "home",       Navigate(scrHome),
    "new",        Set(varEditRef, ""); Navigate(scrNewRequest),
    "review",     Set(varQueue, "review");     Navigate(scrWorkQueue),
    "distribute", Set(varQueue, "distribute"); Navigate(scrWorkQueue),
    "receipt",    Set(varQueue, "receipt");    Navigate(scrWorkQueue),
    "admin",      Navigate(scrAdmin),
    "reports",    Navigate(scrReports)
)
```

   The three work-queue items open the queue for that stage, not Home (DEF-10).

4. Give every control inside the gallery templates `OnSelect: =Select(Parent)`, so a click anywhere on an item selects it.

5. When the frame is finished on `scrHome`, select all its frame controls in the Tree view (Shift-click), copy (Cmd+C or Ctrl+C), select each other screen, and paste. Studio adds `_1` to the copies' names, which doesn't matter: no formula refers to them by name.

> ✓ **You should now see**
>
> In Preview (F5), clicking **Reports** opens `scrReports` and its menu item turns cyan. Clicking **Distribute** opens the work queue.

> **Why not a component?**
>
> A canvas *component* would let you build the frame once and change it in one place. Navigating from inside a component needs component *behaviour properties*, which may not be available in your tenant, so this guide copies the frame instead. You can switch to a component later without changing anything else.

The top bar on each screen

Each screen sets its own title and persona line in its `OnVisible` and binds them:

```
// lblPageTitle.Text
varPageTitle
// lblPersona.Text — "Viewing as: Reviewer · Bor Zonal Office"
"Viewing as: " & varPersonaRole & " · " & varMyOfficeName
    & If(varImpersonating, "  (testing as " & varUserEmail & ")", "")
```

A screen whose persona line differs (Review shows the stage's actor, Reports shows the period) says so in its chapter.

Pieces every screen reuses

These patterns appear on several screens. Each screen chapter gives the full formula where it's used; this is what they mean.

Pattern	Where it is	What it does
Status pill	Every list of requests	A Classic Button shape with `Fill: =Coalesce(LookUp(tblStatus, StatusText = ThisItem.Status.Value).Bg, clrStDraftBg)`, a label with `Color: =Coalesce(LookUp(tblStatus, StatusText = ThisItem.Status.Value).Fg, clrInk2)` and `Text: =LookUp(tblStatus, StatusText = ThisItem.Status.Value).Glyph & " " & ThisItem.Status.Value`
Reference	Lists and headers	`Font: =fntMono`, `Size: =szLabel`, `Wrap: =false` so `SR-202607-0038` never breaks at its hyphens (DEF-109, DEF-114)
Recipe R1–R6	`app-glue.md` §7	The approval chain, routing, who may act, names from emails, "held for", and the request loader
Busy guard	Every button that writes	`DisplayMode: =If(varBusy, DisplayMode.Disabled, DisplayMode.Edit)` plus the screen's own rule
Error line	Under each form	A label with `Text: =varErrorMsg`, `Color: =clrStRejected`, `Visible: =!IsBlank(varErrorMsg)`. Clear it with `Set(varErrorMsg, "")` in `OnVisible`
Empty message	Under each list	A label with `Visible: =IsEmpty(<gallery>.AllItems)` and the message from the screen chapter

4. Home — scrHome

Home is the first screen everyone sees. It shows what is waiting on the user, the user's own requests, and how many items are low on stock in their office.

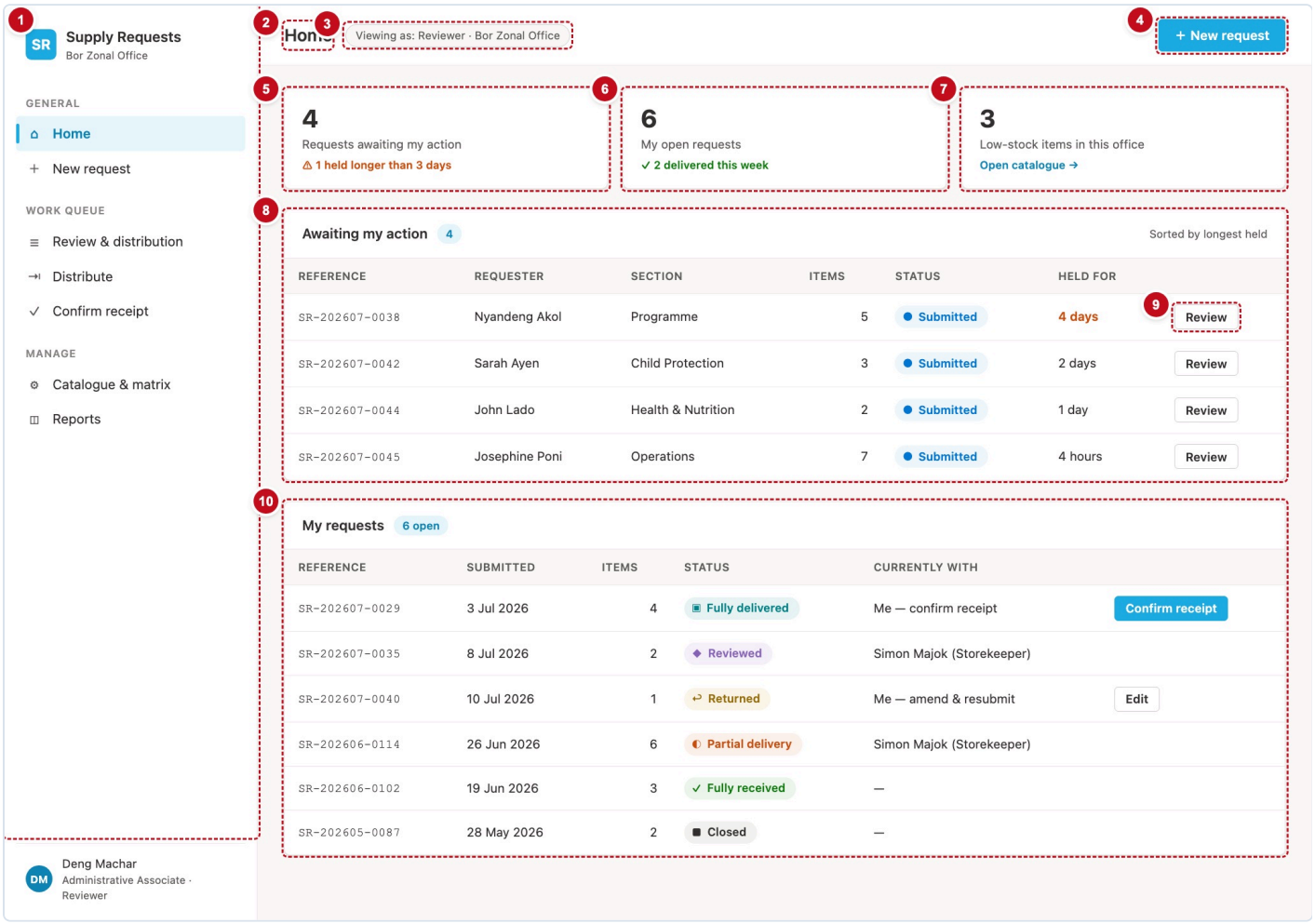


Figure 5. Home, as the mock-up shows it (index.html). The numbers match the table below.

#	Part	Controls	What it shows
1	Menu	the frame (chapter 3)	`varNavActive = "home"`
2	Title	`lblPageTitle`	"Home"
3	Persona	`lblPersona`	"Viewing as: Reviewer · Bor Zonal Office"
4	New request	`btnNewRequest`	Opens a blank New request
5	Tile: awaiting me	`recTile1`, `lblTile1Num`, `lblTile1Cap`, `lblTile1Flag`	How many requests wait on me, and how many have waited more than 3 days
6	Tile: my open requests	`recTile2`, `lblTile2Num`, `lblTile2Cap`	My requests that are not closed or rejected
7	Tile: low stock	`recTile3`, `lblTile3Num`, `lblTile3Cap`, `btnTile3Link`	Items in my office at or below their reorder level, or out of stock
8	Awaiting my action	`recCard1`, header labels, `galAwaiting`	The requests I hold, longest-held first

#	Part	Controls	What it shows
9	Row action	`btnAwaitingAction` (in the gallery)	Review / Approve / Confirm / Distribute / Confirm receipt / Edit
10	My requests	`recCard2`, header labels, `galMine`	Everything I have raised, newest first

Two ways to build it

Paste it. `powerapps/screens/scrHome.pa.yaml` is the Home screen copied out of Studio after a successful paste (84 controls). Paste it, then make the changes in Fix the pasted screen below.

1. On macOS, in a terminal at the repository root, run `pbcopy < powerapps/screens/scrHome.pa.yaml`. On Windows, open the file in a text editor and copy all of it.
2. In Studio, select any screen in the Tree view and press **Cmd+V** (Ctrl+V). Don't use the right-click **Paste** item; it does nothing with text from outside Studio.
3. Studio creates a screen called `scrHome`, or `scrHome_1` if one already exists. Delete the extra screen and rename if needed.

> ⚠ **Watch out**
>
> If the paste is rejected, Studio lists every error with its line number. The usual causes are in `powerapps/oracle/NOTES.md`: a control type without its `Classic/` prefix, a version number Studio doesn't know, or a `Radius…` property on a Rectangle.

Or build it by hand with the control list above and the formulas below, following the frame in chapter 3.

Load the data

`scrHome.OnVisible`:

```
Set(varNavActive, "home");
Set(varPageTitle, "Home");
Set(varErrorMsg, "");

// Awaiting my action: open, and held by me or by me as the officer in charge.
// The delegation contract, verbatim (schema.md). Oldest submission first.
ClearCollect(
    colAwaiting,
    SortByColumns(
        Filter(
            Requests,
            StatusRank < 70,
            Or(CurrentlyWithEmail = varUserEmail, CurrentlyWithOICEmail = varUserEmail)
        ),
        "SubmittedOn", SortOrder.Ascending
    )
);

// My requests, newest first.
ClearCollect(
    colMine,
    SortByColumns(Filter(Requests, RequesterEmail = varUserEmail), "SubmittedOn", SortOrder.Descending)
);
```

```
// My office's active catalogue, for the low-stock tile.
ClearCollect(colHomeCatalogue, Filter(Catalogue, OfficeCode = varMyOffice, IsActive = true))
```

The tiles

```
// lblTile1Num.Text
CountRows(colAwaiting)

// lblTile1Flag.Text and .Visible — measured from the last change, not from submission (DEF-132)
With(
    { n: CountRows(Filter(colAwaiting,
            DateDiff(Coalesce(Modified, SubmittedOn), Now(), TimeUnit.Hours) > numOverdueHours)) },
    "⚠ " & n & " held longer than 3 days"
)
CountRows(Filter(colAwaiting, DateDiff(Coalesce(Modified, SubmittedOn), Now(), TimeUnit.Hours) >
numOverdueHours)) > 0
// lblTile1Flag.Color
clrStPartial

// lblTile2Num.Text
CountRows(Filter(colMine, StatusRank < 70))

// lblTile3Num.Text — compared here, in the app: SharePoint cannot compare two columns
CountRows(Filter(colHomeCatalogue, StockQty = 0 || (!IsBlank(ReorderLevel) && StockQty <= ReorderLevel)))
// btnTile3Link: Text "Open catalogue →", OnSelect
Navigate(scrAdmin)
```

> ⚠ **Watch out**
>
> The pasted `scrHome` counted low stock with `Filter(Catalogue, …, StockQty <= ReorderLevel)` straight against SharePoint. That compares two columns, which SharePoint cannot do, and it fails with "Fetching items failed". Use the formula above, which compares them in the app after a filter SharePoint *can* do.

Awaiting my action

`galAwaiting.Items: =colAwaiting`. Columns, left to right, are labels at fixed `X` positions in the template (`TemplateSize: 44`):

Column	Formula
Reference	`ThisItem.Title` (`Font: =fntMono`)
Requester	Recipe R4 with E = `ThisItem.RequesterEmail`
Section	`ThisItem.Section.Value`
Items	`ThisItem.LineCount` (right-aligned)
Status	the status pill (chapter 3)
Held for	Recipe R5 with Q = `ThisItem`; `Color: =If(DateDiff(Coalesce(ThisItem.Modified, ThisItem.SubmittedOn), Now(), TimeUnit.Hours) > numOverdueHours, clrStPartial, clrInk)`
Action	`btnAwaitingAction`, below

Recipe R4, a name from an email, written out for the Requester column:

```
With({ E: ThisItem.RequesterEmail },
    If(IsBlank(E), txtNoHolder,
       IsBlank(Find("@", E)), E,
       Proper(Substitute(Substitute(Left(E, Find("@", E) - 1), ".", " "), "_", " "))))
```

Recipe R5, "held for", written out:

```
With(
    { h: DateDiff(Coalesce(ThisItem.Modified, ThisItem.SubmittedOn), Now(), TimeUnit.Hours) },
    If(h < 1,  "under an hour",
       h < 24, Text(h) & If(h = 1, " hour", " hours"),
       With({ d: RoundDown(h / 24, 0) }, Text(d) & If(d = 1, " day", " days")))
)
```

The row button names the stage it opens, and a Draft or Returned request offers **Edit** instead:

```
// btnAwaitingAction.Text
If(ThisItem.Status.Value in ["Draft", "Returned"], "Edit",
   Coalesce(LookUp(tblStageAction, StageCode = ThisItem.CurrentStageCode.Value).RowLabel, "Open"))

// btnAwaitingAction.OnSelect   (NOT Select(Parent): the button must not also open the row, DEF-134)
Set(varRef, ThisItem.Title);
If(
    ThisItem.Status.Value in ["Draft", "Returned"],
    Set(varEditRef, ThisItem.Title); Navigate(scrNewRequest),
    Switch(
        LookUp(tblStageAction, StageCode = ThisItem.CurrentStageCode.Value).Screen,
        "review",     Navigate(scrReview),
        "distribute", Navigate(scrDistribute),
        "receipt",    Navigate(scrReceipt),
        Navigate(scrRequestDetail)
    )
)

// galAwaiting.OnSelect — clicking anywhere else on the row opens the request
Set(varRef, ThisItem.Title); Navigate(scrRequestDetail)
```

Empty message: "Nothing is waiting on you right now."

My requests

`galMine.Items: =colMine`.

Column	Formula
Reference	`ThisItem.Title`
Submitted	`If(IsBlank(ThisItem.SubmittedOn), "Draft", Text(ThisItem.SubmittedOn, "d mmm yyyy"))`
Items	`ThisItem.LineCount`
Status	the status pill
Currently with	below
Action	`btnMineAction`, below

```
// lblMineWith.Text — the holder's name, or "—" when it is finished or nobody holds it (DEF-111).
// The current app shows just the name; the mock-up's "Me — confirm receipt" wording is not reproduced.
If(
    ThisItem.StatusRank >= 70 || IsBlank(ThisItem.CurrentlyWithEmail), txtNoHolder,
```

```
    With({ E: ThisItem.CurrentlyWithEmail },
        Proper(Substitute(Substitute(Left(E, Find("@", E & "@") - 1), ".", " "), "_", " ")))
)

// btnMineAction.Visible
ThisItem.Status.Value in ["Draft", "Returned"] || ThisItem.StatusRank = 50
// btnMineAction.Text
If(ThisItem.StatusRank = 50, "Confirm receipt", "Edit")
// btnMineAction.Fill (primary style for Confirm receipt, neutral for Edit)
If(ThisItem.StatusRank = 50, clrBrand, clrSurface)
// btnMineAction.OnSelect
Set(varRef, ThisItem.Title);
If(ThisItem.StatusRank = 50, Navigate(scrReceipt), Set(varEditRef, ThisItem.Title); Navigate(scrNewRequest))

// galMine.OnSelect
Set(varRef, ThisItem.Title); Navigate(scrRequestDetail)
```

Empty message: "You have not submitted any requests yet."

Fix the pasted screen

If you pasted `scrHome.pa.yaml`, change these. Each is a defect found on the pasted screen or in UAT.

1. **Menu.** Replace the three menu galleries' `Items`, highlight and `OnSelect` with the ones in chapter 3 (the pasted ones have targets but no `OnSelect`).
2. **Low-stock tile.** Replace `lblTile3Num.Text` with the formula above.
3. **Requester column.** It shows a raw email; use Recipe R4 as above.
4. **Row buttons.** Replace the pasted "Review" button formulas with `btnAwaitingAction` above (DEF-03, DEF-120, DEF-124, DEF-134), and the "My requests" action with `btnMineAction`.
5. **Held for.** Use Recipe R5: the pasted screen counts from `SubmittedOn` (DEF-132).
6. **Persona and brand line.** Bind `lblPersona` as in chapter 3 and `lblBrandOffice.Text: =varMyOfficeName`.
7. **Tile 2 flag.** Delete the "delivered this week" flag; the current app doesn't show it.
8. **OnVisible.** Replace with the one above (it adds `varNavActive` and the catalogue for tile 3).

> ✓ **You should now see**
>
> Open the app as the seeded reviewer that UAT-040 uses, by adding `&as=<their email from AppRoles>` to the app link. Tile 1 shows that person's awaiting count, each row's button names its stage, and clicking a row (not the button) opens the request.

> **Defects this screen must get right**
>
> DEF-03 and DEF-120 (the button names the stage) · DEF-09 (email case) · DEF-102 (by design: my own Returned and Fully delivered requests count as awaiting me, because I hold them) · DEF-111 ("—" when nobody holds it) · DEF-116 (a user with no office sees "No default office") · DEF-124 (Draft and Returned offer Edit) · DEF-132 (held-for from the last change) · DEF-134 (the button doesn't also open the row)

5. Work queue — scrWorkQueue

The three work-queue items in the menu (**Review & distribution**, **Distribute**, **Confirm receipt**) open this one screen. It lists the requests waiting on the user at that stage. The mock-ups have no page for it: it looks like Home's "Awaiting my action" card on its own (marker 8 on the Home mock-up in chapter 4), with the title of the queue. The current app added it to fix DEF-10, where those three menu items went back to Home.

`varQueue` says which queue to show. The menu sets it (chapter 3) before it navigates here.

varQueue	Title	Stages it lists	Empty message
`review`	Review & distribution	Review, Approval, Confirm	Nothing is waiting for you to review, approve or confirm.
`distribute`	Distribute	Deliver	Nothing is waiting for you to distribute.
`receipt`	Confirm receipt	Receive	Nothing is waiting for you to confirm receipt of.

Build it

1. Paste the frame (chapter 3) onto `scrWorkQueue`.
2. Copy `recCard1`, its header labels and `galAwaiting` from `scrHome`, paste them here, and rename the gallery `galQueue`.
3. Set the formulas below.

```
// scrWorkQueue.OnVisible
Set(varNavActive, varQueue);
Set(varPageTitle, Switch(varQueue, "review", "Review & distribution", "distribute", "Distribute", "Confirm
receipt"));
ClearCollect(
    colAwaiting,
    SortByColumns(
        Filter(Requests, StatusRank < 70,
               Or(CurrentlyWithEmail = varUserEmail, CurrentlyWithOICEmail = varUserEmail)),
        "SubmittedOn", SortOrder.Ascending
    )
)

// galQueue.Items — this queue's stages only, and never a Draft or Returned request (DEF-128)
Filter(
    colAwaiting,
    LookUp(tblStageAction, StageCode = CurrentStageCode.Value).Screen = varQueue,
    !(Status.Value in ["Draft", "Returned"])
)

// lblQueueCount.Text (the pill beside the card title)
CountRows(galQueue.AllItems)

// btnQueueAction.Text — the stage's row label
LookUp(tblStageAction, StageCode = ThisItem.CurrentStageCode.Value).RowLabel

// btnQueueAction.OnSelect
Set(varRef, ThisItem.Title);
Switch(varQueue, "review", Navigate(scrReview), "distribute", Navigate(scrDistribute), Navigate(scrReceipt))

// galQueue.OnSelect
Set(varRef, ThisItem.Title); Navigate(scrRequestDetail)

// lblQueueEmpty.Text and .Visible
```

```
Switch(varQueue,
    "review",     "Nothing is waiting for you to review, approve or confirm.",
    "distribute", "Nothing is waiting for you to distribute.",
    "Nothing is waiting for you to confirm receipt of.")
IsEmpty(galQueue.AllItems)
```

The `Filter` on `galQueue.Items` runs in the app, on `colAwaiting`, which was itself loaded by a filter SharePoint does. That is why it doesn't need to delegate.

> ✓ **You should now see**
>
> As a reviewer with requests at Review, **Review & distribution** lists them with a **Review** button each, and the menu item is highlighted. A request you returned to its requester is not listed.

> **Defects this screen must get right**
>
> DEF-10 (the menu opens a queue, not Home) · DEF-128 (a Returned request isn't offered for review) · DEF-120 (the button names the stage)

6. New request — scrNewRequest

One screen raises a new request and edits an existing Draft or Returned one. `varEditRef` decides which: blank means new, a reference means edit. The menu's **New request** clears it; the **Edit** buttons on Home and Request detail set it.

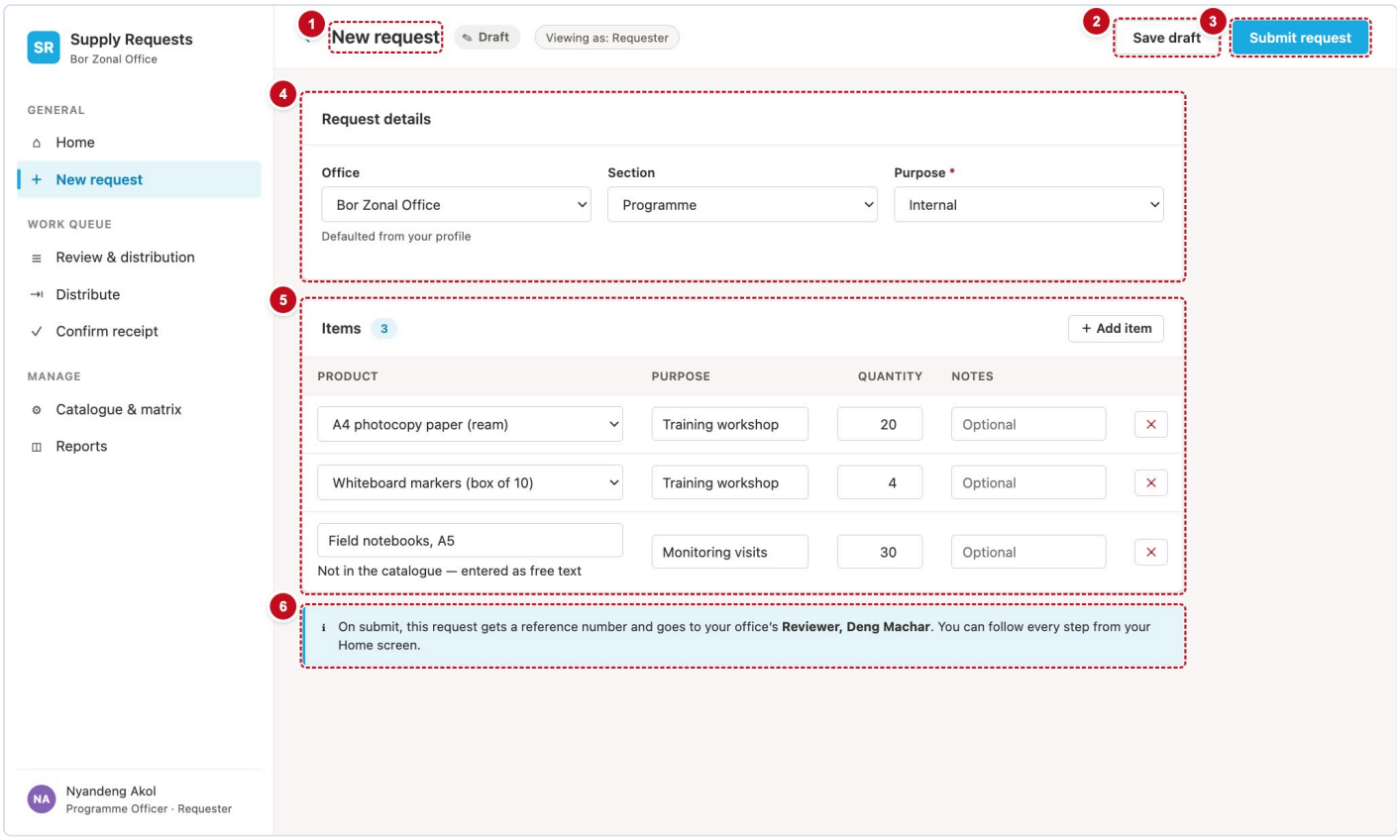


Figure 6. New request, as the mock-up shows it (new-request.html)

#	Part	Controls	What it does
1	Title and status	`icoBack`, `lblPageTitle`, status pill	"New request" with a Draft pill, or "Edit request SR-…" with the request's status
2	Save draft	`btnSaveDraft`	New requests only. Saves without routing (§8)
3	Submit	`btnSubmit`	"Submit request" or "Resubmit request" (§8, §9)
4	Request details	`ddOffice`, `lblOfficeHint`, `ddSection`, `ddPurpose`	Office (defaulted from the user's role), section, purpose
5	Items	`galLines`, `btnAddLine`	One row per item: product (or free text), purpose, quantity, notes, remove
6	Routing preview	`lblRouting` (in a callout)	Who the request will go to first

Also on the screen: `lblReturnReason` (a warning callout shown in edit mode for a Returned request), `lblError` (the error line), and three hidden buttons: `btnChain_New`, `btnCreateCore`, `btnResubmitCore`.

Load the screen

```
// scrNewRequest.OnVisible
Set(varNavActive, If(IsBlank(varEditRef), "new", ""));          // DEF-129: Edit doesn't light "New request"
Set(varPageTitle, If(IsBlank(varEditRef), "New request", "Edit request " & varEditRef));   // DEF-133
Set(varErrorMsg, "");
If(
    IsBlank(varEditRef),
    Set(varEditReq, Blank());
    Set(varInitOffice, varMyOffice);
    ClearCollect(colLines, {
        RowId: Text(GUID()), IsOther: false, ItemCode: "", ItemName: "", IsCatalogueItem: true,
        OfficeItemKey: "", UnitOfMeasure: "", UnitCost: Value(Blank()), LinePurpose: "", LineNote: "",
        QtyRequested: 1
    }),

    Set(varEditReq, LookUp(Requests, Title = varEditRef));
    Set(varInitOffice, varEditReq.OfficeCode);
    ClearCollect(colLines,
        ForAll(Sort(Filter(RequestItems, RequestRefNo = varEditRef), LineNo) As l, {
            RowId: Text(GUID()), IsOther: !l.IsCatalogueItem, ItemCode: Coalesce(l.ItemCode, ""),
            ItemName: l.ItemName, IsCatalogueItem: l.IsCatalogueItem, OfficeItemKey: Coalesce(l.OfficeItemKey,
""),
            UnitOfMeasure: Coalesce(l.UnitOfMeasure, ""), UnitCost: l.UnitCost,
            LinePurpose: Coalesce(l.LinePurpose, ""), LineNote: Coalesce(l.LineNote, ""), QtyRequested:
l.QtyRequested
        }))
);
// The product picker: the office's active catalogue, plus "Other (type it in)…" at the end.
ClearCollect(colPicker,
    ForAll(Sort(Filter(Catalogue, OfficeCode = varInitOffice, IsActive = true), Title) As c,
        { Key: c.OfficeItemKey, Label: c.Title, ItemCode: c.ItemCode, Unit: c.UnitOfMeasure, Cost: c.UnitCost
}));
Collect(colPicker, { Key: "__other", Label: "Other (type it in)…", ItemCode: "", Unit: "", Cost: Value(Blank())
});
Reset(ddOffice); Reset(ddSection); Reset(ddPurpose);
Select(btnChain_New)
```

`Value(Blank())` gives an empty *number*, so the `UnitCost` column is typed as a number from the start.

Request details

```
// ddOffice.Items — active offices, plus the request's own office if it has since been switched off (DEF-14)
Filter(colOffices, IsActive <> false || Title = varInitOffice)
// ddOffice.Value
"OfficeName"
// ddOffice.Default
LookUp(colOffices, Title = varInitOffice).OfficeName
// ddOffice.OnChange — new office, new catalogue and new chain
ClearCollect(colPicker,
    ForAll(Sort(Filter(Catalogue, OfficeCode = Self.Selected.Title, IsActive = true), Title) As c,
        { Key: c.OfficeItemKey, Label: c.Title, ItemCode: c.ItemCode, Unit: c.UnitOfMeasure, Cost: c.UnitCost
}));
Collect(colPicker, { Key: "__other", Label: "Other (type it in)…", ItemCode: "", Unit: "", Cost: Value(Blank())
});
Select(btnChain_New)

// lblOfficeHint.Text and .Visible — only while it really is the profile's office (DEF-115)
"Defaulted from your profile"
IsBlank(varEditRef) && ddOffice.Selected.Title = varMyOffice

// ddSection.Items, .Default, .OnChange
tblSections
If(IsBlank(varEditRef), First(tblSections).Value, varEditReq.Section.Value)
Select(btnChain_New)
```

```
// ddPurpose.Items and .Default
tblPurposes
If(IsBlank(varEditRef), "Internal", varEditReq.Purpose.Value)
```

`btnChain_New` is a hidden button holding Recipe R1 for the office and section on screen. It runs whenever either changes, so the routing preview is always current:

```
// btnChain_New.OnSelect  (Visible: false) — Recipe R1
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = ddOffice.Selected.Title & "|" & ddSection.Selected.Value
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = ddOffice.Selected.Title & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With({ row: If(IsBlank(rowSec), rowAll, rowSec) },
                { Leg: leg.Leg, StageCode: leg.StageCode, LegLabel: leg.LegLabel, ActorLabel: leg.ActorLabel,
                  Live: !IsBlank(row), StageOrder: row.StageOrder, Role: row.ActorRoleCode.Value,
                  Approver: Lower(Trim(row.ApproverEmail)), OIC: Lower(Trim(row.OICEmail)) })
        )
    )
)
```

The items table

`galLines.Items: =colLines`. Each row has a product picker *or* a free-text name, then purpose, quantity, notes and a remove button. Every input saves itself into `colLines` as the user types, found by the row's `RowId`.

```
// cboProduct (Classic ComboBox) — shown unless the row is free text
//   .Items                colPicker
//   .DisplayFields        ["Label"]      .SearchFields ["Label"]      .SelectMultiple false
//   .Visible              !ThisItem.IsOther
//   .DefaultSelectedItems  Filter(colPicker, Key = ThisItem.OfficeItemKey)
//   .OnChange:
If(
    Self.Selected.Key = "__other",
    Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), {
        IsOther: true, ItemName: "", ItemCode: "", OfficeItemKey: "", IsCatalogueItem: false,
        UnitOfMeasure: "", UnitCost: Value(Blank()) }),
    Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), {
        IsOther: false, ItemName: Self.Selected.Label, ItemCode: Self.Selected.ItemCode,
        OfficeItemKey: Self.Selected.Key, IsCatalogueItem: true,
        UnitOfMeasure: Self.Selected.Unit, UnitCost: Self.Selected.Cost })
)

// txtOtherName (Classic TextInput) — the free-text name
//   .Visible  ThisItem.IsOther      .Default  ThisItem.ItemName      .HintText "Describe the item"
//   .OnChange
Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), { ItemName: Self.Text })

// btnBackToCatalogue — the "Pick from the catalogue" link under txtOtherName
//   .Visible  ThisItem.IsOther && CountRows(colPicker) > 1
//   .OnSelect
Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), { IsOther: false, ItemName: "", IsCatalogueItem: true
})

// txtLinePurpose .Default ThisItem.LinePurpose, .OnChange:
Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), { LinePurpose: Self.Text })

// txtQty (Format: TextFormat.Number, Width: =dimQtyBox) .Default ThisItem.QtyRequested, .OnChange:
```

```
Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), { QtyRequested: Max(1, RoundDown(Value(Self.Text),
0)) })

// txtLineNote .Default ThisItem.LineNote, .OnChange (DEF-02: notes are saved):
Patch(colLines, LookUp(colLines, RowId = ThisItem.RowId), { LineNote: Self.Text })

// btnRemoveLine (the ×) .OnSelect
Remove(colLines, LookUp(colLines, RowId = ThisItem.RowId))

// btnAddLine ("+ Add item") .OnSelect — a catalogue row, unless the office has no catalogue (DEF-01)
Collect(colLines, {
    RowId: Text(GUID()), IsOther: CountRows(colPicker) <= 1, ItemCode: "", ItemName: "",
    IsCatalogueItem: CountRows(colPicker) > 1, OfficeItemKey: "", UnitOfMeasure: "",
    UnitCost: Value(Blank()), LinePurpose: "", LineNote: "", QtyRequested: 1 })

// lblLinesCount.Text (the pill beside "Items")
CountRows(colLines) & If(CountRows(colLines) = 1, " item", " items")       // DEF-113
```

> ⚠ **Watch out**
>
> The current app's first build showed a free-text box instead of the product picker (DEF-01). A row is free text only when the user chose "Other (type it in)…", or when the office's catalogue is empty. Don't base it on whether the picker has finished loading.

Top bar, callouts and buttons

```
// Status pill: text and colours from tblStatus for
If(IsBlank(varEditRef), "Draft", varEditReq.Status.Value)

// lblReturnReason.Text and .Visible (DEF-125) — a warning callout above the details card
"Returned: " & varEditReq.ReturnReason
!IsBlank(varEditRef) && varEditReq.Status.Value = "Returned"

// lblRouting.Text — the routing preview (DEF-110: an edit keeps its reference)
With(
    { first: First(Sort(Filter(colChain, Live, StageCode <> "Receive"), Leg)) },
    If(
        IsBlank(first),
        "On submit, this request gets a reference number. This office has no configured approval chain, so it
routes straight to distribution.",
        If(IsBlank(varEditRef),
           "On submit, this request gets a reference number and goes to your office's ",
           "On resubmit, this request keeps its reference " & varEditRef & " and goes back to your office's ")
        & If(first.StageCode = "Review", "Reviewer", first.StageCode)
        & With({ E: Coalesce(first.Approver, first.OIC) },
              If(IsBlank(E), "", ", " & Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " "))))
        & ". You can follow every step from your Home screen."
    )
)

// icoBack.OnSelect
Set(varEditRef, ""); Navigate(scrHome)

// btnSaveDraft.Visible / .OnSelect
IsBlank(varEditRef)
Set(varAsDraft, true); Select(btnCreateCore)

// btnSubmit.Text / .OnSelect
If(IsBlank(varEditRef), "Submit request", "Resubmit request")
Set(varAsDraft, false); If(IsBlank(varEditRef), Select(btnCreateCore), Select(btnResubmitCore))

// btnSubmit.DisplayMode and btnSaveDraft.DisplayMode — and in edit mode, only the requester,
// and only while the request is still a Draft or Returned
```

```
If(
    varBusy || (!IsBlank(varEditRef)
                && (Lower(varEditReq.RequesterEmail) <> varUserEmail
                    || !(varEditReq.Status.Value in ["Draft", "Returned"]))),
    DisplayMode.Disabled, DisplayMode.Edit
)
```

`btnCreateCore.OnSelect` and `btnResubmitCore.OnSelect` are the formulas in **§8** and **§9** of chapter 13. Paste them into the two hidden buttons when you get to chapter 13.

ℹ A small addition

The current app opens the edit form for anyone who reaches it and relies on the Edit buttons only being shown to the requester. The `DisplayMode` above also disables the buttons when someone else opens a request in edit mode, or when it is no longer a Draft or Returned.

✓ You should now see

A new request opens with one empty catalogue row, the office set to your own with "Defaulted from your profile" beneath, and the routing preview naming your office's reviewer. Change the office and the hint disappears and the preview changes. Choose **Other (type it in)…** and the row turns into a text box.

Defects this screen must get right

DEF-01 (the product picker appears) · DEF-02 (item notes are saved) · DEF-06 (a failed save says so) · DEF-110 (an edit keeps its reference) · DEF-113 ("1 item") · DEF-115 (the "defaulted" hint goes when the office changes) · DEF-125 (the return reason is shown) · DEF-129 (Edit doesn't light "New request") · DEF-131 (the brand line isn't blank) · DEF-133 (the Edit title shows the reference)

7. Request detail — scrRequestDetail

Everything about one request: who raised it, who has it now, where it is on its journey, its items, and its timeline, comments and documents. Anyone who can open the app can open any request here. The buttons decide who can act.

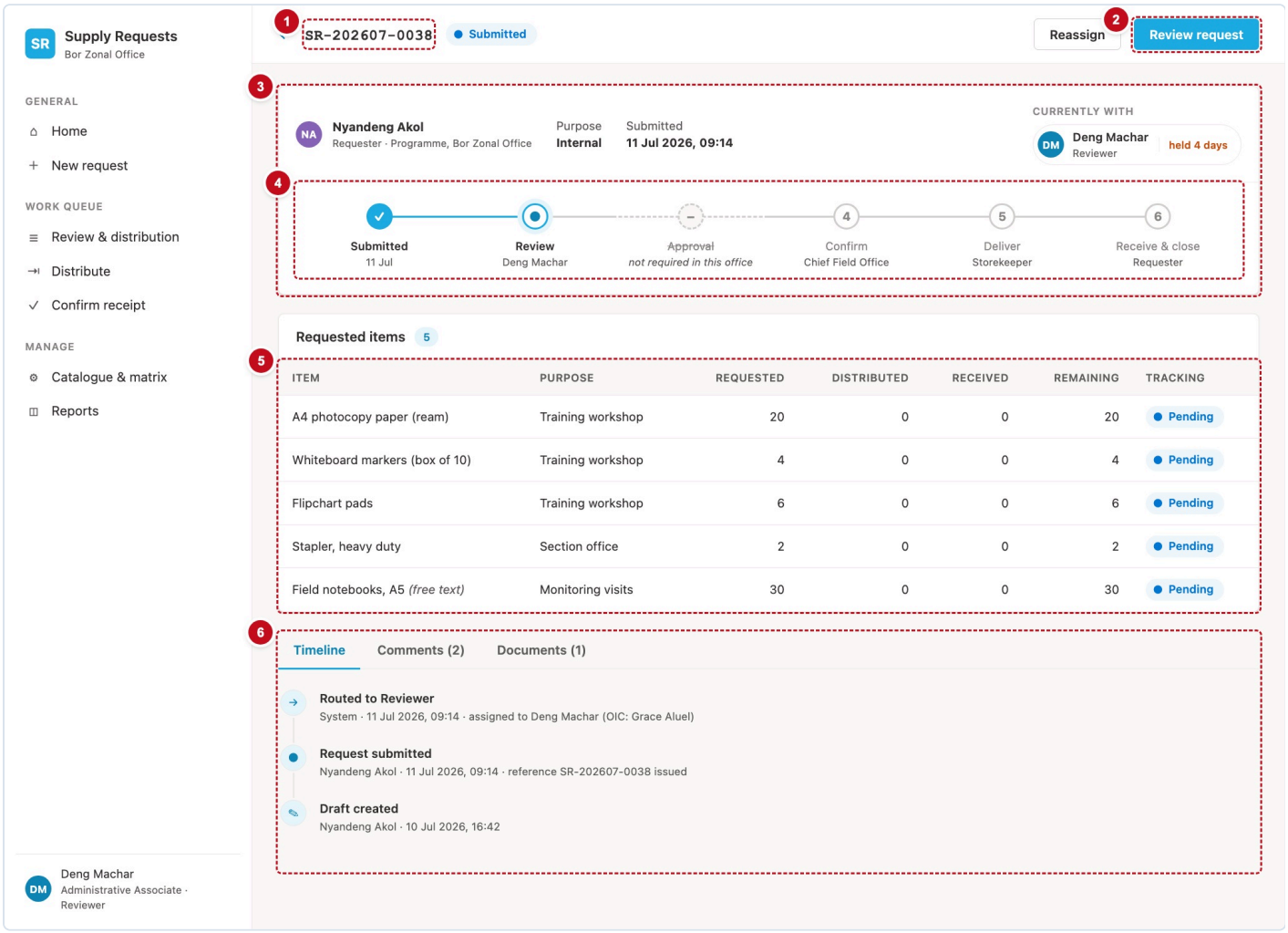


Figure 7. Request detail, as the mock-up shows it (request-detail.html)

#	Part	Controls	What it does
1	Title	`icoBack`, `lblPageTitle`, status pill	"Request SR-…" and its status
2	Primary action	`btnPrimary`	**Edit request** for a Draft or Returned request, otherwise the current stage's action (**Review request**, **Distribute**, …)
3	Summary	`lblRequester`, `lblMeta`, `lblPurpose`, `lblSubmitted`, the "Currently with" chip	Who raised it, when, and who holds it now
4	Journey rail	`galJourney` (horizontal)	The six legs, each done, current, still to come, or skipped
5	Requested items	`galItems`	Requested, approved, distributed, received and remaining per line
6	Tabs	`btnTabTimeline`, `btnTabComments`, `btnTabDocs`, and one panel per tab	Timeline, comments, documents

Load the request

The screen loads from a hidden button so a comment or a document can reload it (Recipe R6).

```
// scrRequestDetail.OnVisible
Set(varNavActive, "");
Set(varDetailTab, "timeline");
Set(varErrorMsg, "");
Select(btnLoad_Detail)

// btnLoad_Detail.OnSelect  (Visible: false)
Set(varReq, LookUp(Requests, Title = varRef));
Set(varPageTitle, "Request " & varRef);
ClearCollect(colLines, Sort(Filter(RequestItems, RequestRefNo = varRef), LineNo));
// Recipe R1 — the approval chain for this request's office and section
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|" & varReq.Section.Value
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With({ row: If(IsBlank(rowSec), rowAll, rowSec) },
                { Leg: leg.Leg, StageCode: leg.StageCode, LegLabel: leg.LegLabel, ActorLabel: leg.ActorLabel,
                  Live: !IsBlank(row), StageOrder: row.StageOrder, Role: row.ActorRoleCode.Value,
                  Approver: Lower(Trim(row.ApproverEmail)), OIC: Lower(Trim(row.OICEmail)) })
        )
    )
);
// Recipe R3 — who may act
Set(varCanAct, And(varReq.StatusRank < 70,
    Or(!IsBlank(varReq.CurrentlyWithEmail)    && Lower(varReq.CurrentlyWithEmail)    = varUserEmail,
       !IsBlank(varReq.CurrentlyWithOICEmail) && Lower(varReq.CurrentlyWithOICEmail) = varUserEmail)));
Set(varIsOriginator, Lower(varReq.RequesterEmail) = varUserEmail);
Set(varIsEditable,   varReq.Status.Value in ["Draft", "Returned"]);
Set(varIsReviewable, varReq.Status.Value in ["Submitted", "Reviewed"]);
Set(varStage,        If(varIsEditable, "", varReq.CurrentStageCode.Value));

// The timeline, newest first; rows in the same minute keep their order by ID (DEF-130).
ClearCollect(colTimeline,
    SortByColumns(Filter(RequestTimeline, RequestRefNo = varRef),
                  "OccurredOn", SortOrder.Descending, "ID", SortOrder.Descending));
ClearCollect(colComments,
    SortByColumns(Filter(RequestComments, RequestRefNo = varRef), "PostedOn", SortOrder.Ascending));
ClearCollect(colDocs, Filter(RequestDocuments, RequestRefNo = varRef));

// The journey, ported from src/domain/stages.ts resolveJourney.
// DEF-127: a Draft or Returned request is back with its requester, so it shows no current stage.
// DEF-141: a Rejected request stopped where it was; only a finished one ticks every leg.
Set(varJourneyStage, If(varIsEditable || varReq.Status.Value = "Rejected", "", varReq.CurrentStageCode.Value));
Set(varJourneyDone,  varReq.StatusRank >= 70 && varReq.Status.Value <> "Rejected");
ClearCollect(colJourney,
    ForAll(tblJourney As leg,
        With(
            {
                live:   leg.StageCode = "" || LookUp(colChain, StageCode = leg.StageCode).Live,
                curLeg: If(IsBlank(varJourneyStage), Blank(), LookUp(tblJourney, StageCode =
varJourneyStage).Leg)
            },
            {
                Leg:      leg.Leg,
                LegLabel: leg.LegLabel,
                State: If(
                    !live,                "skip",
                    leg.StageCode = "",    If(varIsEditable, "now", "done"),
```

```
                    varJourneyDone,        "done",
                    IsBlank(curLeg),       "todo",
                    leg.Leg < curLeg,      "done",
                    leg.Leg = curLeg,      "now",
                    "todo"),
                // DEF-106: the holder's name on the current leg only; otherwise the role label.
                Actor: If(
                    !live, txtLegSkipped,
                    !IsBlank(varReq.CurrentlyWithEmail) && leg.StageCode = varReq.CurrentStageCode.Value,
                        With({ E: varReq.CurrentlyWithEmail },
                             Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " "))),
                    leg.StageCode = "" && !IsBlank(varReq.SubmittedOn), Text(varReq.SubmittedOn, "d mmm"),
                    leg.ActorLabel)
            }
        )
    )
)
```

Header and summary

```
// btnPrimary.Visible — nothing to offer on a finished request (DEF-124)
varIsEditable || (varReq.StatusRank < 70 && !IsBlank(varStage))
// btnPrimary.Text
If(varIsEditable, "Edit request", LookUp(tblStageAction, StageCode = varStage).DetailLabel)
// btnPrimary.DisplayMode — Edit is the requester's; a stage action is the holder's
If(If(varIsEditable, varIsOriginator, varCanAct), DisplayMode.Edit, DisplayMode.Disabled)
// btnPrimary.OnSelect
If(
    varIsEditable,
    Set(varEditRef, varRef); Navigate(scrNewRequest),
    Switch(LookUp(tblStageAction, StageCode = varStage).Screen,
        "review", Navigate(scrReview), "distribute", Navigate(scrDistribute), Navigate(scrReceipt))
)

// lblRequester.Text and lblMeta.Text
With({ E: varReq.RequesterEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " ")))
varReq.Section.Value & " · " & Coalesce(LookUp(colOffices, Title = varReq.OfficeCode).OfficeName,
varReq.OfficeCode)
// lblPurpose.Text and lblSubmitted.Text
varReq.Purpose.Value
If(IsBlank(varReq.SubmittedOn), "Not yet submitted", Text(varReq.SubmittedOn, "d mmm yyyy, hh:mm"))

// The "Currently with" chip: Visible, who, and the held-for (Recipe R5)
varReq.StatusRank < 70 && !IsBlank(varReq.CurrentlyWithEmail)
With({ E: varReq.CurrentlyWithEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " ")))
    & " · " & If(varIsEditable, "Requester", LookUp(tblJourney, StageCode = varStage).ActorLabel)
With({ h: DateDiff(Coalesce(varReq.Modified, varReq.SubmittedOn), Now(), TimeUnit.Hours) },
    "held " & If(h < 1, "under an hour", h < 24, Text(h) & If(h = 1, " hour", " hours"),
                 With({ d: RoundDown(h / 24, 0) }, Text(d) & If(d = 1, " day", " days"))))
```

The journey rail

`galJourney` is a **horizontal** blank gallery with `Items: =colJourney`, `TemplateSize: =Self.Width / 6`, and no scroll bar. Each leg has a 2-pixel connector line, a round dot, a label and a sub-label:

Control	Property	Formula
`recLegLine` (Rectangle)	`Fill`	`If(ThisItem.State = "done", clrBrand, clrLine)` (`Height: =dimRailBar`, across the template at the dot's centre; `Visible: =ThisItem.Leg < 6`)

Control	Property	Formula
`btnLegDot` (Classic Button shape, 28 × 28, radius `radCircle`)	`Fill`	`Switch(ThisItem.State, "done", clrBrand, "skip", clrCanvas, clrSurface)`
	`BorderColor`	`If(ThisItem.State in ["done", "now"], clrBrand, clrLineStrong)`
	`BorderStyle`	`If(ThisItem.State = "skip", BorderStyle.Dashed, BorderStyle.Solid)`
	`Text`	`Switch(ThisItem.State, "done", "✓", "now", "●", "skip", "–", Text(ThisItem.Leg))`
	`Color`	`Switch(ThisItem.State, "done", clrOnBrand, "now", clrBrandDark, clrInk3)`
`lblLeg`	`Text`	`ThisItem.LegLabel`
	`Strikethrough`	`ThisItem.State = "skip"`
	`Color`, `FontWeight`	`If(ThisItem.State in ["todo", "skip"], clrInk3, clrInk)`, `If(ThisItem.State in ["todo", "skip"], FontWeight.Normal, FontWeight.Semibold)`
`lblLegSub`	`Text`	`ThisItem.Actor`
	`Italic`	`ThisItem.State = "skip"`

Requested items

`galItems.Items: =colLines`.

Column	Formula
Item	`ThisItem.ItemName & If(ThisItem.IsCatalogueItem, "", " (free text)")`
Purpose	`Coalesce(ThisItem.LinePurpose, txtNoHolder)`
Notes	`Coalesce(ThisItem.LineNote, txtNoHolder)` (DEF-121)
Requested	`ThisItem.QtyRequested`
Approved	`If(IsBlank(ThisItem.QtyApproved), txtNoHolder, Text(ThisItem.QtyApproved))` (DEF-136)
Distributed	`Coalesce(ThisItem.QtyDistributed, 0)`
Received	`Coalesce(ThisItem.QtyReceived, 0)`
Remaining	`Max(0, Coalesce(ThisItem.QtyApproved, ThisItem.QtyRequested) – Coalesce(ThisItem.QtyDistributed, 0))` (DEF-117)
Tracking	`Coalesce(ThisItem.LineStatus.Value, "Pending")` in a pill (DEF-108)

The tabs

Three Classic Buttons styled as tabs set `varDetailTab` to `"timeline"`, `"comments"` or `"docs"`; each panel's controls have `Visible: =varDetailTab = "…"`. The tab captions carry counts:

```
"Timeline"
"Comments (" & CountRows(colComments) & ")"
"Documents (" & CountRows(colDocs) & ")"
```

Timeline. `galTimeline.Items: =colTimeline`.

```
// lblTlGlyph.Text
Coalesce(LookUp(tblEventGlyph, EventType = ThisItem.EventType.Value).Glyph, "•")
// lblTlTitle.Text
ThisItem.Title
// lblTlMeta.Text — "System" when the row has no actor
If(IsBlank(ThisItem.ActorEmail), txtSystemActor,
   With({ E: ThisItem.ActorEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " "))))
& " · " & Text(ThisItem.OccurredOn, "d mmm yyyy, hh:mm")
// lblTlDetail.Text and .Visible
ThisItem.Detail
!IsBlank(ThisItem.Detail)
```

Comments. `galComments.Items: =colComments`, showing the author (Recipe R4 on `AuthorEmail`), `PostedOn` and `CommentText`. Below it, `txtComment` (multi-line) and **Post comment** (`btnPostComment`, §14).

Documents. `galDocs.Items: =colDocs`, showing `'File name with extension'`, `DocType.Value` and the uploader (`UploadedByEmail`, Recipe R4). The name opens the file: `OnSelect: =Launch(ThisItem.'Link to item')`. Below it, a document-type dropdown `ddDocType` (`Items: =tblDocTypes`), the attachments control `attFile`, and **Attach a file** (`btnAttach`, §14).

Getting an Attachments control

Studio has no stand-alone file picker. The usual way to get one is to borrow the Attachments control from a form:

1. In SharePoint, create a throw-away list called `UploadHelper` (attachments are on by default for a new list). Add it to the app as a data source.
2. Insert an **Edit form**, set its `DataSource` to `UploadHelper`, and find the **Attachments** card.
3. Select the attachments control inside the card, copy it, select `scrRequestDetail`, and paste. Rename it `attFile`.
4. Delete the form and remove `UploadHelper` from the app's data sources. The pasted control keeps working without it.
5. On `attFile`, set `Items: =Blank()`, `MaxAttachments: =1`, `DisplayMode: =DisplayMode.Edit`, and clear any error that refers to the old form by setting those properties to plain values.

> ✓ **You should now see**
>
> Open `SR-202607-0038` from Home. The journey shows Submitted ticked, Review current with the reviewer's name, and Approval struck through with "not required in this office". The timeline lists newest first.

Defects this screen must get right

DEF-02 and DEF-121 (item notes are shown) · DEF-101 (a comment or document adds a timeline row) · DEF-106 (a finished request's legs keep their role labels) · DEF-108 (a new line says "Pending") · DEF-111 ("—" for no holder) · DEF-117 (remaining is approved minus distributed) · DEF-124 (Draft and Returned offer Edit, finished requests offer nothing) · DEF-127 (a Returned request shows the journey back at Submitted) · DEF-130 (same-minute rows keep their order) · DEF-136 (an Approved column) · DEF-141 (a Rejected request's later legs are not ticked)

8. Review, approve and confirm — scrReview

One screen serves three stages: **Review** (the reviewer sets how much of each item to give out), **Approval** (finance) and **Confirm** (the Chief Field Office). Only Review changes quantities. Approval and Confirm see the approved items and move the request on. At any of the three, the holder can also return the request to its requester or reject it.

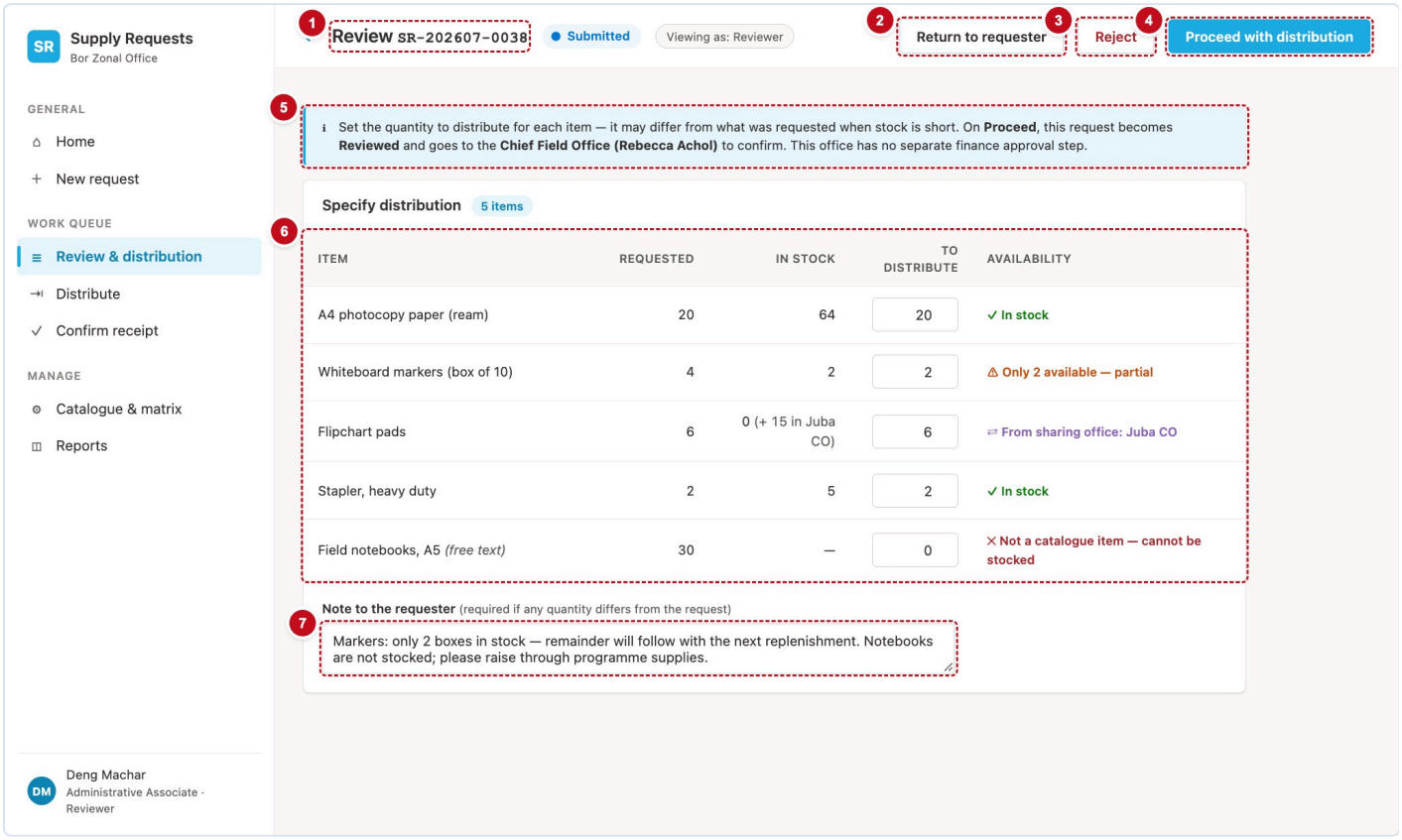


Figure 8. Review, as the mock-up shows it (review.html)

#	Part	Controls	What it does
1	Title	`icoBack`, `lblPageTitle`, status pill, `lblPersona`	"Review SR-…", "Approve SR-…" or "Confirm SR-…" (DEF-107)
2	Return	`btnReturn`	Opens the reason dialog to send it back (§11)
3	Reject	`btnReject`	Opens the reason dialog to reject it (§11)
4	Proceed	`btnProceed`	Moves it to the next stage (§10)
5	What happens next	`lblNextStep` (in a callout)	Says where the request goes on Proceed
6	Specify distribution	`galReview` (Review), or `galApproved` (Approval, Confirm)	Per line: requested, in stock, to distribute, availability
7	Note to the requester	`txtNote`	Required when any quantity differs from what was asked

Also on the screen: the reason dialog (`recDialogShade`, `recDialog`, `lblDialogTitle`, `txtReason`, `btnDialogConfirm`, `btnDialogCancel`), the error line, and `btnLoad_Review`.

Load the request

```
// scrReview.OnVisible
Set(varNavActive, "review");
Set(varShowDialog, false);
Set(varErrorMsg, "");
Select(btnLoad_Review)

// btnLoad_Review.OnSelect  (Visible: false)
Set(varReq, LookUp(Requests, Title = varRef));
ClearCollect(colLines, Sort(Filter(RequestItems, RequestRefNo = varRef), LineNo));
// Recipe R1 — the approval chain
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|" & varReq.Section.Value
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With({ row: If(IsBlank(rowSec), rowAll, rowSec) },
                { Leg: leg.Leg, StageCode: leg.StageCode, LegLabel: leg.LegLabel, ActorLabel: leg.ActorLabel,
                  Live: !IsBlank(row), StageOrder: row.StageOrder, Role: row.ActorRoleCode.Value,
                  Approver: Lower(Trim(row.ApproverEmail)), OIC: Lower(Trim(row.OICEmail)) })
        )
    )
);
// Recipe R3 — who may act
Set(varCanAct, And(varReq.StatusRank < 70,
    Or(!IsBlank(varReq.CurrentlyWithEmail)    && Lower(varReq.CurrentlyWithEmail)    = varUserEmail,
       !IsBlank(varReq.CurrentlyWithOICEmail) && Lower(varReq.CurrentlyWithOICEmail) = varUserEmail)));
Set(varIsOriginator, Lower(varReq.RequesterEmail) = varUserEmail);
Set(varIsEditable,   varReq.Status.Value in ["Draft", "Returned"]);
Set(varIsReviewable, varReq.Status.Value in ["Submitted", "Reviewed"]);
Set(varStage,        If(varIsEditable, "", varReq.CurrentStageCode.Value));
Set(varPageTitle, Coalesce(LookUp(tblStageAction, StageCode = varStage).ScreenTitle, "Review") & " " & varRef);

// Where Proceed would send it (Recipe R2), for the callout.
Set(varRoute,
    With({ fromLeg: Coalesce(LookUp(tblJourney, StageCode = varStage).Leg, 1) },
        With({ next: First(Sort(Filter(colChain, Live, Leg > fromLeg), Leg)) },
            If(IsBlank(next) || next.StageCode = "Receive",
                { NextStage: "Receive", WithEmail: Lower(varReq.RequesterEmail), OICEmail: "" },
                With({ primary: Coalesce(next.Approver, next.OIC) },
                    { NextStage: next.StageCode,
                      WithEmail: Coalesce(primary, Lower(varReq.RequesterEmail)),
                      OICEmail:  If(!IsBlank(next.OIC) && next.OIC <> primary, next.OIC, "") })))));

// Stock: this office's catalogue, and every lending office's, in priority order.
ClearCollect(colCat, Filter(Catalogue, OfficeCode = varReq.OfficeCode));
Clear(colLenderStock);
ForAll(
    Sort(Filter(colSharing, ToOfficeCode = varReq.OfficeCode), SharePriority) As lend,
    Collect(colLenderStock,
        ForAll(Filter(Catalogue, OfficeCode = lend.FromOfficeCode) As c,
            { OfficeCode: c.OfficeCode, ItemCode: c.ItemCode, StockQty: c.StockQty, Priority:
lend.SharePriority }))
);

// One row per line, with the stock and the most that may be given out.
// Ported from ReviewPage.tsx: a short catalogue line looks at the lending offices in
// priority order and takes the first with any stock (src/domain/sharing.ts).
ClearCollect(colReview,
    ForAll(colLines As ln,
        With(
            { localStock: Coalesce(LookUp(colCat, OfficeItemKey = ln.OfficeItemKey).StockQty, 0) },
            With(
```

```
                { lender: If(ln.IsCatalogueItem && localStock < ln.QtyRequested,
                            First(Sort(Filter(colLenderStock, ItemCode = ln.ItemCode, StockQty > 0),
Priority))) },
                With(
                    { maxQty: If(!ln.IsCatalogueItem,              999999,
                                 localStock >= ln.QtyRequested,    localStock,
                                 !IsBlank(lender),                 lender.StockQty,
                                 localStock) },
                    {
                        LineID: ln.ID, LineNo: ln.LineNo, ItemName: ln.ItemName, ItemCode: ln.ItemCode,
                        IsCatalogueItem: ln.IsCatalogueItem, QtyRequested: ln.QtyRequested,
                        LinePurpose: ln.LinePurpose, QtyApproved: ln.QtyApproved,
                        LocalStock: localStock,
                        LenderOffice: lender.OfficeCode, LenderStock: Coalesce(lender.StockQty, 0),
                        MaxQty: maxQty,
                        DefaultQty: If(ln.IsCatalogueItem, Min(ln.QtyRequested, maxQty), 0)
                    }
                )
            )
        )
    )
)
```

The top of the screen

```
// lblPersona.Text
"Viewing as: " & Coalesce(LookUp(tblJourney, StageCode = varStage).ActorLabel, "Reviewer")

// lblNextStep.Text — the callout (ReviewPage.tsx)
If(varStage = "Review",
   "Set the quantity to distribute for each item — it may differ from what was requested when stock is short.
", "")
& "On Proceed, this request " & If(varStage = "Review", "becomes", "stays") & " Reviewed and goes to the "
& LookUp(tblJourney, StageCode = varRoute.NextStage).ActorLabel
& " (" & With({ E: varRoute.WithEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " "))) & ")"
& If(varRoute.NextStage = "Receive", " to confirm receipt.", " to act next.")
```

The `DisplayMode` and `OnSelect` of **Proceed**, **Return**, **Reject** and the dialog's confirm button are in §10 and §11 (chapter 13).

Specify distribution (Review stage)

`galReview.Items: =colReview`, `Visible: =varStage = "Review"`.

Column	Formula
Item	`ThisItem.ItemName & If(ThisItem.IsCatalogueItem, "", " (free text)")`
Requested	`ThisItem.QtyRequested`
In stock	`If(!ThisItem.IsCatalogueItem, txtNoHolder, ThisItem.LocalStock & If(IsBlank(ThisItem.LenderOffice), "", " (+ " & ThisItem.LenderStock & " in " & LookUp(colOffices, Title = ThisItem.LenderOffice).OfficeName & ")"))`
To distribute	`txtApprove` (Classic TextInput, `Format: =TextFormat.Number`, `Width: =dimQtyBox`, `Default: =ThisItem.DefaultQty`)
Availability	`lblAvail`, below

```
// lblAvail.Text
If(
```

```
    !ThisItem.IsCatalogueItem,                          "× Not a catalogue item — cannot be stocked",
    !IsBlank(ThisItem.LenderOffice),                    "⇄ From sharing office: " & LookUp(colOffices, Title =
ThisItem.LenderOffice).OfficeName,
    ThisItem.LocalStock >= ThisItem.QtyRequested,       "✓ In stock",
    ThisItem.LocalStock > 0,                            "⚠ Only " & ThisItem.LocalStock & " available — partial",
    "× Out of stock"
)
// lblAvail.Color
If(
    !ThisItem.IsCatalogueItem,                          clrStockOut,
    !IsBlank(ThisItem.LenderOffice),                    clrStockLend,
    ThisItem.LocalStock >= ThisItem.QtyRequested,       clrStockOk,
    ThisItem.LocalStock > 0,                            clrStockLow,
    clrStockOut
)

// lblOverMax — a small red hint under the box when the number typed is more than can be given
//   .Text     "At most " & ThisItem.MaxQty
//   .Visible  ThisItem.IsCatalogueItem && Value(txtApprove.Text) > ThisItem.MaxQty
```

Proceed saves `Min(Max(typed, 0), MaxQty)`, so a number above the maximum is capped, as the current app caps it. The hint just makes that visible.

Note to the requester. `txtNote` (multi-line). Its label reads "Note to the requester (required if any quantity differs from the request)". §10's `btnProceed.DisplayMode` enforces it.

Approved items (Approval and Confirm stages)

`galApproved.Items: =colReview`, `Visible: =varStage in ["Approval", "Confirm"]`. Three read-only columns: Item, Purpose (`Coalesce(ThisItem.LinePurpose, txtNoHolder)`) and Approved (`Coalesce(ThisItem.QtyApproved, 0)`). The card's title is "Approved items" with the count `CountRows(colReview) & If(CountRows(colReview) = 1, " item", " items")`.

The reason dialog

A dialog in canvas is a set of controls shown together:

1. Insert a full-screen Rectangle `recDialogShade` (`Fill: =RGBA(0, 0, 0, 0.35)`), then a Classic Button shape `recDialog` centred on it (`Fill: =clrSurface`, radius `rad6`), then on top of that `lblDialogTitle`, `txtReason` (multi-line), `lblDialogError`, `btnDialogCancel` and `btnDialogConfirm`. They must be the **last** controls in the Tree view so they draw over everything.
2. Set `Visible: =varShowDialog` on all of them.
3. Set the texts:

```
// lblDialogTitle.Text
If(varDialog = "return", "Return " & varRef & " to the requester", "Reject " & varRef)
// txtReason.HintText
If(varDialog = "return", "What needs to change? The requester sees this.", "Why is this request rejected?")
// btnDialogConfirm.Text
If(varDialog = "return", "Return to requester", "Reject request")
// btnDialogCancel.OnSelect
Set(varShowDialog, false)
// lblDialogError.Text / .Visible — a failed write keeps the dialog open and says why (DEF-06)
varErrorMsg
!IsBlank(varErrorMsg)
```

✓ You should now see

As the reviewer of `SR-202607-0038` : "Review SR-202607-0038", five lines with availability badges (one from the sharing office, one not a catalogue item), and **Proceed** greyed out until you write a note, because the markers line asks for 4 but only 2 are in stock. As someone who doesn't hold the request, all three buttons are greyed out.

Defects this screen must get right

DEF-04 (Return, Reject and Proceed only for the holder, at a review stage) · DEF-06 (a failed save says so) · DEF-107 (the title names the stage) · DEF-113 ("1 item") · DEF-114 (the reference isn't clipped) · DEF-124 (a Returned request can't be acted on here)

9. Distribute — scrDistribute

The storekeeper records what is handed out. Stock comes off the office that supplies each line: the request's own office, or the lending office the reviewer chose. A delivery can be done in parts. The request stays with the storekeeper until everything approved has gone out, then moves to the requester to confirm receipt.

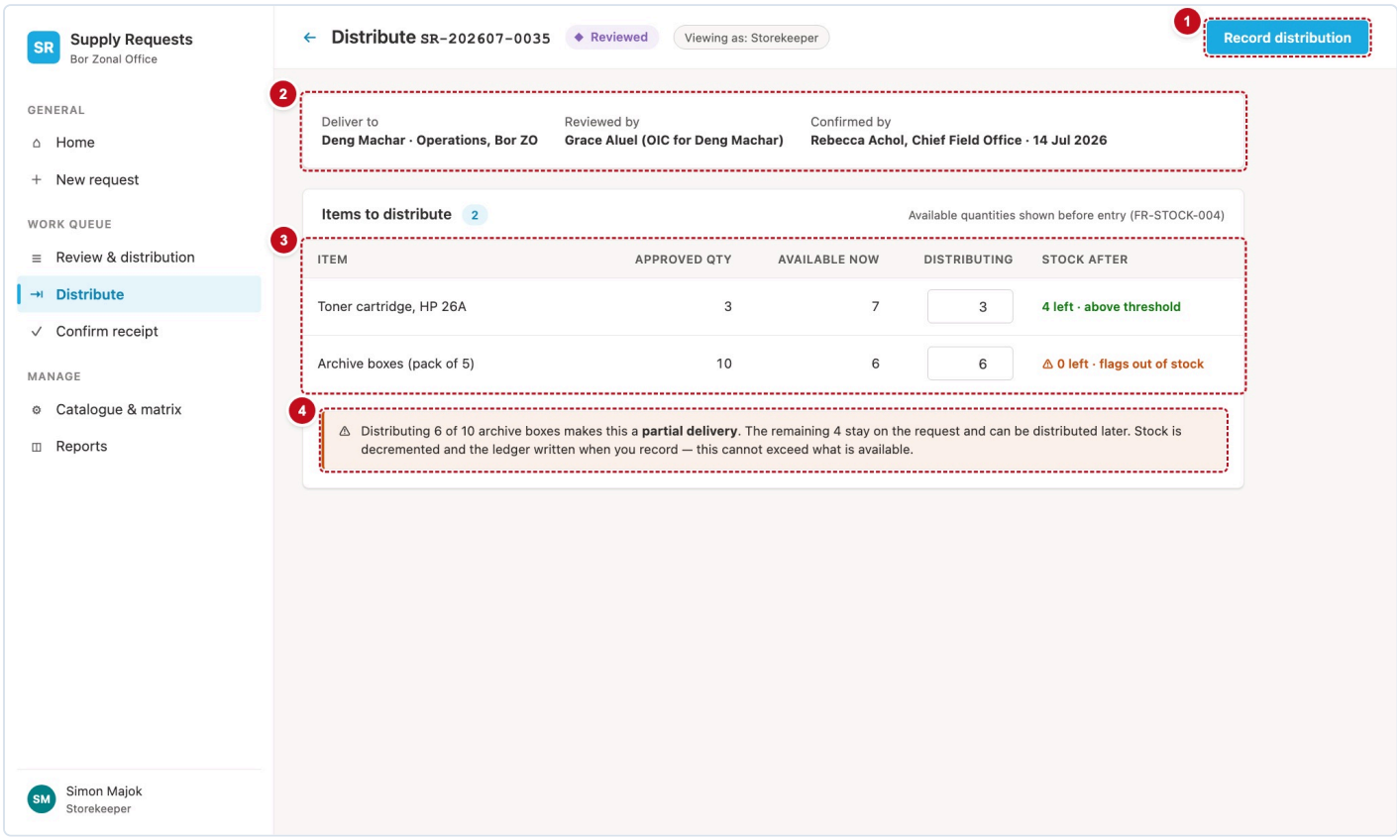


Figure 9. Distribute, as the mock-up shows it (distribute.html)

#	Part	Controls	What it does
1	Record distribution	`btnRecordDistribution`	Writes the ledger, the stock and the lines, then the request (§12)
2	Who and how	`lblDeliverTo`, `lblReviewedBy`, `lblConfirmedBy`	Who it goes to, who reviewed and who confirmed it
3	Items to distribute	`galDist`	Per line: approved, available now, distributing, stock after
4	Partial warning	`lblPartial` (in a warning callout)	Shown when this will not finish the request

Load the request

```
// scrDistribute.OnVisible
Set(varNavActive, "distribute");
Set(varErrorMsg, "");
Set(varDistReady, false);
Select(btnLoad_Distribute)

// btnLoad_Distribute.OnSelect  (Visible: false)
```

```
Set(varReq, LookUp(Requests, Title = varRef));
Set(varPageTitle, "Distribute " & varRef);
ClearCollect(colLines, Sort(Filter(RequestItems, RequestRefNo = varRef), LineNo));
// Recipe R1 — the approval chain (Record distribution needs the Deliver stage's holders)
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|" & varReq.Section.Value
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With({ row: If(IsBlank(rowSec), rowAll, rowSec) },
                { Leg: leg.Leg, StageCode: leg.StageCode, LegLabel: leg.LegLabel, ActorLabel: leg.ActorLabel,
                  Live: !IsBlank(row), StageOrder: row.StageOrder, Role: row.ActorRoleCode.Value,
                  Approver: Lower(Trim(row.ApproverEmail)), OIC: Lower(Trim(row.OICEmail)) })
        )
    )
);
// Recipe R3 — who may act
Set(varCanAct, And(varReq.StatusRank < 70,
    Or(!IsBlank(varReq.CurrentlyWithEmail)    && Lower(varReq.CurrentlyWithEmail)    = varUserEmail,
       !IsBlank(varReq.CurrentlyWithOICEmail) && Lower(varReq.CurrentlyWithOICEmail) = varUserEmail)));
Set(varIsEditable, varReq.Status.Value in ["Draft", "Returned"]);
Set(varStage,      If(varIsEditable, "", varReq.CurrentStageCode.Value));
ClearCollect(colTimeline, SortByColumns(Filter(RequestTimeline, RequestRefNo = varRef), "OccurredOn",
SortOrder.Ascending));

// The catalogue lines still owed, each with the live stock of the office that supplies it.
Refresh(Catalogue);
ClearCollect(colDistView,
    ForAll(Filter(colLines, IsCatalogueItem, Coalesce(QtyApproved, 0) > Coalesce(QtyDistributed, 0)) As ln,
        With(
            { source: Coalesce(ln.SourceOfficeCode, varReq.OfficeCode) },
            With(
                { cat: LookUp(Catalogue, OfficeItemKey = source & "|" & ln.ItemCode) },
                {
                    LineID: ln.ID, ItemName: ln.ItemName, ItemCode: ln.ItemCode,
                    SourceOffice: source, OfficeItemKey: source & "|" & ln.ItemCode,
                    QtyApproved: Coalesce(ln.QtyApproved, 0), QtyDistributed: Coalesce(ln.QtyDistributed, 0),
                    Remaining: Coalesce(ln.QtyApproved, 0) - Coalesce(ln.QtyDistributed, 0),
                    Available: Coalesce(cat.StockQty, 0), ReorderLevel: cat.ReorderLevel
                }
            )
        )
    )
);
Set(varDistReady, true)
```

The gallery and the button wait for `varDistReady`, so the screen never shows every line at zero stock while the figures load (DEF-144).

Who and how

```
// lblDeliverTo.Text
With({ E: varReq.RequesterEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " ")))
& " · " & varReq.Section.Value & ", " & Coalesce(LookUp(colOffices, Title = varReq.OfficeCode).OfficeName,
varReq.OfficeCode)

// lblReviewedBy.Text — the first "Reviewed" timeline row
With({ ev: First(Filter(colTimeline, EventType.Value = "Reviewed")) },
    If(IsBlank(ev), txtNoHolder,
       With({ E: ev.ActorEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " ")))))
```

```
// lblConfirmedBy.Text — the latest "Confirmed" row, or a note when the office has no Confirm stage
With({ ev: Last(Filter(colTimeline, EventType.Value = "Confirmed")) },
    If(
        !LookUp(colChain, StageCode = "Confirm").Live, "Not required in this office",
        IsBlank(ev), txtNoHolder,
        With({ E: ev.ActorEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", " ")))
        & " (" & LookUp(colChain, StageCode = "Confirm").Role & ") · " & Text(ev.OccurredOn, "d mmm")))
```

Items to distribute

`galDist.Items: =colDistView`, `Visible: =varDistReady`.

Column	Formula
Item	`ThisItem.ItemName`
Approved qty	`ThisItem.QtyApproved`
Available now	`ThisItem.Available & If(ThisItem.SourceOffice <> varReq.OfficeCode, " · from " & LookUp(colOffices, Title = ThisItem.SourceOffice).OfficeName, "")` (DEF-112)
Distributing	`txtDist` (Classic TextInput, number, `Width: =dimQtyBox`, `Default: =Min(ThisItem.Remaining, ThisItem.Available)`)
Stock after	`lblAfter`, below

```
// lblAfter.Text — DEF-118: "at threshold" when equal, "below" only under it; DEF-126: nothing without a
threshold
With(
    { after: ThisItem.Available - Min(Max(Coalesce(Value(txtDist.Text), 0), 0), ThisItem.Remaining,
ThisItem.Available),
      lvl:   ThisItem.ReorderLevel },
    With(
        { wording: If(IsBlank(lvl), "", after < lvl, "below threshold", after = lvl, "at threshold", "above
threshold") },
        If(
            after = 0,                                "⚠ 0 left · flags out of stock",
            !IsBlank(lvl) && after <= lvl,            "⚠ " & after & " left · " & wording,
            after & " left" & If(IsBlank(wording), "", " · " & wording)
        )
    )
)
// lblAfter.Color
With({ after: ThisItem.Available - Min(Max(Coalesce(Value(txtDist.Text), 0), 0), ThisItem.Remaining,
ThisItem.Available) },
    If(after = 0, clrStockOut,
       !IsBlank(ThisItem.ReorderLevel) && after <= ThisItem.ReorderLevel, clrStockLow,
       clrStockOk))
```

```
// lblPartial.Visible — some line will still be owed after this step
!IsEmpty(Filter(galDist.AllItems, Min(Max(Coalesce(Value(txtDist.Text), 0), 0), Remaining, Available) <
Remaining))
// lblPartial.Text
"Part of this request will still be owed after this distribution. It stays with you at Deliver until everything
approved has been handed out; then it goes to the requester to confirm receipt."

// btnRecordDistribution.DisplayMode
If(varCanAct && varDistReady && !varBusy && !IsEmpty(colDistView), DisplayMode.Edit, DisplayMode.Disabled)
// btnRecordDistribution.OnSelect — §12, chapter 13
```

Empty message, when every line is already out: "Everything approved on this request has been distributed."

✓ You should now see

As the storekeeper of a request at Deliver: each catalogue line with its approved quantity and the live stock, a line supplied by Juba Country Office says "· from Juba Country Office", and changing a quantity changes the "Stock after" text as you type.

Defects this screen must get right

DEF-05 (the remainder of a partial delivery can be distributed) · DEF-06 (each line that failed is listed) · DEF-112 (the lending office is named) · DEF-118 and DEF-126 (the threshold wording) · DEF-119 (the timeline gives this step's quantity and the running total) · DEF-122 (the hand-over to the requester writes a Routed row) · DEF-144 (no zero stock while loading)

10. Confirm receipt — scrReceipt

The requester records what actually arrived, line by line. When everything approved has been delivered, confirming closes the request, even if something arrived short or damaged; the shortfall is kept on the line.

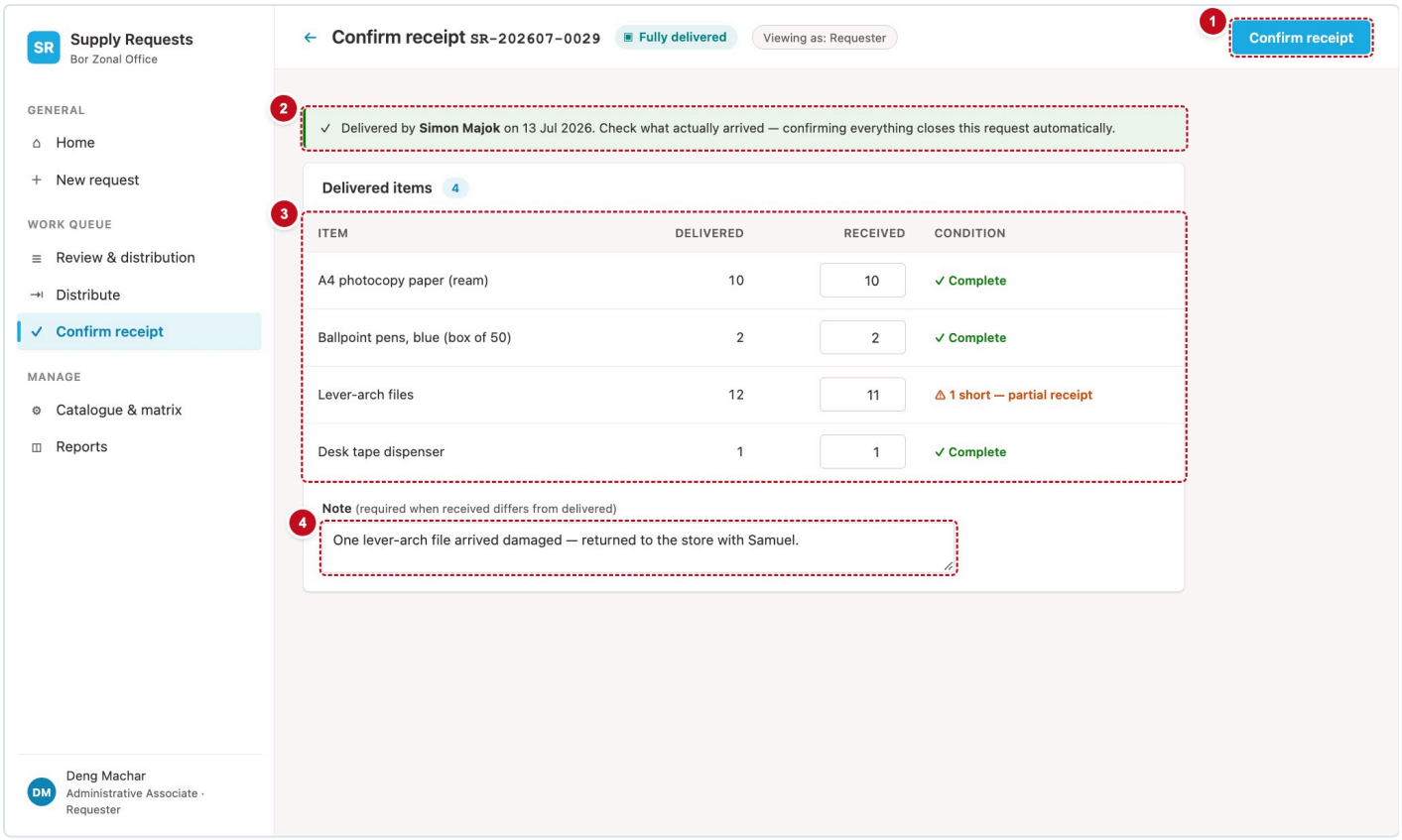


Figure 10. Confirm receipt, as the mock-up shows it (receipt.html)

#	Part	Controls	What it does
1	Confirm receipt	`btnConfirmReceipt`	Saves what arrived and closes the request (§13)
2	Delivery note	`lblDeliveredBy` (in a success callout)	Who delivered it and when; confirming everything closes the request
3	Delivered items	`galReceipt`	Per line: delivered, received, condition
4	Note	`txtReceiptNote`	Recommended when something is short; saved on the lines that aren't Complete

Load the request

```
// scrReceipt.OnVisible
Set(varNavActive, "receipt");
Set(varErrorMsg, "");
Select(btnLoad_Receipt)

// btnLoad_Receipt.OnSelect  (Visible: false)
Set(varReq, LookUp(Requests, Title = varRef));
```

```
Set(varPageTitle, "Confirm receipt " & varRef);
ClearCollect(colLines, Sort(Filter(RequestItems, RequestRefNo = varRef), LineNo));
// Recipe R3 — who may act (no chain is needed on this screen)
Set(varCanAct, And(varReq.StatusRank < 70,
    Or(!IsBlank(varReq.CurrentlyWithEmail)    && Lower(varReq.CurrentlyWithEmail)    = varUserEmail,
       !IsBlank(varReq.CurrentlyWithOICEmail) && Lower(varReq.CurrentlyWithOICEmail) = varUserEmail)));
Set(varIsOriginator, Lower(varReq.RequesterEmail) = varUserEmail);
ClearCollect(colTimeline, SortByColumns(Filter(RequestTimeline, RequestRefNo = varRef), "OccurredOn",
SortOrder.Ascending))
```

The screen

```
// lblPersona.Text
"Viewing as: Requester"

// lblDeliveredBy.Text — the first "Distributed" row
With({ ev: First(Filter(colTimeline, EventType.Value = "Distributed")) },
    If(IsBlank(ev), "",
       "Delivered by " & With({ E: ev.ActorEmail }, Proper(Substitute(Left(E, Find("@", E & "@") - 1), ".", "
")))
       & " on " & Text(ev.OccurredOn, "d mmm yyyy") & ". ")
) & "Confirming everything closes this request."

// galReceipt.Items — the lines that had something delivered
Filter(colLines, Coalesce(QtyDistributed, 0) > 0)
```

Column	Formula
Item	`ThisItem.ItemName`
Delivered	`Coalesce(ThisItem.QtyDistributed, 0)`
Received	`txtReceived` (Classic TextInput, number, `Width: =dimQtyBox`, `Default: =Coalesce(ThisItem.QtyDistributed, 0)`)
Condition	`ddCondition` (Classic DropDown, `Items: =tblConditions`), `Default` below

```
// ddCondition.Default — set from the number received; the user can still change it
With(
    { rec: Min(Max(Coalesce(Value(txtReceived.Text), 0), 0), Coalesce(ThisItem.QtyDistributed, 0)) },
    If(rec <= 0, "Not received", rec < Coalesce(ThisItem.QtyDistributed, 0), "Short", "Complete")
)

// lblShortHint.Visible / .Text — under the note box
!IsEmpty(Filter(galReceipt.AllItems, Coalesce(Value(txtReceived.Text), 0) < Coalesce(QtyDistributed, 0)))
"Something arrived short. Add a note saying what is missing, so the storekeeper can follow it up."

// btnConfirmReceipt.DisplayMode — only the requester, while they hold it
If(varCanAct && varIsOriginator && !varBusy && !IsEmpty(galReceipt.AllItems), DisplayMode.Edit,
DisplayMode.Disabled)
// btnConfirmReceipt.OnSelect — §13, chapter 13
```

> **The note is recommended, not required**
>
> The mock-up labels the note "required" when something is short. The current app only recommends it, and saves it on each line whose condition isn't Complete. This guide does the same.

✓ You should now see

As the requester of a Fully delivered request: every delivered line with its quantity filled in and the condition "Complete". Change a received quantity to less than delivered and the condition turns to "Short" and the hint appears.

Defects this screen must get right

DEF-06 (a failed save says so) · DEF-123 (the receipt timeline row says how many of how many arrived) · DEF-134 (Home's Confirm receipt opens this screen, not Request detail)

11. Catalogue & matrix — scrAdmin

The administrator's screen, with three tabs: the **approval matrix** (who acts at each stage, per office), the **catalogue and stock**, and **offices and sharing** (which offices exist, and which lend stock to which).

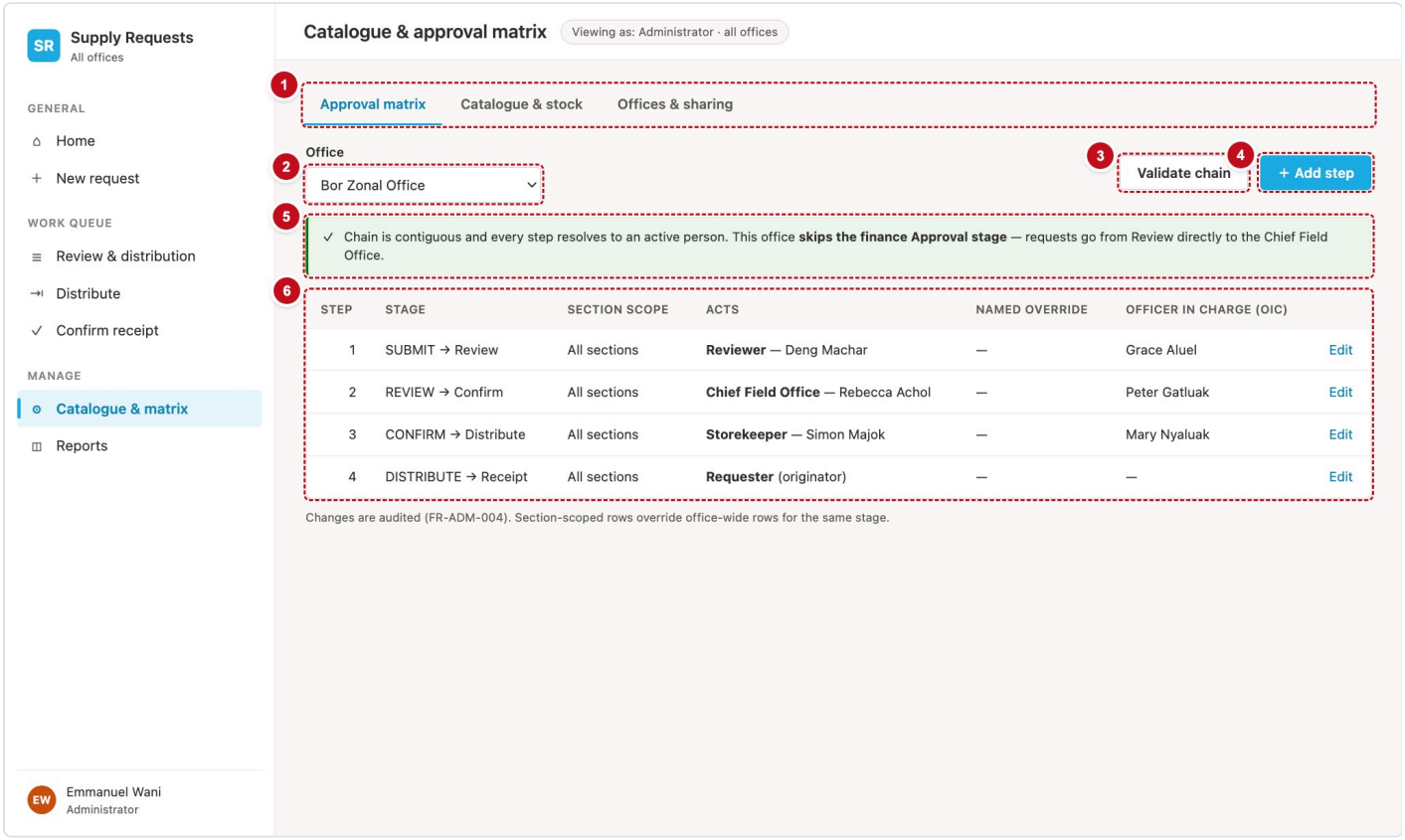


Figure 11. Tab 1, Approval matrix (admin.html)

#	Part	Controls	What it does
1	Tabs	`btnTabMatrix`, `btnTabCatalogue`, `btnTabOffices`	Set `varAdminTab`
2	Office	`ddAdminOffice`	The office the first two tabs show
3	Validate chain	`btnValidateChain`	Checks every stage resolves to a person (§15.2)
4	Add step	`btnAddStep`	Opens the add-step dialog (§15.1)
5	Chain result	`lblChain` (in a callout)	Green when valid, red with the problem steps when not
6	Steps	`galMatrix`	One row per active matrix row, in step order

Load the screen

```
// scrAdmin.OnVisible
Set(varNavActive, "admin");
Set(varPageTitle, "Catalogue & approval matrix");
Set(varAdminTab, Coalesce(varAdminTab, "matrix"));
```

```
Set(varAdminOffice, Coalesce(varAdminOffice, varMyOffice, First(Filter(colOffices, IsActive <> false)).Title));
Set(varChainMsg, "");
Select(btnLoad_AdminCatalogue)

// btnLoad_AdminCatalogue.OnSelect  (Visible: false) — every item, including discontinued ones (DEF-11)
ClearCollect(colAdminCat, Sort(Filter(Catalogue, OfficeCode = varAdminOffice), Title));
// The office-wide chain, for the step labels (Recipe R1 with S = "All sections")
ClearCollect(
    colAdminChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With({ row: LookUp(colMatrix, MatrixKey = varAdminOffice & "|All sections"
                                && StageCode.Value = leg.StageCode) },
            { Leg: leg.Leg, StageCode: leg.StageCode, LegLabel: leg.LegLabel, Live: !IsBlank(row) })
    )
)

// lblPersona.Text
"Viewing as: Administrator · all offices"

// ddAdminOffice — Items, Value, Default, OnChange
Filter(colOffices, IsActive <> false)
"OfficeName"
LookUp(colOffices, Title = varAdminOffice).OfficeName
Set(varAdminOffice, Self.Selected.Title); Set(varChainMsg, ""); Select(btnLoad_AdminCatalogue)
```

Each tab's controls have `Visible: =varAdminTab = "matrix"` (or `"catalogue"`, `"offices"`). The tab buttons set it and are styled from it: `Color: =If(varAdminTab = "matrix", clrBrandDark, clrInk2)` and a 2-pixel cyan bar under the active one.

> **Who can open this screen**
>
> The current app lets every user open and change this screen (DEF-135, by design for now). If the product owner wants it for administrators only, set the menu item's visibility in `galNavManage` to `ThisItem.Target <> "admin" || varIsAdmin`, and in `scrAdmin.OnVisible` add `If(!varIsAdmin, Navigate(scrHome))` as the first line.

Tab 1 — Approval matrix

`galMatrix.Items`:

```
Sort(Filter(colMatrix, OfficeCode = varAdminOffice), StageOrder)
```

Column	Formula
Step	`ThisItem.StageOrder`
Stage	below: derived from the stages before it, never stored
Section scope	`ThisItem.Section`
Acts	`ThisItem.ActorRoleCode.Value & If(ThisItem.StageCode.Value = "Receive", " (originator)", If(IsBlank(ThisItem.ApproverEmail), "", " · " & ThisItem.ApproverEmail))`
Named override	`txtNoHolder` (the current app shows "—": there is no column for it)

Column	Formula
OIC	`Coalesce(ThisItem.OICEmail, txtNoHolder)`
Edit	`btnEditStep`, below

```
// lblStage.Text — "REVIEW → Confirm", "SUBMIT → Review" (src/domain/stages.ts stageTransitionLabel)
With(
    { myLeg: LookUp(tblJourney, StageCode = ThisItem.StageCode.Value).Leg },
    With(
        { prev: Last(Sort(Filter(colAdminChain, Live, Leg < myLeg), Leg)) },
        If(IsBlank(prev), "SUBMIT", Upper(prev.LegLabel)) & " → " & LookUp(tblJourney, Leg = myLeg).LegLabel
    )
)

// btnEditStep.OnSelect — the dialog starts from this row (DEF-105)
Set(varEditStep, ThisItem);
Reset(txtEditSection); Reset(ddEditRole); Reset(txtEditApprover); Reset(txtEditOIC); Reset(txtEditOrder);
Set(varShowEditStep, true)

// btnAddStep.OnSelect — the dialog starts blank every time (DEF-105)
Reset(ddStepStage); Reset(txtStepSection); Reset(ddStepRole); Reset(txtStepApprover); Reset(txtStepOIC);
Reset(txtStepOrder);
Set(varShowAddStep, true)

// lblChain — the validation callout
//   .Text     varChainMsg
//   .Visible  !IsBlank(varChainMsg)
//   .Color    If(varChainOk, clrStReceived, clrStRejected)   (and the callout's fill: clrStReceivedBg /
clrStRejectedBg)
```

The add-step dialog has `ddStepStage` (Items `["Review", "Approval", "Confirm", "Deliver", "Receive"]`), `txtStepSection` (Default `"All sections"`), `ddStepRole` (Items `tblRoles`), `txtStepApprover`, `txtStepOIC` and `txtStepOrder` (number, Default `CountRows(Filter(colMatrix, OfficeCode = varAdminOffice)) + 1`), with **Save** (`btnSaveNewStep`, §15.1). **The edit dialog** has the same fields except the stage, with each `Default` from `varEditStep` (for example `txtEditApprover.Default: =varEditStep.ApproverEmail`), and **Save** (`btnSaveStep`, §15.1). Build both dialogs the way chapter 8 builds the reason dialog: a shade, a panel and the fields, last in the Tree view, shown by their variable.

Tab 2 — Catalogue & stock

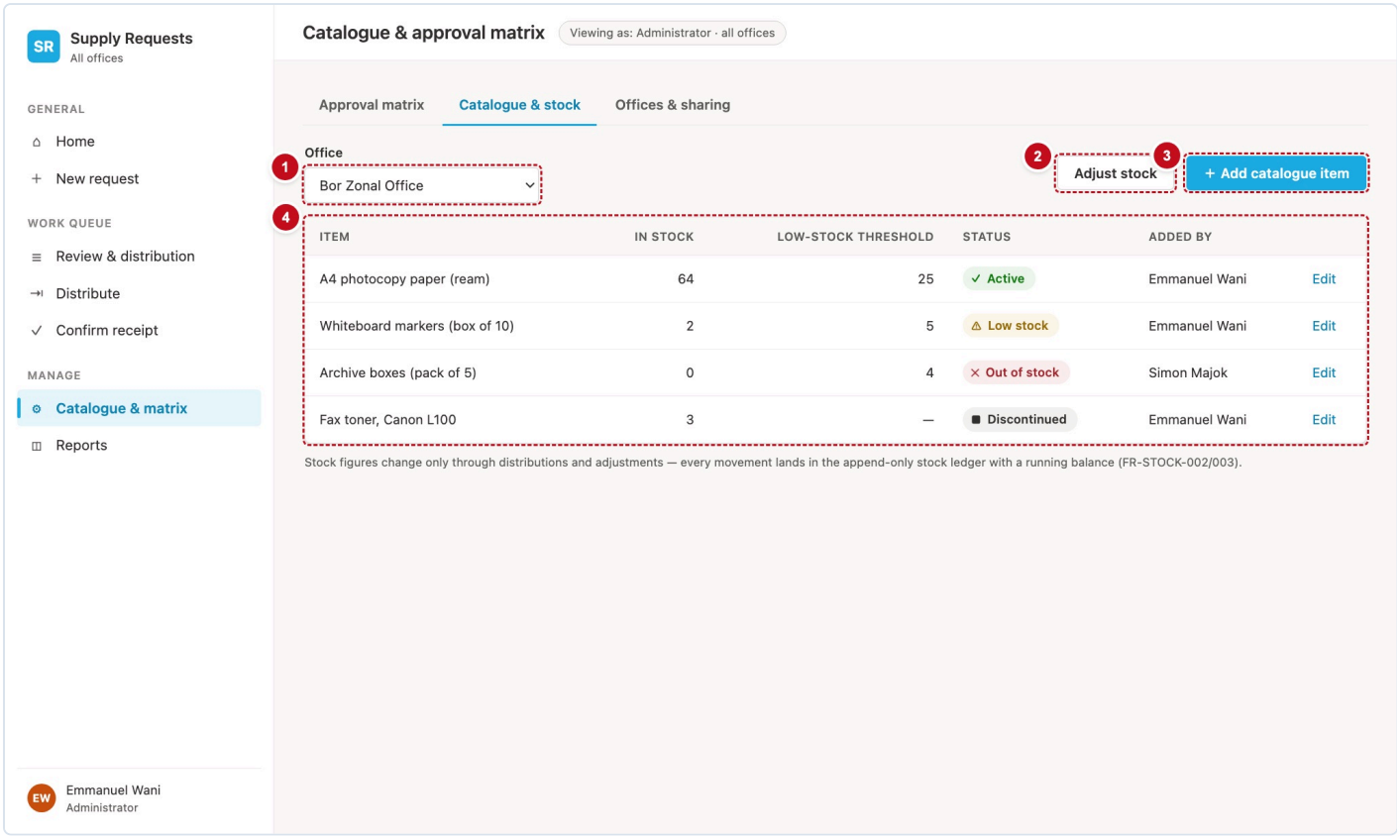


Figure 12. Tab 2, Catalogue & stock

Markers: **1** the office (the same `ddAdminOffice`), **2 Adjust stock** (`btnAdjust`), **3 + Add catalogue item** (`btnAddItem`), **4** the items (`galCatalogue`).

`galCatalogue.Items: =colAdminCat`.

Column	Formula
Item	`ThisItem.Title` with `ThisItem.ItemCode` beneath in `clrInk2`
In stock	`ThisItem.StockQty`
Low-stock threshold	`Coalesce(Text(ThisItem.ReorderLevel), txtNoHolder)`
Status	the stock badge, below
Category	`Coalesce(ThisItem.Category, txtNoHolder)`
Edit	`btnEditItem`: `Set(varEditItem, ThisItem); Reset(txtEditItemName); …; Set(varShowEditItem, true)`

```
// The stock badge: derived, never stored (schema.md §3; src/domain/status.ts stockStateOf)
// lblStockState.Text
With({ st: If(!ThisItem.IsActive, "Discontinued",
              ThisItem.StockQty = 0, "Out of stock",
              !IsBlank(ThisItem.ReorderLevel) && ThisItem.StockQty <= ThisItem.ReorderLevel, "Low stock",
              "Active") },
     LookUp(tblStockState, StateText = st).Glyph & "  " & st)
// lblStockState.Color — the same With(...), then
//   LookUp(tblStockState, StateText = st).Fg      (and .Bg for the pill's Fill)
```

Adjust stock opens a dialog with `ddAdjustItem` (`Items: =Filter(colAdminCat, IsActive)`, `Value: "Title"`), `txtAdjustQty` (a signed number: `-3` takes three off) and `txtAdjustReason`, all required, and **Save** (`btnSaveAdjustment`, §15.3). **Add catalogue item** opens a dialog with name, code, category, unit, reorder level and unit cost (`btnSaveNewItem`, §15.3); stock always starts at 0, and stock is never edited directly, only adjusted. **Edit** has the same fields except the code, plus an **Active** toggle `tglEditItemActive` (`btnSaveItem`, §15.3).

Tab 3 — Offices & sharing

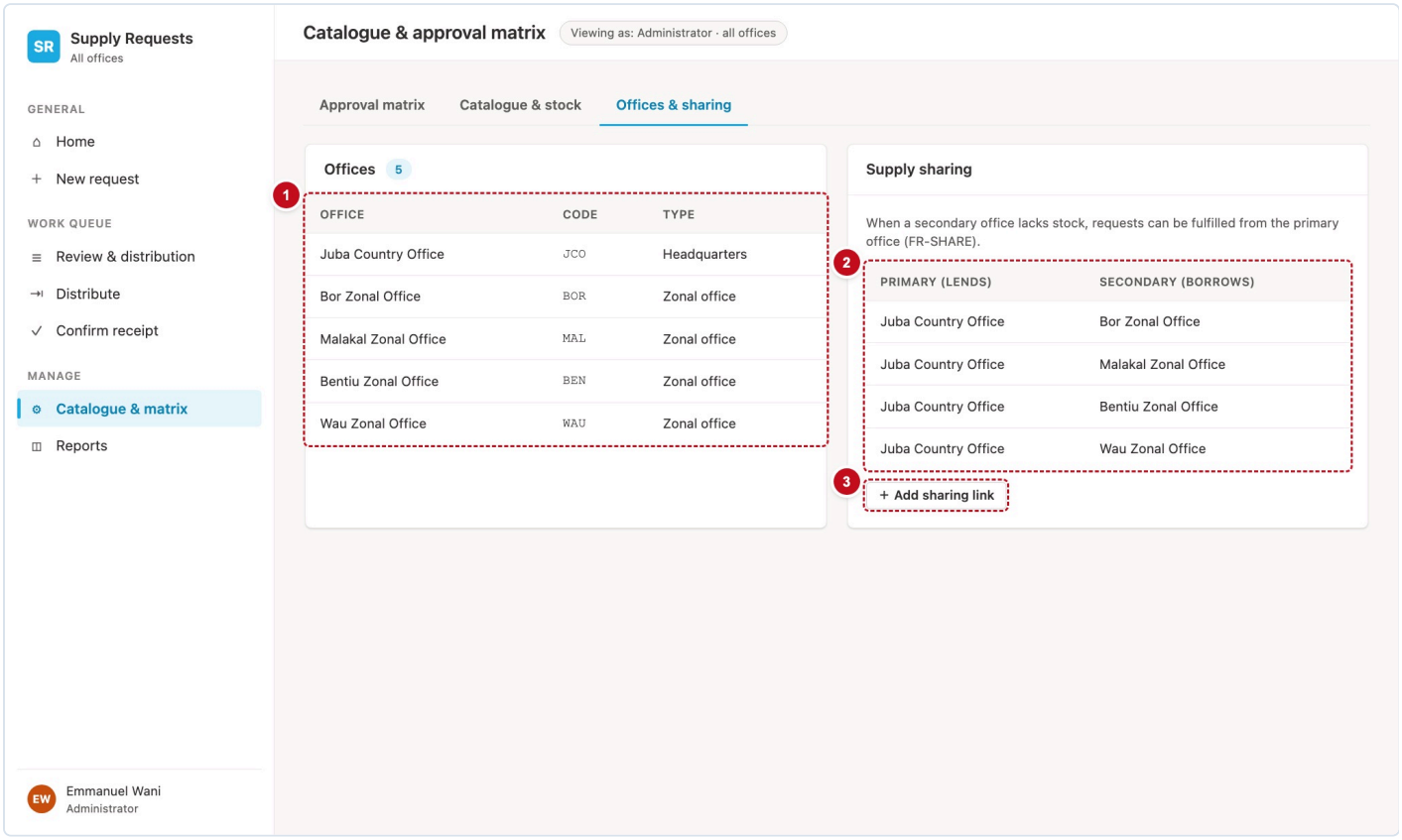


Figure 13. Tab 3, Offices & sharing

Markers: **1** the offices (`galOffices`), **2** the sharing links (`galSharing`), **3** **+ Add office** (`btnAddOffice`).

```
// galOffices.Items — every office, active or not (DEF-14)
Sort(colOffices, OfficeName)
// columns: OfficeName · Title (the code) · OfficeType.Value · If(ThisItem.IsActive <> false, "Active",
"Inactive")

// btnAddOffice.OnSelect
Set(varEditOffice, Blank()); Set(varCodeTouched, false); Set(varFrozenCode, "");
Reset(txtOfficeName); Reset(ddOfficeType); Reset(txtOfficeCountry); Reset(tglOfficeActive);
Reset(txtOfficeCode);
Set(varShowOfficeDialog, true)

// btnEditOffice.OnSelect (in the gallery)
Set(varEditOffice, ThisItem); Set(varCodeTouched, true); Set(varFrozenCode, ThisItem.Title);
Reset(txtOfficeName); Reset(ddOfficeType); Reset(txtOfficeCountry); Reset(tglOfficeActive);
Reset(txtOfficeCode);
Set(varShowOfficeDialog, true)

// galSharing.Items — the active links
colSharing
// columns: "From: " & LookUp(colOffices, Title = ThisItem.FromOfficeCode).OfficeName & " lends"
```

```
//          "To: " & LookUp(colOffices, Title = ThisItem.ToOfficeCode).OfficeName & " borrows"
//          ThisItem.SharePriority
```

The office dialog serves both Add and Edit. Its fields: `txtOfficeName` (Default `varEditOffice.OfficeName`), `ddOfficeType` (Items `tblOfficeTypes`, Default `Coalesce(varEditOffice.OfficeType.Value, "Field office")`), `txtOfficeCountry` (Default `Coalesce(varEditOffice.Country, "South Sudan")`), `tglOfficeActive` (Default `Coalesce(varEditOffice.IsActive, true)`), and the code field, which suggests a code as you type the name and stops following it once you type over it. The code-field formulas, `lblCodeSuggestion`, `lblOfficeCodeError` and both **Save** buttons are in §15.4 and §15.5. Show `btnSaveNewOffice` when `IsBlank(varEditOffice)` and `btnSaveOffice` otherwise.

> ⚠ **Renaming an office code**
>
> Saving an office with a new code rewrites that code in seven lists (§15.5). It takes a while on a busy office, and it stops at the first list that fails, so it is safe to run again. Tell users not to work in that office while it runs.

Add sharing link opens a dialog with `ddShareFrom` and `ddShareTo` (Items: active offices), `txtSharePriority` (Default `1`), and **Save** (`btnSaveNewSharing`, §15.4). The current app has no edit or delete for sharing links; switch a link off by setting `IsActive` to No in SharePoint.

> ✓ **You should now see**
>
> For Bor: four steps (Review, Confirm, Deliver, Receive) labelled "SUBMIT → Review", "REVIEW → Confirm", "CONFIRM → Deliver" and "DELIVER → Receive & close". **Validate chain** says the chain is valid and that this office skips the Approval stage.

> **Defects this screen must get right**
>
> DEF-11 (a discontinued item can be switched back on) · DEF-14 (inactive offices are listed here, hidden from pickers) · DEF-105 (each dialog opens blank) · DEF-135 (by design: any role can open this screen) · DEF-139 (the Active hint in the office dialog matches the toggle)

12. Reports — scrReports

Reports summarise activity for a period and an office: how many requests came in, how long delivery takes, how often requests are sent back, which stage holds requests longest, and which items are used most.

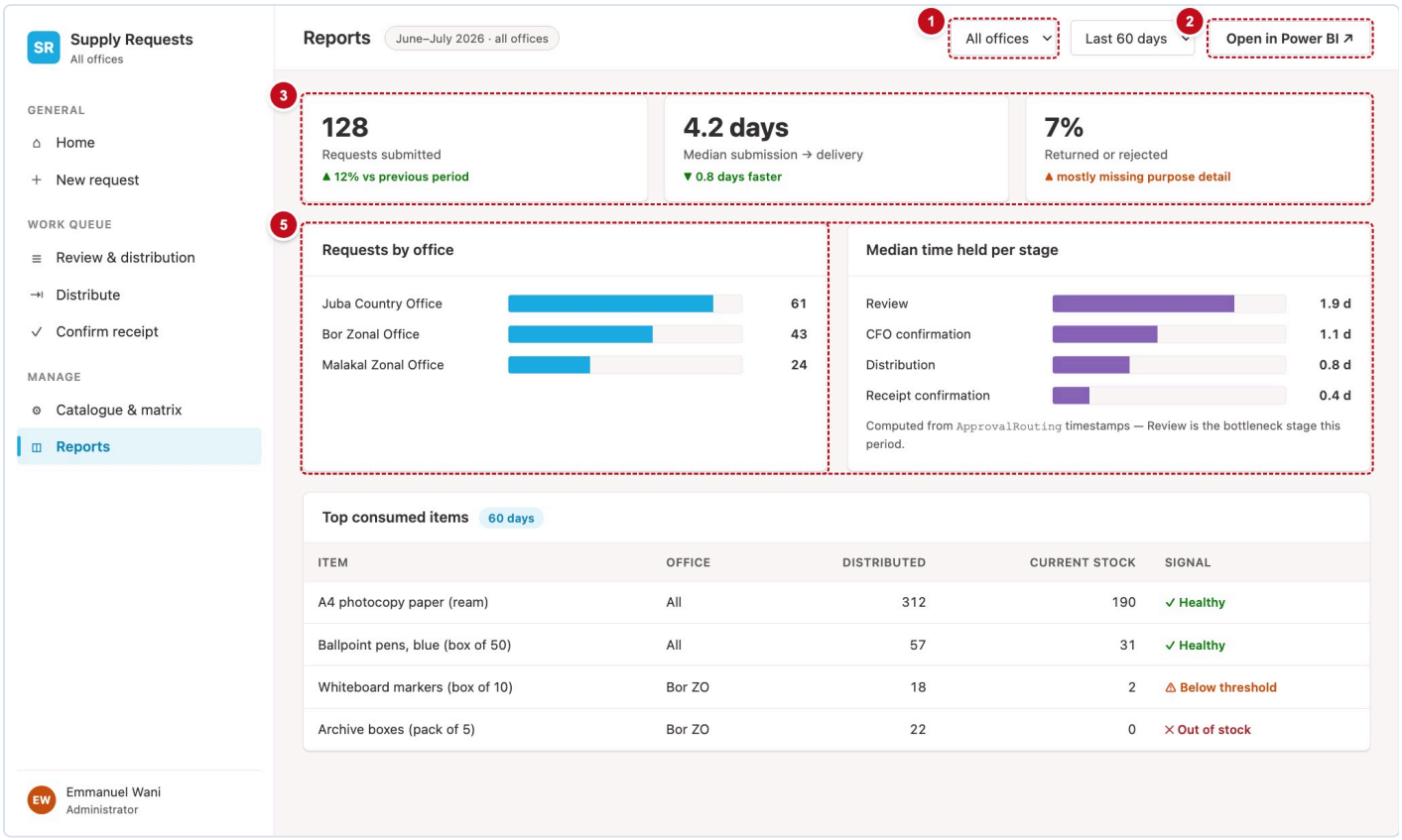


Figure 14. Reports, as the mock-up shows it (reports.html)

#	Part	Controls	What it does
1	Filters	`ddRepOffice`, `ddRepPeriod`	The office (or all) and the period (last 60 days, or this calendar quarter)
2	Power BI	`btnPowerBI`	Disabled: "Not configured yet", as in the current app
3	Tiles	`lblKpiCount`, `lblKpiMedian`, `lblKpiReturned` with their captions	Requests submitted, median submission → delivery, % returned or rejected
4	Charts	`galByOffice`, `galByStage`	Requests by office; median time held per stage
5	Top consumed items	`galTopItems`	The ten items most handed out in the period

How Reports gets its numbers

SharePoint can filter but cannot add up. So Reports fetches a bounded set of rows (at most `numReportCap`, 500, from each list), with filters SharePoint can do, and adds them up in the app. When a list hits the cap, the tile says so, so a cut-off total is never shown as if it were complete (schema.md, "Reports").

```
// scrReports.OnVisible
Set(varNavActive, "reports");
Set(varPageTitle, "Reports");
ClearCollect(colRepOffices, { Code: "", Name: "All offices" });
Collect(colRepOffices, ForAll(Sort(Filter(colOffices, IsActive <> false), OfficeName) As o, { Code: o.Title,
Name: o.OfficeName }));
Reset(ddRepOffice); Reset(ddRepPeriod);
Select(btnLoad_Reports)

// ddRepOffice: Items colRepOffices, Value "Name", Default "All offices", OnChange Select(btnLoad_Reports)
// ddRepPeriod: Items ["Last 60 days", "This quarter"], Default "Last 60 days", OnChange
Select(btnLoad_Reports)
```

```
// btnLoad_Reports.OnSelect  (Visible: false)
Set(varRepOffice, ddRepOffice.Selected.Code);                  // "" = all offices
// DEF-137, DEF-142: "This quarter" is the calendar quarter, from local midnight on its first day.
Set(varSince, If(ddRepPeriod.Selected.Value = "This quarter",
                 DateTime(Year(Today()), RoundDown((Month(Today()) - 1) / 3, 0) * 3 + 1, 1, 0, 0, 0),
                 DateAdd(Now(), -60, TimeUnit.Days)));

// Bounded, delegable reads.
ClearCollect(colRepRequests,
    FirstN(SortByColumns(Filter(Requests, SubmittedOn >= varSince), "SubmittedOn", SortOrder.Descending),
numReportCap));
ClearCollect(colRepLedger,
    FirstN(Filter(StockLedger, MovementType.Value = "Distribution", MovedOn >= varSince), numReportCap));
ClearCollect(colRepTimeline,
    FirstN(Filter(RequestTimeline, OccurredOn >= varSince), numReportCap));

// The office filter, applied in the app to all three (DEF-13, DEF-145). Timeline rows have no
// office, so they follow their request.
ClearCollect(colScopeReq, Filter(colRepRequests, IsBlank(varRepOffice) || OfficeCode = varRepOffice));
ClearCollect(colScopeLedger, Filter(colRepLedger, IsBlank(varRepOffice) || OfficeCode = varRepOffice));
ClearCollect(colScopeTL, Filter(colRepTimeline, IsBlank(varRepOffice) || RequestRefNo in colScopeReq.Title));

// Days from the first "Submitted" row to the first "Distributed" row, per request.
ClearCollect(colDeliveryDays,
    Filter(
        ForAll(Distinct(colScopeTL, RequestRefNo) As r,
            With({
                sub:  First(Sort(Filter(colScopeTL, RequestRefNo = r.Value, EventType.Value = "Submitted"),
OccurredOn)),
                dist: First(Sort(Filter(colScopeTL, RequestRefNo = r.Value, EventType.Value = "Distributed"),
OccurredOn))
            },
            { Days: If(IsBlank(sub) || IsBlank(dist), Blank(),
                       DateDiff(sub.OccurredOn, dist.OccurredOn, TimeUnit.Seconds) / 86400) })),
        !IsBlank(Days)));

// Time held per stage: from the "Routed" row that started it to the first row that ended it
// (src/domain/timeline.ts computeStageDurations; the end events are in tblStageExit).
Clear(colStageDur);
ForAll(tblStageExit As st,
    Collect(colStageDur,
        ForAll(Distinct(colScopeTL, RequestRefNo) As r,
            With({ entry: First(Sort(Filter(colScopeTL, RequestRefNo = r.Value, EventType.Value = "Routed",
                                            StageCode.Value = st.StageCode), OccurredOn)) },
                With({ leave: If(IsBlank(entry), Blank(),
                         First(Sort(Filter(colScopeTL, RequestRefNo = r.Value, StageCode.Value = st.StageCode,
                                           ("|" & EventType.Value & "|") in st.Exits,
                                           OccurredOn >= entry.OccurredOn), OccurredOn))) },
                    { Stage: st.StageCode, Leg: st.Leg, Label: st.ChartLabel,
                      Days: If(IsBlank(leave), Blank(), DateDiff(entry.OccurredOn, leave.OccurredOn,
TimeUnit.Seconds) / 86400) })))));

// The median per stage.
ClearCollect(colStageMedian,
    Filter(
        ForAll(tblStageExit As st,
```

```
            With({ v: Sort(Filter(colStageDur, Stage = st.StageCode, !IsBlank(Days)), Days) },
                With({ n: CountRows(v) },
                    { Stage: st.StageCode, Leg: st.Leg, Label: st.ChartLabel,
                      Median: If(n = 0, Blank(),
                                 Mod(n, 2) = 1, Index(v, (n + 1) / 2).Days,
                                 (Index(v, n / 2).Days + Index(v, n / 2 + 1).Days) / 2) }))),
        !IsBlank(Median)));
// DEF-143: show minutes, hours or days, whichever suits the longest bar.
Set(varStageMax, Max(colStageMedian, Median));
Set(varStageUnit, If(varStageMax < 1 / 24, "min", varStageMax < 1, "h", "d"));
Set(varStagePerDay, Switch(varStageUnit, "min", 1440, "h", 24, 1));

// Requests by office.
ClearCollect(colByOffice,
    SortByColumns(
        ForAll(Distinct(colScopeReq, OfficeCode) As o,
            { Office: o.Value, Name: Coalesce(LookUp(colOffices, Title = o.Value).OfficeName, o.Value),
              Count: CountRows(Filter(colScopeReq, OfficeCode = o.Value)) }),
        "Count", SortOrder.Descending));

// Top consumed items: the units handed out per item, most first, top ten.
ClearCollect(colTopItems,
    FirstN(
        SortByColumns(
            ForAll(Distinct(colScopeLedger, OfficeItemKey) As k,
                With({ rows: Filter(colScopeLedger, OfficeItemKey = k.Value),
                       cat:  LookUp(Catalogue, OfficeItemKey = k.Value) },
                    { Key: k.Value, Item: Coalesce(cat.Title, First(rows).ItemCode), Office:
First(rows).OfficeCode,
                      Distributed: Sum(rows, Abs(QtyDelta)), Stock: cat.StockQty, Reorder: cat.ReorderLevel
})),
            "Distributed", SortOrder.Descending),
        10))
```

`Distinct(…)` returns a one-column table whose column is called `Value`.

Studio puts a delegation warning on each `FirstN`. That one is expected: the `Filter` inside it is sent to SharePoint, and `FirstN` only trims what comes back (chapter 15).

> ⚠ **Watch out**
>
> If your Studio shows `Distinct` results with a column called `Result` instead of `Value`, you are on an older authoring version. Change `.Value` to `.Result` in the formulas above, or update Studio (**Settings › Support › Authoring version**).

The tiles

```
// lblPersona.Text — the period and the office
Text(varSince, "d mmm yyyy") & " – " & Text(Today(), "d mmm yyyy") & " · "
    & If(IsBlank(varRepOffice), "all offices", ddRepOffice.Selected.Name)

// lblKpiCount.Text and its flag
CountRows(colScopeReq)
If(CountRows(colRepRequests) >= numReportCap, "Capped at " & numReportCap & ": the period may hold more", "")

// lblKpiMedian.Text — DEF-140: "< 0.1 days" rather than "0.0 days"
With({ v: Sort(colDeliveryDays, Days), n: CountRows(colDeliveryDays) },
    With({ m: If(n = 0, Blank(), Mod(n, 2) = 1, Index(v, (n + 1) / 2).Days,
                 (Index(v, n / 2).Days + Index(v, n / 2 + 1).Days) / 2) },
        If(IsBlank(m), txtNoHolder, m < 0.05, "< 0.1 days", Text(m, "0.0") & " days")))

// lblKpiReturned.Text
```

```
If(IsEmpty(colScopeReq), "0%",
   Text(Round(CountRows(Filter(colScopeReq, Status.Value in ["Returned", "Rejected"])) / CountRows(colScopeReq)
* 100, 0)) & "%")
```

The charts

Canvas apps have chart controls, but a bar is easier to style as a gallery row with a rounded shape whose width is the value. Each chart is a blank vertical gallery:

```
// galByOffice.Items
colByOffice
// in the template: lblOfficeName (ThisItem.Name), a bar btnOfficeBar (Classic Button shape, Fill clrBrand),
lblOfficeCount
// btnOfficeBar.Width
Max(4, ThisItem.Count / Max(colByOffice, Count) * (Parent.TemplateWidth - 220))

// galByStage.Items
Sort(colStageMedian, Leg)
// btnStageBar.Width
Max(4, ThisItem.Median / varStageMax * (Parent.TemplateWidth - 260))
// lblStageValue.Text
Text(ThisItem.Median * varStagePerDay, "0.0") & " " & varStageUnit

// lblStageCaption.Text — DEF-146: name the stage that is actually slowest
With({ s: First(SortByColumns(colStageMedian, "Median", SortOrder.Descending, "Leg", SortOrder.Ascending)) },
    If(IsBlank(s), "No stage has both a start and an end in this period.",
       s.Label & " holds requests longest: a median of " & Text(s.Median * varStagePerDay, "0.0") & " " &
varStageUnit & "."))
```

Top consumed items

`galTopItems.Items: =colTopItems`.

Column	Formula
Item	`ThisItem.Item`
Office	`Coalesce(LookUp(colOffices, Title = ThisItem.Office).OfficeName, ThisItem.Office)`
Distributed	`ThisItem.Distributed`
Current stock	`Coalesce(Text(ThisItem.Stock), txtNoHolder)`
Signal	the stock badge, with `st: If(IsBlank(ThisItem.Stock), "", ThisItem.Stock = 0, "Out of stock", !IsBlank(ThisItem.Reorder) && ThisItem.Stock <= ThisItem.Reorder, "Low stock", "Active")`

✓ You should now see

For "Last 60 days" and all offices: the request count, a median in days, the returned-or-rejected percentage, one bar per office, one bar per stage in journey order with a caption naming the slowest, and up to ten items. Pick one office and every number changes, including both charts.

Defects this screen must get right

DEF-13 and DEF-145 (every figure follows the office filter) · DEF-137 and DEF-142 ("This quarter" is the calendar quarter from its first day) · DEF-138 (the stage chart finds its start rows, which needs the Routed rows to carry their stage code: §8, §10, §12) · DEF-140 ("< 0.1 days") · DEF-143 (bars in minutes, hours or days) · DEF-146 (the caption names the slowest stage)

13. The buttons that save

This chapter holds every formula that writes to SharePoint. The sections keep the numbers they have in `powerapps/app-glue.md` (§7 to §15), so the file and this guide always agree. The screen chapters point here by those numbers.

Wire them up in this order, testing each against its screen before the next:

Order	Section	Button	Screen
1	§8	`btnCreateCore` (Save draft, Submit)	New request
2	§9	`btnResubmitCore` (Resubmit)	New request, in edit mode
3	§10	`btnProceed`	Review
4	§11	`btnDialogConfirm` (Return, Reject)	Review
5	§12	`btnRecordDistribution`	Distribute
6	§13	`btnConfirmReceipt`	Confirm receipt
7	§14	`btnPostComment`, `btnAttach`	Request detail
8	§15	the dialogs' Save buttons, `btnValidateChain`	Catalogue & matrix

> ⚠ **Before you press any of them**
>
> Raise the data row limit back to 2,000 first (chapter 15). With the limit at 1, a total the formula adds up comes from one row, and the wrong number would be saved.

Each diagram shows what one button writes, in order. A step that fails stops what comes after it, and the button tells the user what was and wasn't saved.

Section 7 — Shared recipes

Sections 7 to 15 were added for the full rebuild. They port the write side of the SPFx build (`pa.init.officesupplies/src/hooks/*.ts`) and the rules behind it (`src/domain/*.ts`) to Power Fx, including every fix from UAT round 2. Where a section replaces an earlier one, it says so: §8 replaces §6, §12 replaces §5.3.

The recipes below are used by more than one screen. Power Fx in a canvas app has no shared functions you can rely on in this tenant, so each recipe is **copied** into the formula that needs it. Where a section says "Recipe R1 here", paste the recipe's text at that point with the inputs named.

Rules every write in this app follows.

Rule	Why
Write a Choice column as `{ Value: "…" }`, clear it with `Blank()`	The SharePoint connector needs the record shape. `Blank()` really clears the column; the SPFx build sent `undefined`, which left the old stage code in place (the root of DEF-124/127/128/141).

Rule	Why
`Status` and `StatusRank` always go in the same `Patch`	Every query filters on `StatusRank`; a rank that disagrees with the status hides the request.
Store and compare emails lower-case	DEF-09. SharePoint filters ignore case; Power Fx `=` does not.
Leave Person columns blank; write the `…Email` shadow	The seeded people have no accounts, and every filter keys on the shadow (schema.md rule 3).
`varBusy` around every multi-step write	A second click while the first is still writing would write twice. `DisplayMode: =If(varBusy, DisplayMode.Disabled, DisplayMode.Edit)` on the button.
Never wrap dependent writes in `Concurrent()`	`Concurrent` really does run in parallel.
Catch every write with `IfError` and tell the user	DEF-06: "Failed saves are mostly silent".

Recipe R1 — build the approval chain for an office and section

Inputs: **O** = office code (text), **S** = section (text). Output: the collection `colChain`, one row per stage in journey order, `Live` true when the office has an active matrix row for it. A section-scoped row beats the office's "All sections" row (`src/domain/matrix.ts`, `resolveMatrixRow`). A stage with no row is skipped (schema.md, "The skip rule is the absence of a row").

```
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = O & "|" & S
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = O & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With(
                { row: If(IsBlank(rowSec), rowAll, rowSec) },
                {
                    Leg:        leg.Leg,
                    StageCode:  leg.StageCode,
                    LegLabel:   leg.LegLabel,
                    ActorLabel: leg.ActorLabel,
                    Live:       !IsBlank(row),
                    StageOrder: row.StageOrder,
                    Role:       row.ActorRoleCode.Value,
                    Approver:   Lower(Trim(row.ApproverEmail)),
                    OIC:        Lower(Trim(row.OICEmail))
                }
            )
        )
    )
)
```

`leg.StageCode` is the journey's text code; the bare `StageCode` inside each `LookUp` is the matrix row's Choice column. The `As leg` alias is what keeps the two apart (open question 8 at the end of this file).

Recipe R2 — where a request goes next

Inputs: **F** = the stage being left (text, `""` at submission), **R** = the requester's email, lower-case. Needs `colChain` from R1. Output: a record `{ NextStage, WithEmail, OICEmail }`, stored with `Set(varRoute, …)`.

Ported from `src/domain/routing.ts`:

- the next stage is the first live stage after **F**, in journey order;
- if there is none, or it is Receive, the request goes to the requester at Receive;
- otherwise the matrix approver holds it and the officer in charge (OIC) holds it beside them (DEF-103: the OIC is a second holder, not a replacement);
- a stage with nobody configured falls back to the requester, so a request is never orphaned.

```
Set(
    varRoute,
    With(
        { fromLeg: Coalesce(LookUp(tblJourney, StageCode = F).Leg, 1) },
        With(
            { next: First(Sort(Filter(colChain, Live, Leg > fromLeg), Leg)) },
            If(
                IsBlank(next) || next.StageCode = "Receive",
                { NextStage: "Receive", WithEmail: R, OICEmail: "" },
                With(
                    { primary: Coalesce(next.Approver, next.OIC) },
                    {
                        NextStage: next.StageCode,
                        WithEmail: Coalesce(primary, R),
                        OICEmail:  If(!IsBlank(next.OIC) && next.OIC <> primary, next.OIC, "")
                    }
                )
            )
        )
    )
)
```

Recipe R3 — who may act on the open request

Run after `Set(varReq, LookUp(Requests, Title = varRef))`. Ported from `src/domain/roles.ts` (`canActOnRequest`, `isOriginator`) and `src/domain/stageAction.ts` (`isEditableStatus`, `isReviewableStatus`).

```
Set(
    varCanAct,
    And(
        varReq.StatusRank < 70,
        Or(
            !IsBlank(varReq.CurrentlyWithEmail)    && Lower(varReq.CurrentlyWithEmail)    = varUserEmail,
            !IsBlank(varReq.CurrentlyWithOICEmail) && Lower(varReq.CurrentlyWithOICEmail) = varUserEmail
        )
    )
);
Set(varIsOriginator, Lower(varReq.RequesterEmail) = varUserEmail);
Set(varIsEditable,   varReq.Status.Value in ["Draft", "Returned"]);     // DEF-124
Set(varIsReviewable, varReq.Status.Value in ["Submitted", "Reviewed"]);
Set(varStage,        If(varIsEditable, "", varReq.CurrentStageCode.Value))
```

`varStage` treats a Draft or Returned request as having no stage, whatever the column still holds. That keeps old rows written by the SPFx build (which never cleared the column) showing correctly: DEF-127 and DEF-141.

Recipe R4 — a name from an email

The Person columns are blank by design, so a name is made from the email: `deng.machar@contoso.org` → `Deng Machar`. **E** = the email.

```
If(
    IsBlank(E), txtNoHolder,
    IsBlank(Find("@", E)), E,
    Proper(Substitute(Substitute(Left(E, Find("@", E) - 1), ".", " "), "_", " "))
)
```

Recipe R5 — "held for"

DEF-132: the clock starts at the request's last change (`Modified`), falling back to `SubmittedOn`, so a hand-off restarts it. **Q** = the request record.

```
With(
    { h: DateDiff(Coalesce(Q.Modified, Q.SubmittedOn), Now(), TimeUnit.Hours) },
    If(
        h < 1,  "under an hour",
        h < 24, Text(h) & If(h = 1, " hour", " hours"),
        With({ d: RoundDown(h / 24, 0) }, Text(d) & If(d = 1, " day", " days"))
    )
)
```

Overdue (red, and counted in Home's tile flag) when `DateDiff(Coalesce(Q.Modified, Q.SubmittedOn), Now(), TimeUnit.Hours) > numOverdueHours`.

Recipe R6 — the request screens' loader

Every screen that shows one request (`scrRequestDetail`, `scrReview`, `scrDistribute`, `scrReceipt`) loads it from a hidden button called `btnLoad_<screen>`, and its `OnVisible` is `Select(btnLoad_<screen>)`. A write that leaves the user on the screen ends with the same `Select(...)`, so the numbers on screen are re-read from SharePoint rather than patched by hand. Every loader starts with:

```
Set(varReq, LookUp(Requests, Title = varRef));
ClearCollect(colLines, Sort(Filter(RequestItems, RequestRefNo = varRef), LineNo));
// Recipe R1 — the approval chain for varReq.OfficeCode / varReq.Section.Value
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|" & varReq.Section.Value
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = varReq.OfficeCode & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With(
                { row: If(IsBlank(rowSec), rowAll, rowSec) },
                {
                    Leg:        leg.Leg,
                    StageCode:  leg.StageCode,
                    LegLabel:   leg.LegLabel,
                    ActorLabel: leg.ActorLabel,
                    Live:       !IsBlank(row),
                    StageOrder: row.StageOrder,
                    Role:       row.ActorRoleCode.Value,
```

```
                    Approver:   Lower(Trim(row.ApproverEmail)),
                    OIC:        Lower(Trim(row.OICEmail))
                }
            )
        )
    )
);
// Recipe R3 — who may act
Set(
    varCanAct,
    And(
        varReq.StatusRank < 70,
        Or(
            !IsBlank(varReq.CurrentlyWithEmail)    && Lower(varReq.CurrentlyWithEmail)    = varUserEmail,
            !IsBlank(varReq.CurrentlyWithOICEmail) && Lower(varReq.CurrentlyWithOICEmail) = varUserEmail
        )
    )
);
Set(varIsOriginator, Lower(varReq.RequesterEmail) = varUserEmail);
Set(varIsEditable,   varReq.Status.Value in ["Draft", "Returned"]);     // DEF-124
Set(varIsReviewable, varReq.Status.Value in ["Submitted", "Reviewed"]);
Set(varStage,        If(varIsEditable, "", varReq.CurrentStageCode.Value));
```

Each screen's chapter gives its full loader, with these lines already in it.

A hidden button: Insert → Button (Classic), name it, set `Visible: =false`. If your tenant ignores `Select()` on a hidden control, leave it visible with `Width: =0` and `Height: =0` instead.

Section 8 — Save draft and Submit (a new request)

Replaces Section 6. Section 6's retry loop puts `Set()` inside `ForAll`, which Power Apps does not allow, and `ForAll` has no guaranteed order. This version writes the five attempts out one after another, so they run strictly in sequence and stop at the first success. Ported from `src/hooks/useNewRequest.ts` (`useSubmitNewRequest`).

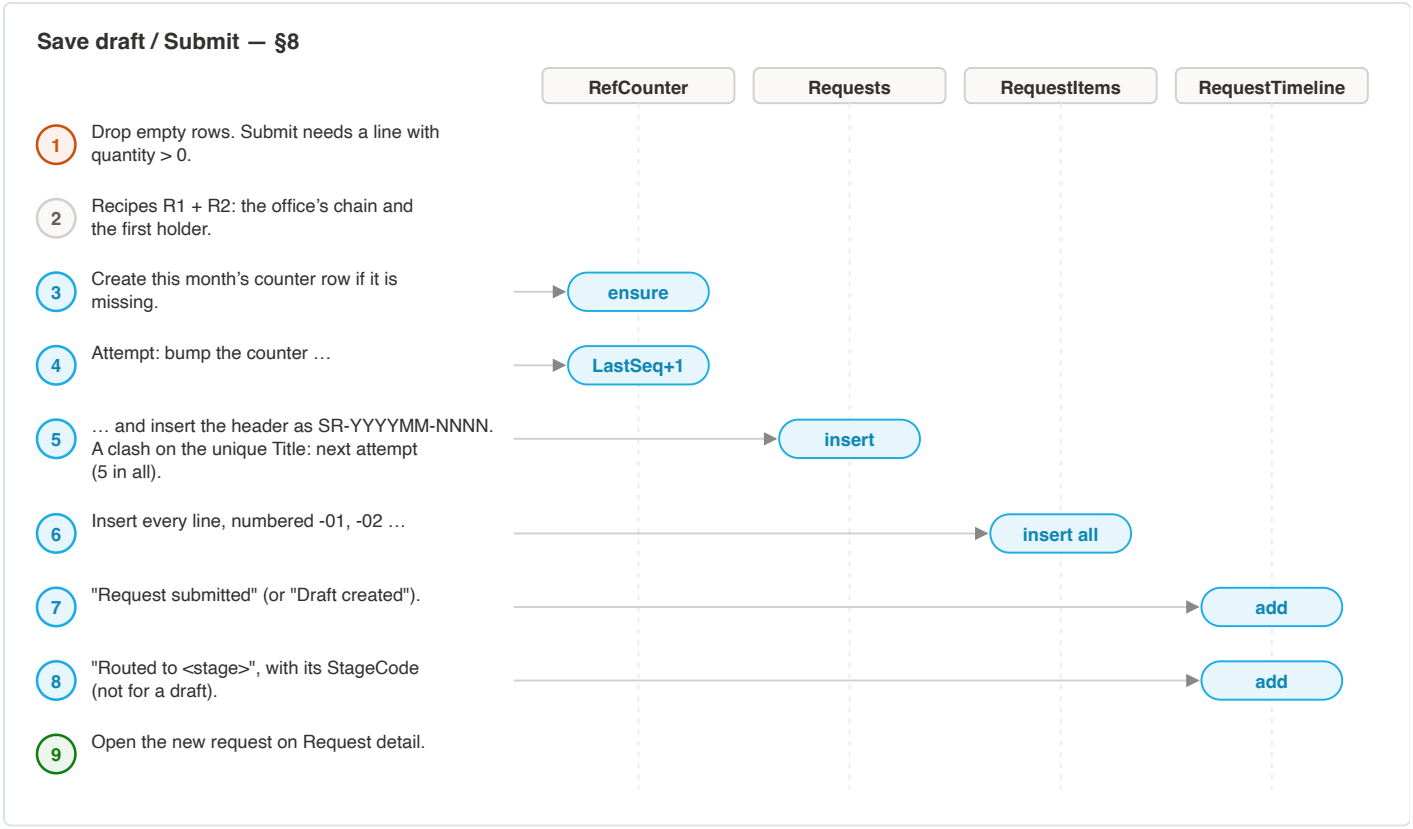


Figure 15. Save draft and Submit: what is written, in order

Where it goes. `scrNewRequest` has two visible buttons and one hidden one.

```
// btnSaveDraft.OnSelect   (shown only for a new request: Visible = IsBlank(varEditRef))
Set(varAsDraft, true);  Select(btnCreateCore)

// btnSubmit.OnSelect      (Text: =If(IsBlank(varEditRef), "Submit request", "Resubmit request"))
Set(varAsDraft, false); If(IsBlank(varEditRef), Select(btnCreateCore), Select(btnResubmitCore))
```

Inputs. `ddOffice.Selected.Title`, `ddSection.Selected.Value`, `ddPurpose.Selected.Value`, and `colLines`, the rows of the Items card. Each `colLines` row is `{ RowId, IsOther, ItemCode, ItemName, IsCatalogueItem, OfficeItemKey, UnitOfMeasure, UnitCost, LinePurpose, LineNote, QtyRequested }` (see the scrNewRequest chapter).

`btnCreateCore.OnSelect`

```
If(varBusy, false,
Set(varBusy, true);
Set(varErrorMsg, "");
Set(varNewRef, "");
Set(varFormOffice,  ddOffice.Selected.Title);
Set(varFormSection, ddSection.Selected.Value);
Set(varFormPurpose, ddPurpose.Selected.Value);
Set(varNewStatus,   If(varAsDraft, "Draft", "Submitted"));

// Rows with nothing picked and nothing typed are dropped, as in NewRequestPage.tsx.
ClearCollect(colSave, Filter(colLines, !IsBlank(Trim(ItemName))));

If(
    !varAsDraft && IsEmpty(Filter(colSave, QtyRequested > 0)),
    Set(varErrorMsg, "Add at least one item with a quantity greater than zero before submitting."),

    IsEmpty(colSave),
    Set(varErrorMsg, "Add at least one item before saving."),
```

```
// ── Route it (Recipes R1 and R2; a draft keeps no route) ──────────────
// Recipe R1 — the approval chain for varFormOffice / varFormSection
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = varFormOffice & "|" & varFormSection
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = varFormOffice & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With(
                { row: If(IsBlank(rowSec), rowAll, rowSec) },
                {
                    Leg:        leg.Leg,
                    StageCode:  leg.StageCode,
                    LegLabel:   leg.LegLabel,
                    ActorLabel: leg.ActorLabel,
                    Live:       !IsBlank(row),
                    StageOrder: row.StageOrder,
                    Role:       row.ActorRoleCode.Value,
                    Approver:   Lower(Trim(row.ApproverEmail)),
                    OIC:        Lower(Trim(row.OICEmail))
                }
            )
        )
    )
);
// Recipe R2 — where it goes next (leaving nothing: first route)
Set(
    varRoute,
    With(
        { fromLeg: Coalesce(LookUp(tblJourney, StageCode = "").Leg, 1) },
        With(
            { next: First(Sort(Filter(colChain, Live, Leg > fromLeg), Leg)) },
            If(
                IsBlank(next) || next.StageCode = "Receive",
                { NextStage: "Receive", WithEmail: varUserEmail, OICEmail: "" },
                With(
                    { primary: Coalesce(next.Approver, next.OIC) },
                    {
                        NextStage: next.StageCode,
                        WithEmail: Coalesce(primary, varUserEmail),
                        OICEmail:  If(!IsBlank(next.OIC) && next.OIC <> primary, next.OIC, "")
                    }
                )
            )
        )
    )
);

// ── Mint the reference and insert the header ──────────────────────────
// Text(Year(...), "0000") and not Text(Now(), "yyyymm"): see Section 6.
Set(varPeriod, Text(Year(Now()), "0000") & Text(Month(Now()), "00"));
If(
    IsBlank(LookUp(RefCounter, Title = varPeriod)),
    IfError(Patch(RefCounter, Defaults(RefCounter), { Title: varPeriod, LastSeq: 0 }), false)
);

// ── Five attempts, written out one after another (numMintAttempts) ────
// Each attempt does nothing once varNewRef is set. A failed header insert is
// taken to be a clash on the unique Requests.Title, so the next attempt
// re-reads the counter and tries the next number.
// Attempt 1 of 5
If(IsBlank(varNewRef),
    Refresh(RefCounter);
    With({ row: LookUp(RefCounter, Title = varPeriod) },
        IfError(
```

```
            Patch(RefCounter, row, { LastSeq: row.LastSeq + 1 }),
            Set(varErrorMsg, FirstError.Message),
            With({ candidate: "SR-" & varPeriod & "-" & Text(row.LastSeq + 1, "0000") },
                IfError(
                    Patch(Requests, Defaults(Requests), {
                        Title:                candidate,
                        OfficeCode:           varFormOffice,
                        Section:              { Value: varFormSection },
                        Purpose:              { Value: varFormPurpose },
                        RequesterEmail:       varUserEmail,
                        Status:               { Value: varNewStatus },
                        StatusRank:           LookUp(tblStatus, StatusText = varNewStatus).Rank,
                        CurrentStageCode:     If(varAsDraft, Blank(), { Value: varRoute.NextStage }),
                        CurrentlyWithEmail:   If(varAsDraft, "", varRoute.WithEmail),
                        CurrentlyWithOICEmail: If(varAsDraft, "", varRoute.OICEmail),
                        SubmittedOn:          If(varAsDraft, Blank(), Now()),
                        LineCount:            CountRows(colSave),
                        TotalQtyRequested:    Sum(colSave, QtyRequested),
                        SharingOfficeCode:    ""
                    }),
                    Set(varErrorMsg, FirstError.Message),
                    Set(varNewRef, candidate); Set(varErrorMsg, "")
                )
            )
        )
    )
);
// Attempt 2 of 5
If(IsBlank(varNewRef),
    Refresh(RefCounter);
    With({ row: LookUp(RefCounter, Title = varPeriod) },
        IfError(
            Patch(RefCounter, row, { LastSeq: row.LastSeq + 1 }),
            Set(varErrorMsg, FirstError.Message),
            With({ candidate: "SR-" & varPeriod & "-" & Text(row.LastSeq + 1, "0000") },
                IfError(
                    Patch(Requests, Defaults(Requests), {
                        Title:                candidate,
                        OfficeCode:           varFormOffice,
                        Section:              { Value: varFormSection },
                        Purpose:              { Value: varFormPurpose },
                        RequesterEmail:       varUserEmail,
                        Status:               { Value: varNewStatus },
                        StatusRank:           LookUp(tblStatus, StatusText = varNewStatus).Rank,
                        CurrentStageCode:     If(varAsDraft, Blank(), { Value: varRoute.NextStage }),
                        CurrentlyWithEmail:   If(varAsDraft, "", varRoute.WithEmail),
                        CurrentlyWithOICEmail: If(varAsDraft, "", varRoute.OICEmail),
                        SubmittedOn:          If(varAsDraft, Blank(), Now()),
                        LineCount:            CountRows(colSave),
                        TotalQtyRequested:    Sum(colSave, QtyRequested),
                        SharingOfficeCode:    ""
                    }),
                    Set(varErrorMsg, FirstError.Message),
                    Set(varNewRef, candidate); Set(varErrorMsg, "")
                )
            )
        )
    )
);
// Attempt 3 of 5
If(IsBlank(varNewRef),
    Refresh(RefCounter);
    With({ row: LookUp(RefCounter, Title = varPeriod) },
        IfError(
            Patch(RefCounter, row, { LastSeq: row.LastSeq + 1 }),
            Set(varErrorMsg, FirstError.Message),
            With({ candidate: "SR-" & varPeriod & "-" & Text(row.LastSeq + 1, "0000") },
                IfError(
                    Patch(Requests, Defaults(Requests), {
                        Title:                candidate,
                        OfficeCode:           varFormOffice,
```

```
                        Section:              { Value: varFormSection },
                        Purpose:              { Value: varFormPurpose },
                        RequesterEmail:       varUserEmail,
                        Status:               { Value: varNewStatus },
                        StatusRank:           LookUp(tblStatus, StatusText = varNewStatus).Rank,
                        CurrentStageCode:     If(varAsDraft, Blank(), { Value: varRoute.NextStage }),
                        CurrentlyWithEmail:   If(varAsDraft, "", varRoute.WithEmail),
                        CurrentlyWithOICEmail: If(varAsDraft, "", varRoute.OICEmail),
                        SubmittedOn:          If(varAsDraft, Blank(), Now()),
                        LineCount:            CountRows(colSave),
                        TotalQtyRequested:    Sum(colSave, QtyRequested),
                        SharingOfficeCode:    ""
                    }),
                    Set(varErrorMsg, FirstError.Message),
                    Set(varNewRef, candidate); Set(varErrorMsg, "")
                )
            )
        )
    )
);
// Attempt 4 of 5
If(IsBlank(varNewRef),
    Refresh(RefCounter);
    With({ row: LookUp(RefCounter, Title = varPeriod) },
        IfError(
            Patch(RefCounter, row, { LastSeq: row.LastSeq + 1 }),
            Set(varErrorMsg, FirstError.Message),
            With({ candidate: "SR-" & varPeriod & "-" & Text(row.LastSeq + 1, "0000") },
                IfError(
                    Patch(Requests, Defaults(Requests), {
                        Title:                candidate,
                        OfficeCode:           varFormOffice,
                        Section:              { Value: varFormSection },
                        Purpose:              { Value: varFormPurpose },
                        RequesterEmail:       varUserEmail,
                        Status:               { Value: varNewStatus },
                        StatusRank:           LookUp(tblStatus, StatusText = varNewStatus).Rank,
                        CurrentStageCode:     If(varAsDraft, Blank(), { Value: varRoute.NextStage }),
                        CurrentlyWithEmail:   If(varAsDraft, "", varRoute.WithEmail),
                        CurrentlyWithOICEmail: If(varAsDraft, "", varRoute.OICEmail),
                        SubmittedOn:          If(varAsDraft, Blank(), Now()),
                        LineCount:            CountRows(colSave),
                        TotalQtyRequested:    Sum(colSave, QtyRequested),
                        SharingOfficeCode:    ""
                    }),
                    Set(varErrorMsg, FirstError.Message),
                    Set(varNewRef, candidate); Set(varErrorMsg, "")
                )
            )
        )
    )
);
// Attempt 5 of 5
If(IsBlank(varNewRef),
    Refresh(RefCounter);
    With({ row: LookUp(RefCounter, Title = varPeriod) },
        IfError(
            Patch(RefCounter, row, { LastSeq: row.LastSeq + 1 }),
            Set(varErrorMsg, FirstError.Message),
            With({ candidate: "SR-" & varPeriod & "-" & Text(row.LastSeq + 1, "0000") },
                IfError(
                    Patch(Requests, Defaults(Requests), {
                        Title:                candidate,
                        OfficeCode:           varFormOffice,
                        Section:              { Value: varFormSection },
                        Purpose:              { Value: varFormPurpose },
                        RequesterEmail:       varUserEmail,
                        Status:               { Value: varNewStatus },
                        StatusRank:           LookUp(tblStatus, StatusText = varNewStatus).Rank,
                        CurrentStageCode:     If(varAsDraft, Blank(), { Value: varRoute.NextStage }),
                        CurrentlyWithEmail:   If(varAsDraft, "", varRoute.WithEmail),
```

```
                            CurrentlyWithOICEmail: If(varAsDraft, "", varRoute.OICEmail),
                            SubmittedOn:           If(varAsDraft, Blank(), Now()),
                            LineCount:             CountRows(colSave),
                            TotalQtyRequested:     Sum(colSave, QtyRequested),
                            SharingOfficeCode:     ""
                        }),
                        Set(varErrorMsg, FirstError.Message),
                        Set(varNewRef, candidate); Set(varErrorMsg, "")
                    )
                )
            )
        )
    );

    If(
        IsBlank(varNewRef),
        Set(varErrorMsg, "Could not allocate a reference after 5 attempts. " & varErrorMsg),

        // ── The lines, in one call ──────────────────────────────────────
        IfError(
            Collect(
                RequestItems,
                ForAll(
                    Sequence(CountRows(colSave)) As n,
                    With({ ln: Index(colSave, n.Value) },
                        {
                            Title:           varNewRef & "-" & Text(n.Value, "00"),
                            RequestRefNo:    varNewRef,
                            OfficeCode:      varFormOffice,
                            LineNo:          n.Value,
                            ItemCode:        ln.ItemCode,
                            ItemName:        Trim(ln.ItemName),
                            IsCatalogueItem: ln.IsCatalogueItem,
                            OfficeItemKey:   ln.OfficeItemKey,
                            UnitOfMeasure:   ln.UnitOfMeasure,
                            UnitCost:        ln.UnitCost,
                            LinePurpose:     ln.LinePurpose,
                            LineNote:        ln.LineNote,        // DEF-02
                            QtyRequested:    ln.QtyRequested
                        }
                    )
                )
            ),
            Set(varErrorMsg, "Request " & varNewRef & " was created, but its items were not saved. " &
FirstError.Message)
        );

        // ── Timeline ────────────────────────────────────────────────────
        Collect(RequestTimeline, {
            Title:        If(varAsDraft, "Draft created", "Request submitted"),
            RequestRefNo: varNewRef,
            EventType:    { Value: If(varAsDraft, "Draft created", "Submitted") },
            ToStatus:     varNewStatus,
            ActorEmail:   varUserEmail,
            OccurredOn:   Now(),
            Detail:       If(varAsDraft, "", "reference " & varNewRef & " issued")
        });
        If(!varAsDraft && !IsBlank(varRoute.WithEmail),
            Collect(RequestTimeline, {
                Title:        "Routed to " & varRoute.NextStage,
                RequestRefNo: varNewRef,
                EventType:    { Value: "Routed" },
                StageCode:    { Value: varRoute.NextStage },     // DEF-138
                OccurredOn:   Now(),
                Detail:       "assigned to " & varRoute.WithEmail
            })
        )
    )
);

Set(varBusy, false);
```

```
If(
    IsBlank(varErrorMsg),
    Set(varRef, varNewRef);
    Notify(If(varAsDraft, "Draft " & varNewRef & " saved.", "Request " & varNewRef & " submitted."),
NotificationType.Success);
    Navigate(scrRequestDetail, ScreenTransition.Fade),
    Notify(varErrorMsg, NotificationType.Error)
)
)
```

Check after building it: submit one request and confirm exactly **one** new row in `Requests`, with its lines in `RequestItems` and two rows in `RequestTimeline` (three for nothing; one for a draft).

Section 9 — Resubmit (a Draft or Returned request)

A Draft is finished, and a Returned request is corrected, through the same screen in edit mode (`varEditRef` = the reference). The reference does not change. Ported from `useResubmitRequest`.

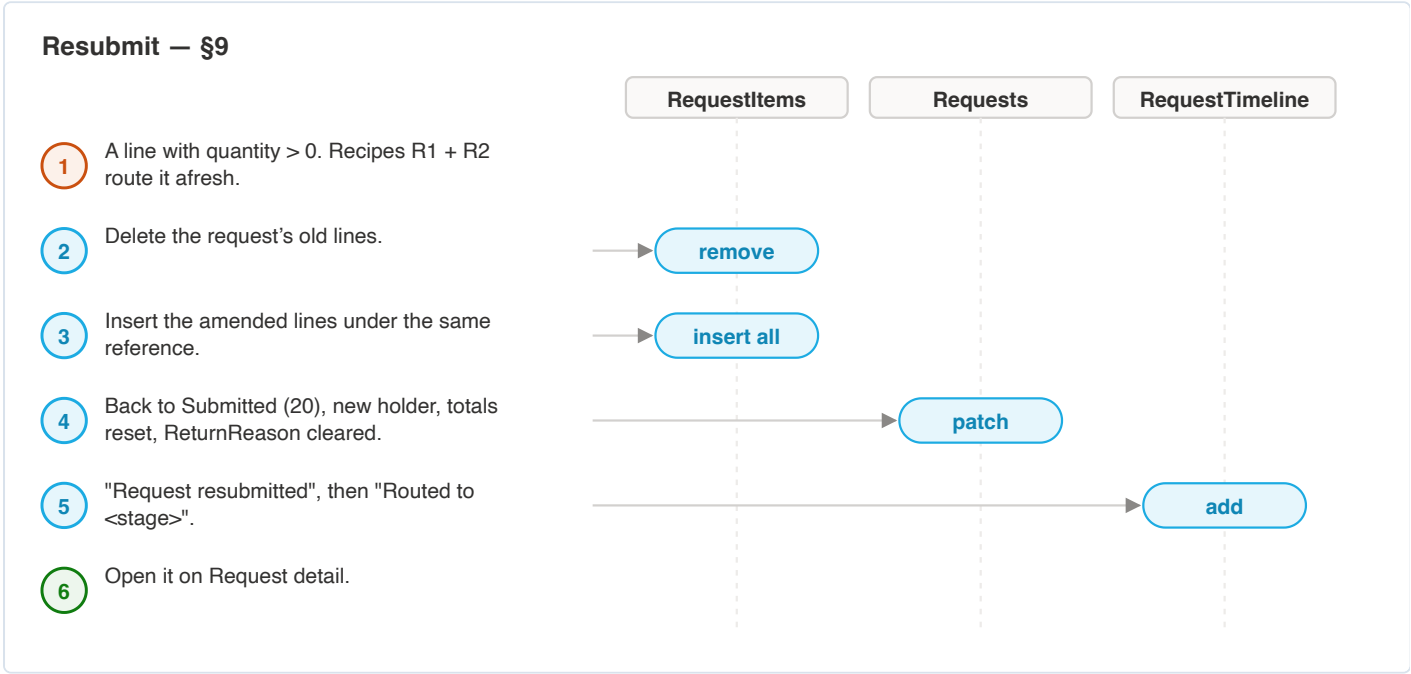


Figure 16. Resubmit: what is written, in order

`btnResubmitCore.OnSelect`

```
If(varBusy, false,
Set(varBusy, true);
Set(varErrorMsg, "");
Set(varFormOffice,  ddOffice.Selected.Title);
Set(varFormSection, ddSection.Selected.Value);
Set(varFormPurpose, ddPurpose.Selected.Value);
ClearCollect(colSave, Filter(colLines, !IsBlank(Trim(ItemName))));

If(
    IsEmpty(Filter(colSave, QtyRequested > 0)),
    Set(varErrorMsg, "Add at least one item with a quantity greater than zero before submitting."),

    // Recipes R1 and R2: route it afresh.
    // Recipe R1 — the approval chain for varFormOffice / varFormSection
    ClearCollect(
        colChain,
        ForAll(
```

```
            Filter(tblJourney, Skippable) As leg,
            With(
                {
                    rowSec: LookUp(colMatrix, MatrixKey = varFormOffice & "|" & varFormSection
                            && StageCode.Value = leg.StageCode),
                    rowAll: LookUp(colMatrix, MatrixKey = varFormOffice & "|All sections"
                            && StageCode.Value = leg.StageCode)
                },
                With(
                    { row: If(IsBlank(rowSec), rowAll, rowSec) },
                    {
                        Leg:        leg.Leg,
                        StageCode:  leg.StageCode,
                        LegLabel:   leg.LegLabel,
                        ActorLabel: leg.ActorLabel,
                        Live:       !IsBlank(row),
                        StageOrder: row.StageOrder,
                        Role:       row.ActorRoleCode.Value,
                        Approver:   Lower(Trim(row.ApproverEmail)),
                        OIC:        Lower(Trim(row.OICEmail))
                    }
                )
            )
        )
    );
    // Recipe R2 — where it goes next (leaving nothing: first route)
    Set(
        varRoute,
        With(
            { fromLeg: Coalesce(LookUp(tblJourney, StageCode = "").Leg, 1) },
            With(
                { next: First(Sort(Filter(colChain, Live, Leg > fromLeg), Leg)) },
                If(
                    IsBlank(next) || next.StageCode = "Receive",
                    { NextStage: "Receive", WithEmail: varUserEmail, OICEmail: "" },
                    With(
                        { primary: Coalesce(next.Approver, next.OIC) },
                        {
                            NextStage: next.StageCode,
                            WithEmail: Coalesce(primary, varUserEmail),
                            OICEmail:  If(!IsBlank(next.OIC) && next.OIC <> primary, next.OIC, "")
                        }
                    )
                )
            )
        )
    );
    IfError(
        // 1. Clear the old lines (the request may have been amended).
        Remove(RequestItems, Filter(RequestItems, RequestRefNo = varEditRef)),
        Set(varErrorMsg, "The old items could not be cleared. " & FirstError.Message),

        // 2. Insert the amended lines — same ForAll as Section 8, with varEditRef.
        Collect(RequestItems,
            ForAll(Sequence(CountRows(colSave)) As n,
                With({ ln: Index(colSave, n.Value) },
                    {
                        Title: varEditRef & "-" & Text(n.Value, "00"), RequestRefNo: varEditRef,
                        OfficeCode: varFormOffice, LineNo: n.Value, ItemCode: ln.ItemCode,
                        ItemName: Trim(ln.ItemName), IsCatalogueItem: ln.IsCatalogueItem,
                        OfficeItemKey: ln.OfficeItemKey, UnitOfMeasure: ln.UnitOfMeasure,
                        UnitCost: ln.UnitCost, LinePurpose: ln.LinePurpose, LineNote: ln.LineNote,
                        QtyRequested: ln.QtyRequested
                    }))),
        Set(varErrorMsg, "The amended items were not saved. " & FirstError.Message),

        // 3. The header, back to Submitted with fresh routing and zeroed rollups.
        Patch(Requests, LookUp(Requests, Title = varEditRef), {
            OfficeCode:            varFormOffice,
            Section:               { Value: varFormSection },
            Purpose:               { Value: varFormPurpose },
```

```
            Status:                { Value: "Submitted" },
            StatusRank:            20,
            CurrentStageCode:      { Value: varRoute.NextStage },
            CurrentlyWithEmail:    varRoute.WithEmail,
            CurrentlyWithOICEmail: varRoute.OICEmail,
            SubmittedOn:           Now(),
            LineCount:             CountRows(colSave),
            TotalQtyRequested:     Sum(colSave, QtyRequested),
            TotalQtyApproved:      0,
            TotalQtyDistributed:   0,
            TotalQtyReceived:      0,
            SharingOfficeCode:     "",
            ReturnReason:          ""
        }),
        Set(varErrorMsg, "The request was not resubmitted. " & FirstError.Message),

        // 4. Timeline.
        Collect(RequestTimeline, {
            Title: "Request resubmitted", RequestRefNo: varEditRef,
            EventType: { Value: "Submitted" }, ToStatus: "Submitted",
            ActorEmail: varUserEmail, OccurredOn: Now(), Detail: "amended and resubmitted"
        });
        If(!IsBlank(varRoute.WithEmail),
            Collect(RequestTimeline, {
                Title: "Routed to " & varRoute.NextStage, RequestRefNo: varEditRef,
                EventType: { Value: "Routed" }, StageCode: { Value: varRoute.NextStage },
                OccurredOn: Now(), Detail: "assigned to " & varRoute.WithEmail
            }))
    )
);

Set(varBusy, false);
If(
    IsBlank(varErrorMsg),
    Set(varRef, varEditRef); Set(varEditRef, "");
    Notify("Request " & varRef & " resubmitted.", NotificationType.Success);
    Navigate(scrRequestDetail, ScreenTransition.Fade),
    Notify(varErrorMsg, NotificationType.Error)
)
)
```

`RejectReason` is left alone, as in the SPFx build.

Section 10 — Proceed (Review, Approval and Confirm)

One screen, `scrReview`, serves all three stages. Only Review changes quantities; Approval and Confirm move the request on unchanged. The status written is always `Reviewed` (DEF-12, by design). Ported from `useProceedReview`.

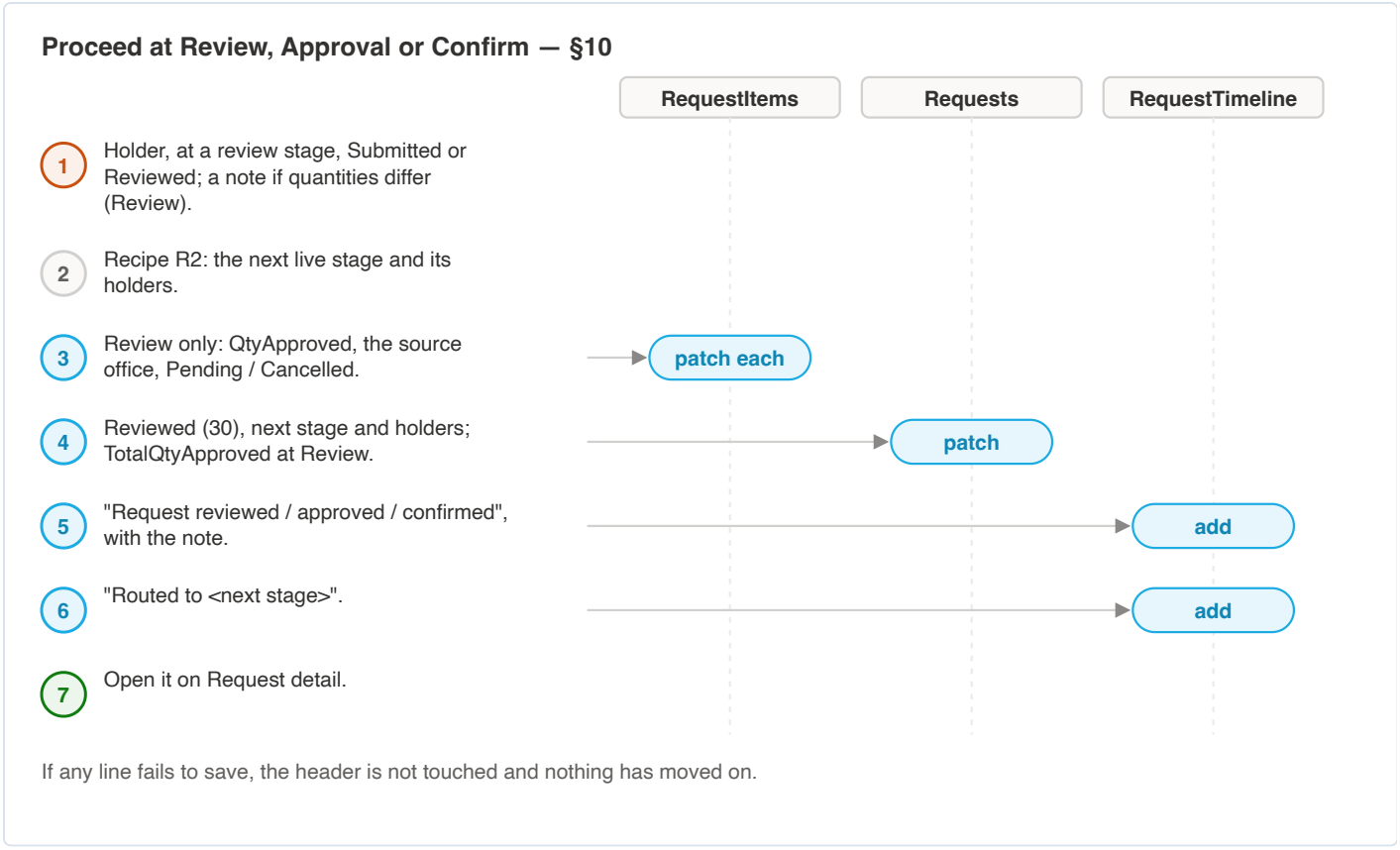


Figure 17. Proceed: what is written, in order

Inputs. `varReq`, `colChain` and `varStage` from the loader (Recipe R6); the gallery `galReview` (one row per line, columns described in the scrReview chapter) with its text box `txtApprove`; the note box `txtNote`.

`btnProceed.DisplayMode` (DEF-04: open, held by me, at a review stage, and not a Draft/Returned request; a note is required when any quantity differs):

```
If(
    varCanAct && varStage in ["Review", "Approval", "Confirm"] && varIsReviewable && !varBusy
    // the note is required when any quantity to distribute differs from the request
    && Not(varStage = "Review" && IsBlank(Trim(txtNote.Text))
           && !IsEmpty(Filter(galReview.AllItems,
                  Min(Max(Coalesce(Value(txtApprove.Text), 0), 0), MaxQty) <> QtyRequested))),
    DisplayMode.Edit,
    DisplayMode.Disabled
)
```

`btnProceed.OnSelect`

```
If(varBusy, false,
Set(varBusy, true);
Set(varErrorMsg, "");
Clear(colErrors);
Set(varFromStage, varStage);
// Recipe R2 (colChain was built by the loader, Recipe R6).
// Recipe R2 — where it goes next (leaving varFromStage)
Set(
    varRoute,
    With(
        { fromLeg: Coalesce(LookUp(tblJourney, StageCode = varFromStage).Leg, 1) },
        With(
            { next: First(Sort(Filter(colChain, Live, Leg > fromLeg), Leg)) },
            If(
                IsBlank(next) || next.StageCode = "Receive",
```

```
                { NextStage: "Receive", WithEmail: Lower(varReq.RequesterEmail), OICEmail: "" },
                With(
                    { primary: Coalesce(next.Approver, next.OIC) },
                    {
                        NextStage: next.StageCode,
                        WithEmail: Coalesce(primary, Lower(varReq.RequesterEmail)),
                        OICEmail:  If(!IsBlank(next.OIC) && next.OIC <> primary, next.OIC, "")
                    }
                )
            )
        )
    )
);

// 1. The lines — Review stage only.
If(varFromStage = "Review",
    ClearCollect(colApprove,
        ForAll(galReview.AllItems As g,
            With({ q: Min(Max(Coalesce(Value(g.txtApprove.Text), 0), 0), g.MaxQty) },
                {
                    LineID: g.LineID,
                    Qty:    q,
                    // src/domain/sharing.ts resolveStockSource: own office if it covers
                    // the quantity, else the lending office if it does, else own office.
                    Source: If(
                        !g.IsCatalogueItem,                                varReq.OfficeCode,
                        q <= g.LocalStock,                                 varReq.OfficeCode,
                        !IsBlank(g.LenderOffice) && q <= g.LenderStock,    g.LenderOffice,
                        varReq.OfficeCode)
                }))));
    ForAll(colApprove As a,
        IfError(
            Patch(RequestItems, LookUp(RequestItems, ID = a.LineID), {
                QtyApproved:      a.Qty,
                SourceOfficeCode: a.Source,
                LineStatus:       { Value: If(a.Qty > 0, "Pending", "Cancelled") }
            }),
            Collect(colErrors, { Msg: FirstError.Message }))));

// 2. The header, only if every line saved.
If(
    !IsEmpty(colErrors),
    Set(varErrorMsg, "Nothing was moved on. " & First(colErrors).Msg),

    IfError(
        Patch(Requests, LookUp(Requests, Title = varRef), {
            Status:                { Value: "Reviewed" },
            StatusRank:            30,
            CurrentStageCode:      { Value: varRoute.NextStage },
            CurrentlyWithEmail:    varRoute.WithEmail,
            CurrentlyWithOICEmail: varRoute.OICEmail,
            TotalQtyApproved:      If(varFromStage = "Review", Sum(colApprove, Qty), varReq.TotalQtyApproved)
        }),
        Set(varErrorMsg, "Nothing was moved on. " & FirstError.Message),

        // 3. Timeline. DEF-104: the title names the stage.
        Collect(RequestTimeline, {
            Title:        Switch(varFromStage, "Review", "Request reviewed", "Approval", "Request approved",
"Request confirmed"),
            RequestRefNo: varRef,
            EventType:    { Value: If(varFromStage = "Review", "Reviewed", "Confirmed") },
            FromStatus:   varReq.Status.Value,
            ToStatus:     "Reviewed",
            StageCode:    { Value: varFromStage },
            ActorEmail:   varUserEmail,
            OccurredOn:   Now(),
            Detail:       Trim(txtNote.Text)
        });
        If(!IsBlank(varRoute.WithEmail),
            Collect(RequestTimeline, {
```

```
                Title: "Routed to " & varRoute.NextStage, RequestRefNo: varRef,
                EventType: { Value: "Routed" }, StageCode: { Value: varRoute.NextStage },   // DEF-138
                OccurredOn: Now(), Detail: "assigned to " & varRoute.WithEmail
            }))
    )
);

Set(varBusy, false);
If(
    IsBlank(varErrorMsg),
    Notify("Request " & varRef & " moved on to " & varRoute.NextStage & ".", NotificationType.Success);
    Navigate(scrRequestDetail, ScreenTransition.Fade),
    Notify(varErrorMsg, NotificationType.Error)
)
)
```

Section 11 — Return and Reject

Both open the same reason dialog on `scrReview`; a reason is required (NFR-AUD-003). Ported from `useReturnRequest` and `useRejectRequest`.

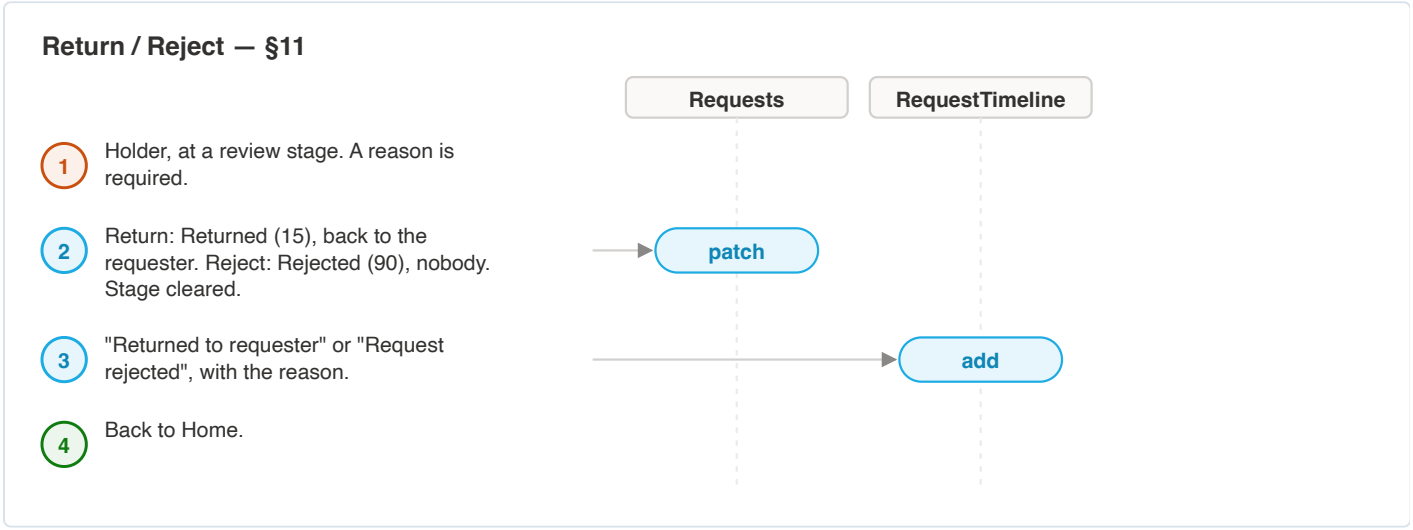


Figure 18. Return and Reject: what is written

```
// btnReturn.OnSelect           btnReject.OnSelect
Set(varDialog, "return");       Set(varDialog, "reject");
Reset(txtReason);               Reset(txtReason);
Set(varShowDialog, true)        Set(varShowDialog, true)

// btnReturn.DisplayMode and btnReject.DisplayMode (DEF-04, DEF-124)
If(varCanAct && varStage in ["Review", "Approval", "Confirm"] && varIsReviewable && !varBusy,
   DisplayMode.Edit, DisplayMode.Disabled)

// btnDialogConfirm.DisplayMode
If(IsBlank(Trim(txtReason.Text)) || varBusy, DisplayMode.Disabled, DisplayMode.Edit)
```

`btnDialogConfirm.OnSelect`

```
If(varBusy || IsBlank(Trim(txtReason.Text)), false,
Set(varBusy, true);
Set(varErrorMsg, "");
Set(varReason, Trim(txtReason.Text));
If(
    varDialog = "return",
    IfError(
```

```
        Patch(Requests, LookUp(Requests, Title = varRef), {
            Status:                { Value: "Returned" },
            StatusRank:            15,
            CurrentStageCode:      Blank(),           // really cleared (see Section 7)
            CurrentlyWithEmail:    Lower(varReq.RequesterEmail),
            CurrentlyWithOICEmail: "",
            ReturnReason:          varReason
        }),
        Set(varErrorMsg, "The request was not returned. " & FirstError.Message),
        Collect(RequestTimeline, {
            Title: "Returned to requester", RequestRefNo: varRef,
            EventType: { Value: "Returned" }, FromStatus: varReq.Status.Value, ToStatus: "Returned",
            StageCode: { Value: varStage },
            ActorEmail: varUserEmail, OccurredOn: Now(), Detail: varReason
        })
    ),
    IfError(
        Patch(Requests, LookUp(Requests, Title = varRef), {
            Status:                { Value: "Rejected" },
            StatusRank:            90,
            CurrentStageCode:      Blank(),
            CurrentlyWithEmail:    "",
            CurrentlyWithOICEmail: "",
            RejectReason:          varReason
        }),
        Set(varErrorMsg, "The request was not rejected. " & FirstError.Message),
        Collect(RequestTimeline, {
            Title: "Request rejected", RequestRefNo: varRef,
            EventType: { Value: "Rejected" }, FromStatus: varReq.Status.Value, ToStatus: "Rejected",
            StageCode: { Value: varStage },
            ActorEmail: varUserEmail, OccurredOn: Now(), Detail: varReason
        })
    )
);
Set(varBusy, false);
If(
    IsBlank(varErrorMsg),
    Set(varShowDialog, false);
    Notify(If(varDialog = "return", "Returned to the requester.", "Request rejected."),
NotificationType.Success);
    Navigate(scrHome, ScreenTransition.Fade),
    Notify(varErrorMsg, NotificationType.Error)      // the dialog stays open
)
)
```

The Return and Reject timeline rows carry the stage they left. The SPFx build left `StageCode` empty on them, so Reports never counted a return as the end of a stage; writing it lets the stage chart count them, as `src/domain/timeline.ts` intends.

Section 12 — Record distribution

Replaces Section 5.3. Same mitigations (live re-read, ledger first, rollup only if something moved), plus the UAT round 2 fixes: a partial delivery stays with the storekeeper at Deliver (DEF-05); a full delivery goes to the requester with a Routed row (DEF-122); the timeline gives this step's quantity and the running total (DEF-119); each failed line is reported (DEF-06). Ported from `src/hooks/useDistributeAction.ts`.

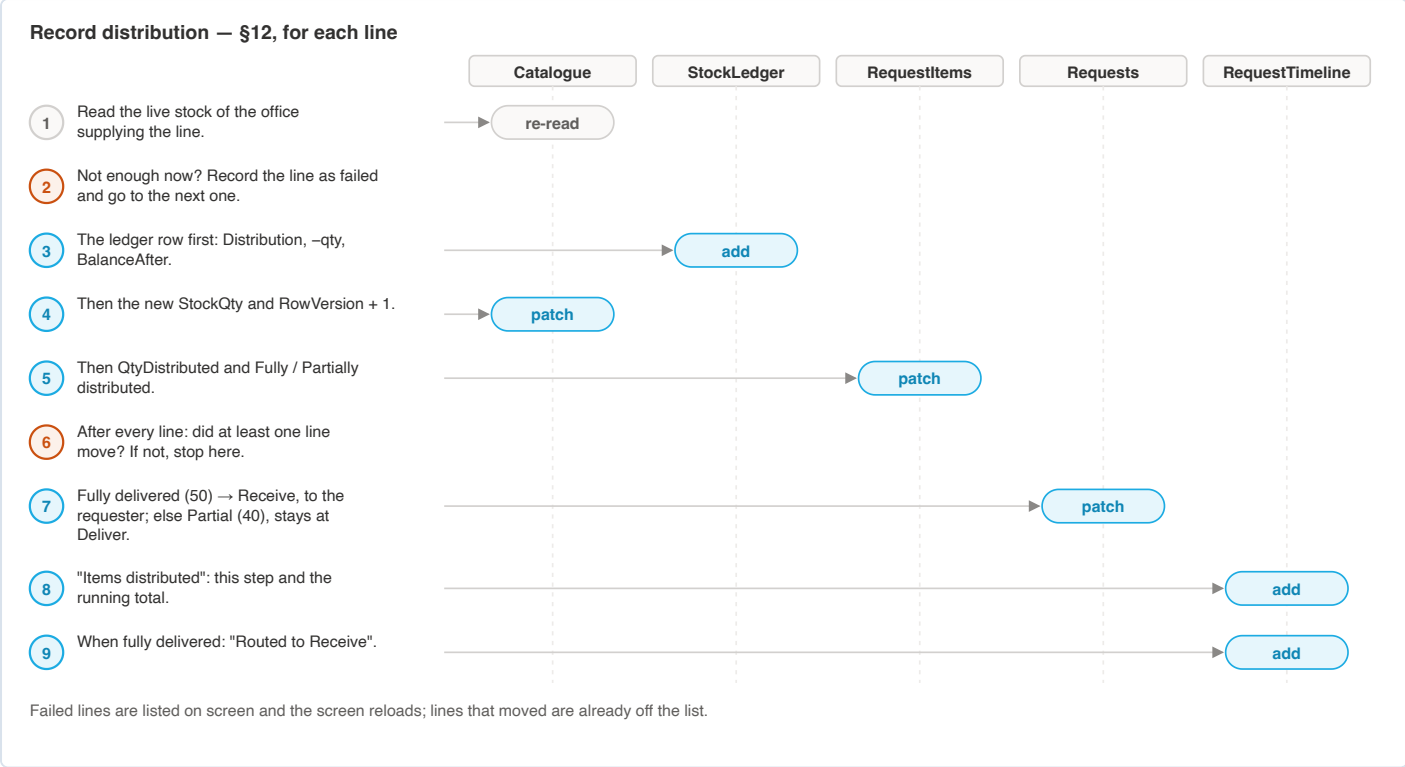


Figure 19. Record distribution: what is written, in order

Inputs. `varReq` and `colChain` from the loader; the gallery `galDist`, one row per catalogue line still owed (`QtyApproved > QtyDistributed`), with the columns `LineID, ItemName, ItemCode, SourceOffice, OfficeItemKey, QtyApproved, QtyDistributed, Remaining, Available` and the text box `txtDist`.

`btnRecordDistribution.OnSelect`

```
If(varBusy, false,
Set(varBusy, true);
Set(varErrorMsg, "");
Clear(colDistResult);

// The quantities typed, capped to what is owed and what is on the shelf.
ClearCollect(colDist,
    Filter(
        ForAll(galDist.AllItems As g,
            {
                LineID: g.LineID, ItemName: g.ItemName, ItemCode: g.ItemCode,
                SourceOffice: g.SourceOffice, OfficeItemKey: g.OfficeItemKey,
                QtyApproved: g.QtyApproved, QtyBefore: g.QtyDistributed,
                Qty: Min(Max(Coalesce(Value(g.txtDist.Text), 0), 0), g.Remaining, g.Available)
            }),
        Qty > 0));

// ── Line by line. NOT inside Concurrent(). ──────────────────────────
Refresh(Catalogue);
ForAll(colDist As ln,
    With({ live: LookUp(Catalogue, OfficeItemKey = ln.OfficeItemKey) },   // live re-read
        If(
            IsBlank(live),
            Collect(colDistResult, { LineID: ln.LineID, Ok: false,
                Msg: ln.ItemName & ": no catalogue row for " & ln.OfficeItemKey & "." }),

            ln.Qty > live.StockQty,
            Collect(colDistResult, { LineID: ln.LineID, Ok: false,
                Msg: ln.ItemName & ": only " & Text(live.StockQty) & " in stock now (someone else moved it); "
                     & Text(ln.Qty) & " requested." }),
```

```
            IfError(
                // 1. Ledger first — audit before mutation.
                Collect(StockLedger, {
                    Title:         ln.OfficeItemKey & " " & Text(Today(), "yyyy-mm-dd"),
                    OfficeItemKey: ln.OfficeItemKey,
                    OfficeCode:    ln.SourceOffice,
                    ItemCode:      ln.ItemCode,
                    MovementType:  { Value: "Distribution" },
                    QtyDelta:      -ln.Qty,
                    BalanceAfter:  live.StockQty - ln.Qty,
                    RequestRefNo:  varRef,
                    MovedOn:       Now(),
                    MovedByEmail:  varUserEmail,
                    Notes:         "Distributed against " & varRef & "."
                }),
                Collect(colDistResult, { LineID: ln.LineID, Ok: false, Msg: ln.ItemName & ": " &
FirstError.Message }),

                // 2. Then the stock figure.
                Patch(Catalogue, live, { StockQty: live.StockQty - ln.Qty, RowVersion: live.RowVersion + 1 }),
                Collect(colDistResult, { LineID: ln.LineID, Ok: false, Msg: ln.ItemName & ": " &
FirstError.Message }),

                // 3. Then the line.
                Patch(RequestItems, LookUp(RequestItems, ID = ln.LineID), {
                    QtyDistributed:   ln.QtyBefore + ln.Qty,
                    SourceOfficeCode: ln.SourceOffice,
                    LineStatus:       { Value: If(ln.QtyBefore + ln.Qty >= ln.QtyApproved,
                                          "Fully distributed", "Partially distributed") }
                }),
                Collect(colDistResult, { LineID: ln.LineID, Ok: false, Msg: ln.ItemName & ": " &
FirstError.Message }),

                Collect(colDistResult, { LineID: ln.LineID, Ok: true, Msg: "" })
            )
        )
    )
);

// ── The header, only if at least one line moved ──────────────────────
If(
    !IsEmpty(Filter(colDistResult, Ok)),

    ClearCollect(colLinesNow, Filter(RequestItems, RequestRefNo = varRef));
    Set(varTotDist, Sum(colLinesNow, Coalesce(QtyDistributed, 0)));
    Set(varTotAppr, Sum(colLinesNow, Coalesce(QtyApproved, 0)));
    Set(varDistNow, Sum(Filter(colDist, LineID in Filter(colDistResult, Ok).LineID), Qty));
    // Fully delivered = every CATALOGUE line has everything approved out (free text excluded).
    Set(varFullyOut, IsEmpty(Filter(colLinesNow, IsCatalogueItem,
                                Coalesce(QtyDistributed, 0) < Coalesce(QtyApproved, 0))));
    Set(varNewStatus, If(varFullyOut, "Fully delivered", "Partial delivery"));
    // DEF-05: who keeps a part-delivered request — the Deliver stage's holders.
    Set(varDeliverRow, LookUp(colChain, StageCode = "Deliver"));
    Set(varDeliverWith, Coalesce(varDeliverRow.Approver, varDeliverRow.OIC, Lower(varReq.RequesterEmail)));
    Set(varDeliverOIC,  If(!IsBlank(varDeliverRow.OIC) && varDeliverRow.OIC <> varDeliverWith,
varDeliverRow.OIC, ""));

    IfError(
        Patch(Requests, LookUp(Requests, Title = varRef), {
            Status:                { Value: varNewStatus },
            StatusRank:            If(varFullyOut, 50, 40),
            TotalQtyDistributed:   varTotDist,
            CurrentStageCode:      { Value: If(varFullyOut, "Receive", "Deliver") },
            CurrentlyWithEmail:    If(varFullyOut, Lower(varReq.RequesterEmail), varDeliverWith),
            CurrentlyWithOICEmail: If(varFullyOut, "", varDeliverOIC)
        }),
        Set(varErrorMsg, "Stock moved, but the request header was not updated. " & FirstError.Message),

        Collect(RequestTimeline, {
            Title: "Items distributed", RequestRefNo: varRef,
            EventType: { Value: "Distributed" }, FromStatus: varReq.Status.Value, ToStatus: varNewStatus,
```

```
            StageCode: { Value: "Deliver" }, ActorEmail: varUserEmail, OccurredOn: Now(),
            // DEF-119
            Detail: Text(varDistNow) & " distributed now; " & Text(varTotDist) & " of " & Text(varTotAppr) & "
approved in total."
        });
        If(varFullyOut,                                                    // DEF-122
            Collect(RequestTimeline, {
                Title: "Routed to Receive", RequestRefNo: varRef,
                EventType: { Value: "Routed" }, StageCode: { Value: "Receive" },
                OccurredOn: Now(), Detail: "assigned to " & Lower(varReq.RequesterEmail)
            }))
    )
);

Set(varBusy, false);
// DEF-06: say exactly what did and did not save (src/domain/errors.ts lineFailureSummary).
With({ okN: CountRows(Filter(colDistResult, Ok)), badN: CountRows(Filter(colDistResult, !Ok)) },
    If(
        badN = 0 && IsBlank(varErrorMsg),
        Notify("Distribution recorded.", NotificationType.Success);
        Navigate(scrRequestDetail, ScreenTransition.Fade),

        Set(varErrorMsg,
            Coalesce(varErrorMsg,
                If(okN = 0, "Nothing was recorded. ",
                   "Recorded " & okN & If(okN = 1, " line", " lines") & ", but " & badN & " could not be saved.
")
                & Concat(Filter(colDistResult, !Ok), Msg, " ")));
        Notify(varErrorMsg, NotificationType.Error);
        Select(btnLoad_Distribute)        // re-read stock and lines (Recipe R6)
    )
)
)
```

Free-text lines never reach this formula: `galDist` lists catalogue lines only.

Section 13 — Confirm receipt

The requester records what arrived. Ported from `src/hooks/useConfirmReceiptAction.ts`. A request whose delivery is complete closes on receipt, even when something arrived short or damaged; the shortfall stays on the line (`ReceiptCondition`, `LineNote`). That is a judgment call carried over from the SPFx build.

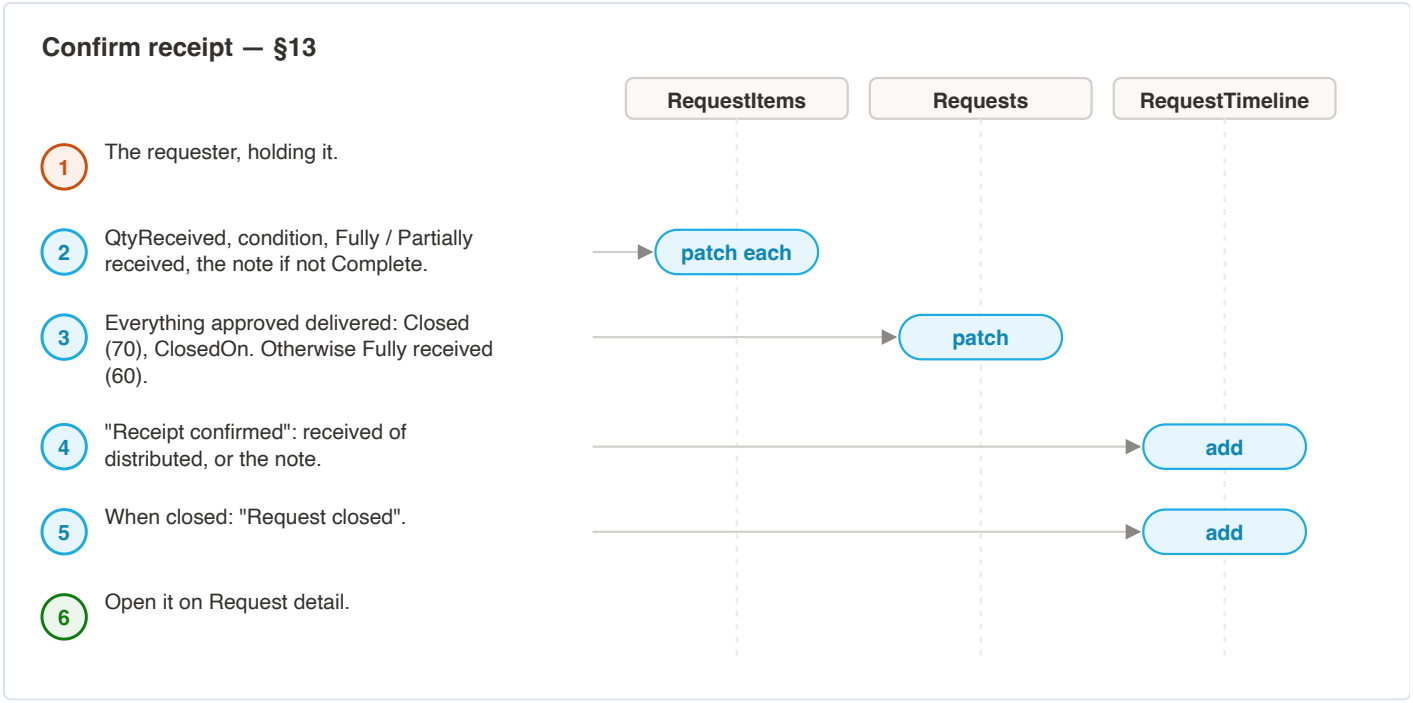


Figure 20. Confirm receipt: what is written, in order

Inputs. The gallery `galReceipt` (lines with `QtyDistributed > 0`) with `txtReceived` and the dropdown `ddCondition`; the note box `txtReceiptNote`.

`btnConfirmReceipt.OnSelect`

```
If(varBusy, false,
Set(varBusy, true);
Set(varErrorMsg, "");
Clear(colErrors);
Set(varReceiptNote, Trim(txtReceiptNote.Text));
ClearCollect(colRec,
    ForAll(galReceipt.AllItems As g,
        {
            LineID:    g.ID,
            Approved:  Coalesce(g.QtyApproved, 0),
            Delivered: Coalesce(g.QtyDistributed, 0),
            Received:  Min(Max(Coalesce(Value(g.txtReceived.Text), 0), 0), Coalesce(g.QtyDistributed, 0)),
            Condition: g.ddCondition.Selected.Value,
            OldNote:   g.LineNote
        }));

ForAll(colRec As r,
    IfError(
        Patch(RequestItems, LookUp(RequestItems, ID = r.LineID), {
            QtyReceived:      r.Received,
            ReceiptCondition: { Value: r.Condition },
            LineStatus:       { Value: If(r.Received >= r.Delivered, "Fully received", "Partially received") },
            LineNote:         If(r.Condition <> "Complete" && !IsBlank(varReceiptNote), varReceiptNote,
r.OldNote)
        }),
        Collect(colErrors, { Msg: FirstError.Message })));

If(
    !IsEmpty(colErrors),
    Set(varErrorMsg, "Receipt was not recorded. " & First(colErrors).Msg),

    Set(varTotRec,   Sum(colRec, Received));
    Set(varTotDel,   Sum(colRec, Delivered));
    Set(varCloseNow, IsEmpty(Filter(colRec, Delivered < Approved)));
    IfError(
        If(varCloseNow,
```

```
            Patch(Requests, LookUp(Requests, Title = varRef), {
                Status: { Value: "Closed" }, StatusRank: 70, ClosedOn: Now(),
                TotalQtyReceived: varTotRec, CurrentStageCode: Blank(),
                CurrentlyWithEmail: "", CurrentlyWithOICEmail: ""
            }),
            Patch(Requests, LookUp(Requests, Title = varRef), {
                Status: { Value: "Fully received" }, StatusRank: 60,
                TotalQtyReceived: varTotRec, CurrentlyWithEmail: "", CurrentlyWithOICEmail: ""
            })),
        Set(varErrorMsg, "The lines were saved, but the request was not updated. " & FirstError.Message),

        Collect(RequestTimeline, {
            Title: "Receipt confirmed", RequestRefNo: varRef,
            EventType: { Value: "Received" }, FromStatus: varReq.Status.Value,
            ToStatus: If(varCloseNow, "Closed", "Fully received"),
            StageCode: { Value: "Receive" }, ActorEmail: varUserEmail, OccurredOn: Now(),
            // DEF-123
            Detail: Coalesce(varReceiptNote, Text(varTotRec) & " of " & Text(varTotDel) & " distributed units
received.")
        });
        If(varCloseNow,
            Collect(RequestTimeline, {
                Title: "Request closed", RequestRefNo: varRef,
                EventType: { Value: "Closed" }, ToStatus: "Closed", OccurredOn: Now()
            }))
    )
);

Set(varBusy, false);
If(
    IsBlank(varErrorMsg),
    Notify(If(varCloseNow, "Receipt confirmed. The request is closed.", "Receipt confirmed."),
NotificationType.Success);
    Navigate(scrRequestDetail, ScreenTransition.Fade),
    Notify(varErrorMsg, NotificationType.Error)
)
)
```

Section 14 — Comments and documents

Both live on `scrRequestDetail`, and anyone who can open the request may use them. Each also writes a timeline row (DEF-101). Ported from `src/hooks/useRequestComments.ts` and `useRequestDocuments.ts`.

`btnPostComment.OnSelect` (`DisplayMode` disabled while `IsBlank(Trim(txtComment.Text)) || varBusy`)

```
If(varBusy || IsBlank(Trim(txtComment.Text)), false,
Set(varBusy, true);
IfError(
    Collect(RequestComments, {
        Title: varRef, RequestRefNo: varRef, CommentText: Trim(txtComment.Text),
        AuthorEmail: varUserEmail, PostedOn: Now()
    }),
    Notify("The comment was not posted. " & FirstError.Message, NotificationType.Error),
    Collect(RequestTimeline, {
        Title: "Comment added", RequestRefNo: varRef, EventType: { Value: "Comment added" },
        ActorEmail: varUserEmail, OccurredOn: Now(), Detail: Trim(txtComment.Text)
    });
    Reset(txtComment);
    Select(btnLoad_Detail)
);
Set(varBusy, false)
)
```

Attaching a document. A canvas app cannot write a file into a document library by itself, so it hands the file to the flow in `flows/UploadRequestDocument.md`. The file comes from an **Attachments** control (`attFile`, how to get one is in the scrRequestDetail chapter).

The stored file name is `<Ref>_<name>`, with the characters SharePoint refuses replaced by `-`, a leading dot removed and the prefix never added twice (`src/domain/documents.ts`, DEF-07):

```
// btnAttach.OnSelect  (DisplayMode disabled while IsEmpty(attFile.Attachments) || varBusy)
If(varBusy || IsEmpty(attFile.Attachments), false,
Set(varBusy, true);
With(
    { original: First(attFile.Attachments).Name },
    With(
        { clean: Concat(ForAll(Sequence(Len(original)) As i, Mid(original, i.Value, 1)) As ch,
                        If(ch.Value in "~""#%&*:<>?/\{|}", "-", ch.Value)) },
        With(
            { trimmed: If(Left(clean, 1) = ".", Mid(clean, 2), clean) },
            Set(varStoredName, If(StartsWith(trimmed, varRef & "_"), trimmed, varRef & "_" & trimmed))
        )
    )
);
If(
    !IsEmpty(Filter(colDocs, Lower('File name with extension') = Lower(varStoredName))),
    Notify("A file called """ & First(attFile.Attachments).Name & """ is already attached to " & varRef
           & ". Rename it and try again.", NotificationType.Error),

    Set(varUpload, UploadRequestDocument.Run(
        varRef, ddDocType.Selected.Value, varStoredName, varUserEmail,
        { contentBytes: First(attFile.Attachments).Value, name: varStoredName }));
    If(
        varUpload.ok = "true",
        Collect(RequestTimeline, {
            Title: "Document attached", RequestRefNo: varRef, EventType: { Value: "Comment added" },
            ActorEmail: varUserEmail, OccurredOn: Now(),
            Detail: ddDocType.Selected.Value & ": " & First(attFile.Attachments).Name
        });
        Reset(attFile);
        Notify("Document attached.", NotificationType.Success);
        Select(btnLoad_Detail),
        Notify(varUpload.message, NotificationType.Error)
    )
);
Set(varBusy, false)
)
```

The `"…"` inside the character list is Power Fx's way of writing one `"`. `'File name with extension'` is how the connector shows a library's file name column; if your Data pane shows it as `Name`, use that.

Section 15 — Administration writes

All on `scrAdmin`. Each dialog resets its inputs every time it opens (DEF-105): the button that opens it calls `Reset()` on each input before showing it. After each write the matching collection is reloaded, so every screen sees the change. Ported from `src/hooks/useAdminActions.ts`.

15.1 Approval matrix — add and edit a step

```
// btnSaveNewStep.OnSelect (dialog: ddStepStage, txtStepSection, ddStepRole,
//                          txtStepApprover, txtStepOIC, txtStepOrder)
If(varBusy, false,
Set(varBusy, true);
With({ section: Coalesce(Trim(txtStepSection.Text), "All sections") },
```

```
    IfError(
        Patch(ApprovalMatrix, Defaults(ApprovalMatrix), {
            Title:         varAdminOffice & " · " & ddStepStage.Selected.Value,
            OfficeCode:    varAdminOffice,
            Section:       section,
            MatrixKey:     varAdminOffice & "|" & section,
            StageCode:     { Value: ddStepStage.Selected.Value },
            StageOrder:    Value(txtStepOrder.Text),
            ActorRoleCode: { Value: ddStepRole.Selected.Value },
            ApproverEmail: Lower(Trim(txtStepApprover.Text)),
            OICEmail:      Lower(Trim(txtStepOIC.Text)),
            IsActive:      true
        }),
        Notify("The step was not added. " & FirstError.Message, NotificationType.Error),
        ClearCollect(colMatrix, Filter(ApprovalMatrix, IsActive = true));
        Set(varShowAddStep, false);
        Notify("Step added.", NotificationType.Success)
    )
);
Set(varBusy, false)
)

// btnSaveStep.OnSelect (edit dialog; varEditStep = the row's record from colMatrix).
// OfficeCode and StageCode cannot change after creation, as in the SPFx build.
If(varBusy, false,
Set(varBusy, true);
With({ section: Coalesce(Trim(txtEditSection.Text), "All sections") },
    IfError(
        Patch(ApprovalMatrix, LookUp(ApprovalMatrix, ID = varEditStep.ID), {
            Section:       section,
            MatrixKey:     varEditStep.OfficeCode & "|" & section,
            ActorRoleCode: { Value: ddEditRole.Selected.Value },
            ApproverEmail: Lower(Trim(txtEditApprover.Text)),
            OICEmail:      Lower(Trim(txtEditOIC.Text)),
            StageOrder:    Value(txtEditOrder.Text)
        }),
        Notify("The step was not saved. " & FirstError.Message, NotificationType.Error),
        ClearCollect(colMatrix, Filter(ApprovalMatrix, IsActive = true));
        Set(varShowEditStep, false);
        Notify("Step saved.", NotificationType.Success)
    )
);
Set(varBusy, false)
)
```

15.2 Validate chain

Ported from `src/domain/validation.ts`. Admin always validates the office-wide chain (`"All sections"`). A skipped stage is not an error; a non-Receive stage whose row names nobody is.

```
// btnValidateChain.OnSelect
// Recipe R1 — the approval chain for varAdminOffice / "All sections"
ClearCollect(
    colChain,
    ForAll(
        Filter(tblJourney, Skippable) As leg,
        With(
            {
                rowSec: LookUp(colMatrix, MatrixKey = varAdminOffice & "|" & "All sections"
                        && StageCode.Value = leg.StageCode),
                rowAll: LookUp(colMatrix, MatrixKey = varAdminOffice & "|All sections"
                        && StageCode.Value = leg.StageCode)
            },
            With(
                { row: If(IsBlank(rowSec), rowAll, rowSec) },
                {
                    Leg:        leg.Leg,
```

```
                    StageCode:  leg.StageCode,
                    LegLabel:   leg.LegLabel,
                    ActorLabel: leg.ActorLabel,
                    Live:       !IsBlank(row),
                    StageOrder: row.StageOrder,
                    Role:       row.ActorRoleCode.Value,
                    Approver:   Lower(Trim(row.ApproverEmail)),
                    OIC:        Lower(Trim(row.OICEmail))
                }
            )
        )
    )
);
With(
    {
        bad:     Filter(colChain, Live, StageCode <> "Receive", IsBlank(Approver) && IsBlank(OIC)),
        skipped: Filter(colChain, !Live)
    },
    Set(varChainOk, IsEmpty(bad));
    Set(varChainMsg,
        If(IsEmpty(bad),
            "Chain is contiguous and every step resolves to an active person."
            & If(IsEmpty(skipped), "",
                 " This office skips the " & Concat(skipped, LegLabel, ", ")
                 & If(CountRows(skipped) = 1, " stage.", " stages.")),
            "Chain has " & CountRows(bad) & If(CountRows(bad) = 1, " step", " steps")
            & " with no resolvable person: "
            & Concat(bad, "step " & StageOrder & " (" & StageCode & ")", ", ") & "."))
)
```

15.3 Catalogue — add, edit, adjust stock

```
// btnSaveNewItem.OnSelect  (name and code required; stock starts at 0)
If(varBusy, false,
Set(varBusy, true);
With({ code: Upper(Trim(txtItemCode.Text)) },
    IfError(
        Patch(Catalogue, Defaults(Catalogue), {
            Title: Trim(txtItemName.Text), ItemCode: code, OfficeCode: varAdminOffice,
            OfficeItemKey: varAdminOffice & "|" & code,
            Category: Trim(txtItemCategory.Text), UnitOfMeasure: Trim(txtItemUnit.Text),
            StockQty: 0, ReorderLevel: If(IsBlank(txtItemReorder.Text), Blank(), Value(txtItemReorder.Text)),
            UnitCost: If(IsBlank(txtItemCost.Text), Blank(), Value(txtItemCost.Text)),
            RowVersion: 1, IsActive: true
        }),
        // OfficeItemKey is unique in SharePoint, so a repeated code fails here.
        Notify("The item was not added. " & FirstError.Message, NotificationType.Error),
        Select(btnLoad_AdminCatalogue);
        Set(varShowAddItem, false);
        Notify("Item added.", NotificationType.Success)
    )
);
Set(varBusy, false)
)

// btnSaveItem.OnSelect  (varEditItem = the row; stock is never edited here — use Adjust stock)
If(varBusy, false,
Set(varBusy, true);
IfError(
    Patch(Catalogue, LookUp(Catalogue, ID = varEditItem.ID), {
        Title: Trim(txtEditItemName.Text), Category: Trim(txtEditItemCategory.Text),
        UnitOfMeasure: Trim(txtEditItemUnit.Text),
        ReorderLevel: If(IsBlank(txtEditItemReorder.Text), Blank(), Value(txtEditItemReorder.Text)),
        UnitCost: If(IsBlank(txtEditItemCost.Text), Blank(), Value(txtEditItemCost.Text)),
        IsActive: tglEditItemActive.Value                    // DEF-11: can be switched back on
    }),
    Notify("The item was not saved. " & FirstError.Message, NotificationType.Error),
    Select(btnLoad_AdminCatalogue);
    Set(varShowEditItem, false);
```

```
    Notify("Item saved.", NotificationType.Success)
);
Set(varBusy, false)
)

// btnSaveAdjustment.OnSelect  (ddAdjustItem = active items; txtAdjustQty signed; txtAdjustReason)
If(varBusy || IsBlank(Trim(txtAdjustReason.Text)) || Value(txtAdjustQty.Text) = 0, false,
Set(varBusy, true);
Refresh(Catalogue);
With(
    { live: LookUp(Catalogue, OfficeItemKey = ddAdjustItem.Selected.OfficeItemKey), delta:
Value(txtAdjustQty.Text) },
    If(
        live.StockQty + delta < 0,
        Notify("That would leave " & (live.StockQty + delta) & " in stock. Stock cannot go below zero.",
NotificationType.Error),
        IfError(
            Collect(StockLedger, {
                Title: live.OfficeItemKey & " " & Text(Today(), "yyyy-mm-dd"),
                OfficeItemKey: live.OfficeItemKey, OfficeCode: live.OfficeCode, ItemCode: live.ItemCode,
                MovementType: { Value: "Adjustment" }, QtyDelta: delta, BalanceAfter: live.StockQty + delta,
                MovedOn: Now(), MovedByEmail: varUserEmail, Notes: Trim(txtAdjustReason.Text)
            }),
            Notify("Stock was not adjusted. " & FirstError.Message, NotificationType.Error),
            Patch(Catalogue, live, { StockQty: live.StockQty + delta, RowVersion: live.RowVersion + 1 }),
            Notify("The ledger row was written but the stock figure was not. " & FirstError.Message,
NotificationType.Error),
            Select(btnLoad_AdminCatalogue);
            Set(varShowAdjust, false);
            Notify("Stock adjusted.", NotificationType.Success)
        )
    )
);
Set(varBusy, false)
)
```

15.4 Offices and sharing

Office code. A new office gets a suggested code that follows the name until the user types over it (`src/domain/officeCode.ts`): a headquarters or a "Country Office" is its first letter + `CO` (Juba Country Office → `JCO`); anything else is the first three letters of the place (Bor Zonal Office → `BOR`); a clash gets a number (`BOR2`). The rules for any code: 2–10 capital letters or digits, unique ignoring case.

```
// lblCodeSuggestion.Text  (a hidden label in the office dialog)
With(
    { name: Trim(txtOfficeName.Text) },
    With(
        // The place name: drop "(…)" and a trailing "Zonal Office" / "Field Office" / "Country Office"…
        { place: Coalesce(
              Match(If(IsBlank(Find("(", name)), name, Left(name, Find("(", name) - 1)),
                    "^(?<p>.*?)\s*((country|zonal|zone|field|sub|area|regional)[\s-]*)?offices?\s*$",
                    MatchOptions.IgnoreCase).p,
              name) },
        With(
            { letters: Upper(Concat(ForAll(Sequence(Len(place)) As i, Mid(place, i.Value, 1)) As ch,
                                    If(IsMatch(ch.Value, "[A-Za-z]"), ch.Value, ""))) },
            With(
                { base: If(
                      IsBlank(letters), "OFC",
                      ddOfficeType.Selected.Value = "Headquarters" || IsMatch(name, "country\s+office",
MatchOptions.Contains & MatchOptions.IgnoreCase),
                      Left(letters, 1) & "CO",
                      Len(letters) >= 3, Left(letters, 3),
                      Left(letters & "XX", 2)) },
                If(
                    IsBlank(LookUp(colOffices, Upper(Title) = base)),
```

```
                    base,
                    base & First(Filter(Sequence(98, 2) As s, IsBlank(LookUp(colOffices, Upper(Title) = base &
s.Value)))).Value
                )
            )
        )
    )
)

// txtOfficeCode.Default    If(varCodeTouched, varFrozenCode, lblCodeSuggestion.Text)
// txtOfficeCode.OnChange   Set(varCodeTouched, true); Set(varFrozenCode, Upper(Trim(Self.Text)))
// Opening the dialog to ADD:   Set(varEditOffice, Blank()); Set(varCodeTouched, false); ...
// Opening the dialog to EDIT:  Set(varEditOffice, ThisItem); Set(varCodeTouched, true);
//                              Set(varFrozenCode, ThisItem.Title); ...

// lblOfficeCodeError.Text  (the code is checked against every office except the one being edited)
With({ code: Upper(Trim(txtOfficeCode.Text)) },
    If(
        IsBlank(code), "An office code is required.",
        !IsMatch(code, "^[A-Z0-9]{2,10}$"), "Use 2–10 capital letters or digits, with no spaces.",
        !IsBlank(LookUp(colOffices, Upper(Title) = code && ID <> Coalesce(varEditOffice.ID, -1))),
            "The code " & code & " is already used by another office.",
        ""
    ))
```

```
// btnSaveNewOffice.OnSelect  (DisplayMode disabled while lblOfficeCodeError.Text <> "")
If(varBusy || !IsBlank(lblOfficeCodeError.Text), false,
Set(varBusy, true);
With({ code: Upper(Trim(txtOfficeCode.Text)) },
    If(
        !IsBlank(LookUp(Offices, Title = code)),       // live check, someone may have just taken it
        Notify("The code " & code & " is already used by another office.", NotificationType.Error),
        IfError(
            Patch(Offices, Defaults(Offices), {
                Title: code, OfficeName: Trim(txtOfficeName.Text),
                OfficeType: { Value: ddOfficeType.Selected.Value },
                Country: Trim(txtOfficeCountry.Text), IsActive: tglOfficeActive.Value
            }),
            Notify("The office was not added. " & FirstError.Message, NotificationType.Error),
            ClearCollect(colOffices, Offices);
            Set(varShowOfficeDialog, false);
            Notify("Office " & code & " added.", NotificationType.Success)
        )
    )
);
Set(varBusy, false)
)

// btnSaveNewSharing.OnSelect  (ddShareFrom, ddShareTo <> ddShareFrom, txtSharePriority default 1)
If(varBusy || ddShareFrom.Selected.Title = ddShareTo.Selected.Title, false,
Set(varBusy, true);
IfError(
    Patch(OfficeSharing, Defaults(OfficeSharing), {
        Title: ddShareFrom.Selected.Title & " → " & ddShareTo.Selected.Title,
        FromOfficeCode: ddShareFrom.Selected.Title, ToOfficeCode: ddShareTo.Selected.Title,
        SharePriority: Coalesce(Value(txtSharePriority.Text), 1), IsActive: true
    }),
    Notify("The sharing link was not added. " & FirstError.Message, NotificationType.Error),
    ClearCollect(colSharing, Filter(OfficeSharing, IsActive = true));
    Set(varShowAddSharing, false);
    Notify("Sharing link added.", NotificationType.Success)
);
Set(varBusy, false)
)
```

15.5 Edit an office, and renaming its code

Changing an office's code rewrites every row that carries it, in the order of `OFFICE_CODE_TARGETS` (`src/domain/officeCode.ts`), with the office row itself last, so a run that fails half-way can simply be run again.

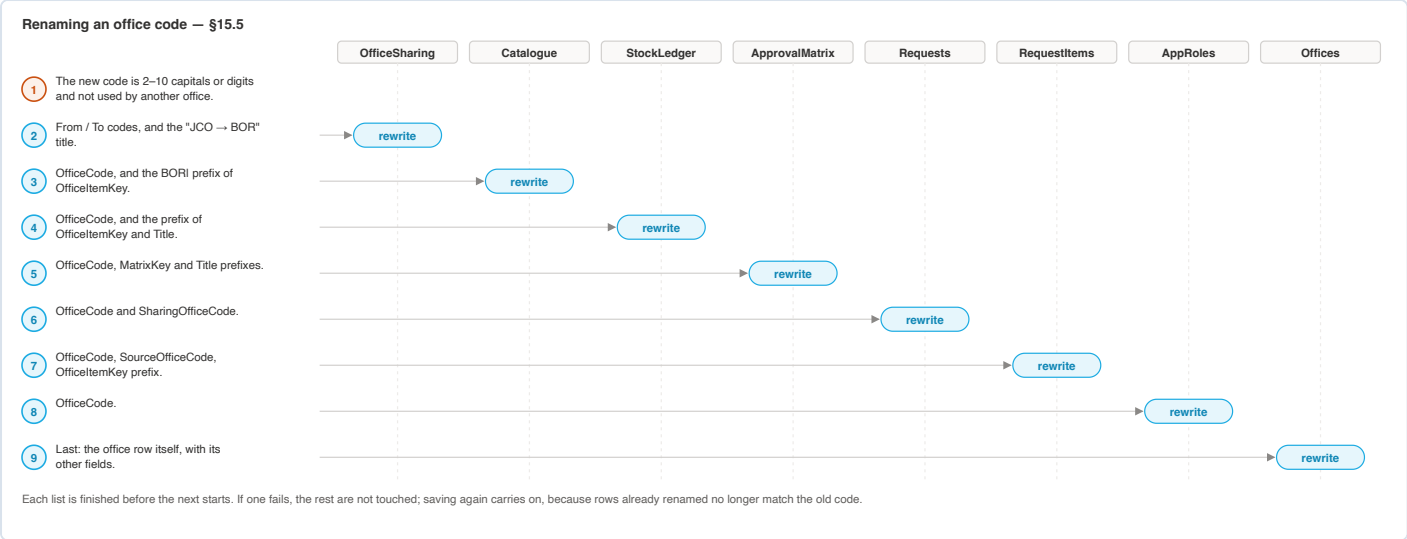


Figure 21. Renaming an office code: the lists rewritten, in order

```
// btnSaveOffice.OnSelect  (varEditOffice = the Offices row being edited)
// Power Apps will not let ForAll patch the list it is looping over, so each
// list's rows are first copied into a collection of their own.
If(varBusy || !IsBlank(lblOfficeCodeError.Text), false,
Set(varBusy, true);
Clear(colErrors);
Set(varOld, varEditOffice.Title);
Set(varNew, Upper(Trim(txtOfficeCode.Text)));

If(varNew <> varOld && !IsBlank(LookUp(Offices, Title = varNew)),
    Collect(colErrors, { Msg: "The code " & varNew & " is already used by another office." }));

If(varNew <> varOld && IsEmpty(colErrors),
    // 1. OfficeSharing: both code columns, and "JCO → BOR" in Title
    ClearCollect(colRnSharing, Filter(OfficeSharing, Or(FromOfficeCode = varOld, ToOfficeCode = varOld)));
    ForAll(colRnSharing As r, IfError(
        Patch(OfficeSharing, r, {
            FromOfficeCode: If(r.FromOfficeCode = varOld, varNew, r.FromOfficeCode),
            ToOfficeCode:   If(r.ToOfficeCode = varOld, varNew, r.ToOfficeCode),
            Title: Concat(Split(r.Title, " → ") As p, If(p.Value = varOld, varNew, p.Value), " → ")
        }),
        Collect(colErrors, { Msg: "OfficeSharing: " & FirstError.Message })))
);
If(varNew <> varOld && IsEmpty(colErrors),
    // 2. Catalogue: OfficeCode, and the "BOR|" prefix of OfficeItemKey
    ClearCollect(colRnCatalogue, Filter(Catalogue, OfficeCode = varOld));
    ForAll(colRnCatalogue As r, IfError(
        Patch(Catalogue, r, {
            OfficeCode:    varNew,
            OfficeItemKey: If(StartsWith(r.OfficeItemKey, varOld & "|"), varNew & Mid(r.OfficeItemKey,
Len(varOld) + 1), r.OfficeItemKey)
        }),
        Collect(colErrors, { Msg: "Catalogue: " & FirstError.Message })))
);
If(varNew <> varOld && IsEmpty(colErrors),
    // 3. StockLedger: OfficeCode, and the "BOR|" prefix of OfficeItemKey and Title
    ClearCollect(colRnLedger, Filter(StockLedger, OfficeCode = varOld));
    ForAll(colRnLedger As r, IfError(
        Patch(StockLedger, r, {
            OfficeCode:    varNew,
            OfficeItemKey: If(StartsWith(r.OfficeItemKey, varOld & "|"), varNew & Mid(r.OfficeItemKey,
```

```
Len(varOld) + 1), r.OfficeItemKey),
            Title:         If(StartsWith(r.Title, varOld & "|"), varNew & Mid(r.Title, Len(varOld) + 1),
r.Title)
        }),
        Collect(colErrors, { Msg: "StockLedger: " & FirstError.Message })))
);
If(varNew <> varOld && IsEmpty(colErrors),
    // 4. ApprovalMatrix: OfficeCode, the "BOR|" prefix of MatrixKey, the "BOR · " prefix of Title
    ClearCollect(colRnMatrix, Filter(ApprovalMatrix, OfficeCode = varOld));
    ForAll(colRnMatrix As r, IfError(
        Patch(ApprovalMatrix, r, {
            OfficeCode: varNew,
            MatrixKey:  If(StartsWith(r.MatrixKey, varOld & "|"), varNew & Mid(r.MatrixKey, Len(varOld) + 1),
r.MatrixKey),
            Title:      If(StartsWith(r.Title, varOld & " · "), varNew & Mid(r.Title, Len(varOld) + 1),
r.Title)
        }),
        Collect(colErrors, { Msg: "ApprovalMatrix: " & FirstError.Message })))
);
If(varNew <> varOld && IsEmpty(colErrors),
    // 5. Requests: OfficeCode and SharingOfficeCode
    ClearCollect(colRnRequests, Filter(Requests, Or(OfficeCode = varOld, SharingOfficeCode = varOld)));
    ForAll(colRnRequests As r, IfError(
        Patch(Requests, r, {
            OfficeCode:        If(r.OfficeCode = varOld, varNew, r.OfficeCode),
            SharingOfficeCode: If(r.SharingOfficeCode = varOld, varNew, r.SharingOfficeCode)
        }),
        Collect(colErrors, { Msg: "Requests: " & FirstError.Message })))
);
If(varNew <> varOld && IsEmpty(colErrors),
    // 6. RequestItems: OfficeCode, SourceOfficeCode, and the "BOR|" prefix of OfficeItemKey
    ClearCollect(colRnItems, Filter(RequestItems, Or(OfficeCode = varOld, SourceOfficeCode = varOld)));
    ForAll(colRnItems As r, IfError(
        Patch(RequestItems, r, {
            OfficeCode:       If(r.OfficeCode = varOld, varNew, r.OfficeCode),
            SourceOfficeCode: If(r.SourceOfficeCode = varOld, varNew, r.SourceOfficeCode),
            OfficeItemKey:    If(StartsWith(r.OfficeItemKey, varOld & "|"), varNew & Mid(r.OfficeItemKey,
Len(varOld) + 1), r.OfficeItemKey)
        }),
        Collect(colErrors, { Msg: "RequestItems: " & FirstError.Message })))
);
If(varNew <> varOld && IsEmpty(colErrors),
    // 7. AppRoles
    ClearCollect(colRnRoles, Filter(AppRoles, OfficeCode = varOld));
    ForAll(colRnRoles As r, IfError(
        Patch(AppRoles, r, { OfficeCode: varNew }),
        Collect(colErrors, { Msg: "AppRoles: " & FirstError.Message })))
);

// 8. Last: the office row itself, with the other fields.
If(IsEmpty(colErrors),
    IfError(
        Patch(Offices, LookUp(Offices, ID = varEditOffice.ID), {
            Title: varNew, OfficeName: Trim(txtOfficeName.Text),
            OfficeType: { Value: ddOfficeType.Selected.Value },
            Country: Trim(txtOfficeCountry.Text), IsActive: tglOfficeActive.Value
        }),
        Collect(colErrors, { Msg: "Offices: " & FirstError.Message })));

Set(varBusy, false);
ClearCollect(colOffices, Offices);
ClearCollect(colMatrix, Filter(ApprovalMatrix, IsActive = true));
ClearCollect(colSharing, Filter(OfficeSharing, IsActive = true));
If(
    IsEmpty(colErrors),
    Set(varShowOfficeDialog, false); Notify("Office saved.", NotificationType.Success),
    Notify("The save stopped part-way (" & First(colErrors).Msg & "). Fix the cause and save again: "
           & "rows already renamed are skipped, so it carries on from where it stopped.",
NotificationType.Error)
```

```
)
)
```

Limits to know about. Each `Filter` above returns at most the app's data row limit (2,000 rows). `RequestItems.OfficeCode`, `SourceOfficeCode` and `Requests.SharingOfficeCode` are not indexed, so once those lists pass 5,000 rows SharePoint refuses to filter on them. Renaming is rare; when an office has more rows than that, run the rename as a Power Automate flow instead (same lists, same order).

Column-reference audit

Every column named anywhere in this file, checked against `docs/schema.md`:

List	Columns used here	All present in schema.md
`Offices`	(whole list collected)	yes
`OfficeSharing`	`IsActive`	yes
`Catalogue`	`Title`, `ItemCode`, `OfficeCode`, `OfficeItemKey`, `StockQty`, `ReorderLevel`, `RowVersion`, `IsActive`, `ID`	yes
`StockLedger`	`Title`, `OfficeItemKey`, `OfficeCode`, `ItemCode`, `MovementType`, `QtyDelta`, `BalanceAfter`, `RequestRefNo`, `MovedOn`, `MovedByEmail`, `MovedBy`, `Notes`, `ID`	yes
`ApprovalMatrix`	`MatrixKey`, `StageCode`, `ApproverEmail`, `IsActive`, `ID`	yes
`Requests`	`Title`, `OfficeCode`, `Section`, `RequesterEmail`, `CurrentlyWithEmail`, `CurrentlyWithOICEmail`, `Status`, `StatusRank`, `CurrentStageCode`, `Purpose`, `SubmittedOn`, `LineCount`, `TotalQtyRequested`, `TotalQtyDistributed`, `SharingOfficeCode`	yes
`RequestItems`	`RequestRefNo`, `LineNo`, `QtyApproved`, `QtyDistributed`, `SourceOfficeCode`, `LineStatus`, `IsCatalogueItem`, `ID`	yes
`RequestTimeline`	`Title`, `RequestRefNo`, `EventType`, `FromStatus`, `ToStatus`, `StageCode`, `ActorEmail`, `OccurredOn`, `Detail`	yes
`AppRoles`	`UserEmail`, `OfficeCode`, `RoleCode`, `IsActive`	yes
`RefCounter`	`Title`, `LastSeq`, `ID`	yes

`ID` is SharePoint's built-in item id, present on every list and not enumerated in `schema.md`. `RequestComments` and `RequestDocuments` are added as data sources but are not referenced by any formula in this file — they belong to `scrRequestDetail` in Phase 3, where `RequestComments.CommentAuthor` (**not** `Author`) and `RequestComments.AuthorEmail` are the columns to use.

Open uncertainties, collected

1. Whether `App.Formulas` may reference a collection (`colMyRoles`). Section 1 assumes not and puts the role booleans in `OnStart`. Test once; prefer the named-formula form if Studio accepts it.
2. `Sort(StockLedger, ID, SortOrder.Descending)` delegation. Verify at row limit 1.
3. ~~`Font.'Courier New'` as the only guaranteed monospace face.~~ **RESOLVED in Studio:** confirmed present and the only monospace offered. `Segoe UI` is also confirmed present, so §1.7 stands unchanged.

4. Unicode glyphs `◐` and `▣` in a canvas Label; Icon-enum fallback given.
5. The Person-column `'@odata.type'` patch shape for `StockLedger.MovedBy`.
6. `Text(Year(Now()))` and locale thousands separators.
7. `Select(btnReconcile)` across screens — inline the body if the button is not on the calling screen.
8. `AddColumns` name collision between `tblJourney.StageCode` and `colMatrix.StageCode`; needs `As` aliases in Phase 3. **Resolved in Section 7, Recipe R1:** `ForAll(... As leg, ...)` with `leg.StageCode` against the matrix row's bare `StageCode`.

14. The document flow

The one part of the app that is not in Studio. The text below is `powerapps/flows/UploadRequestDocument.md`.

A canvas app can read the `RequestDocuments` library but cannot put a file into it, because the library is a document library, not a list with attachments (schema.md §10). This Power Automate flow does it for the app. The app calls it from **Attach a file** on Request detail (`app-glue.md` §14).

What it does: it receives the file and its details, refuses a file whose stored name is already in the library, creates the file, fills in its three metadata columns, and tells the app whether it worked.

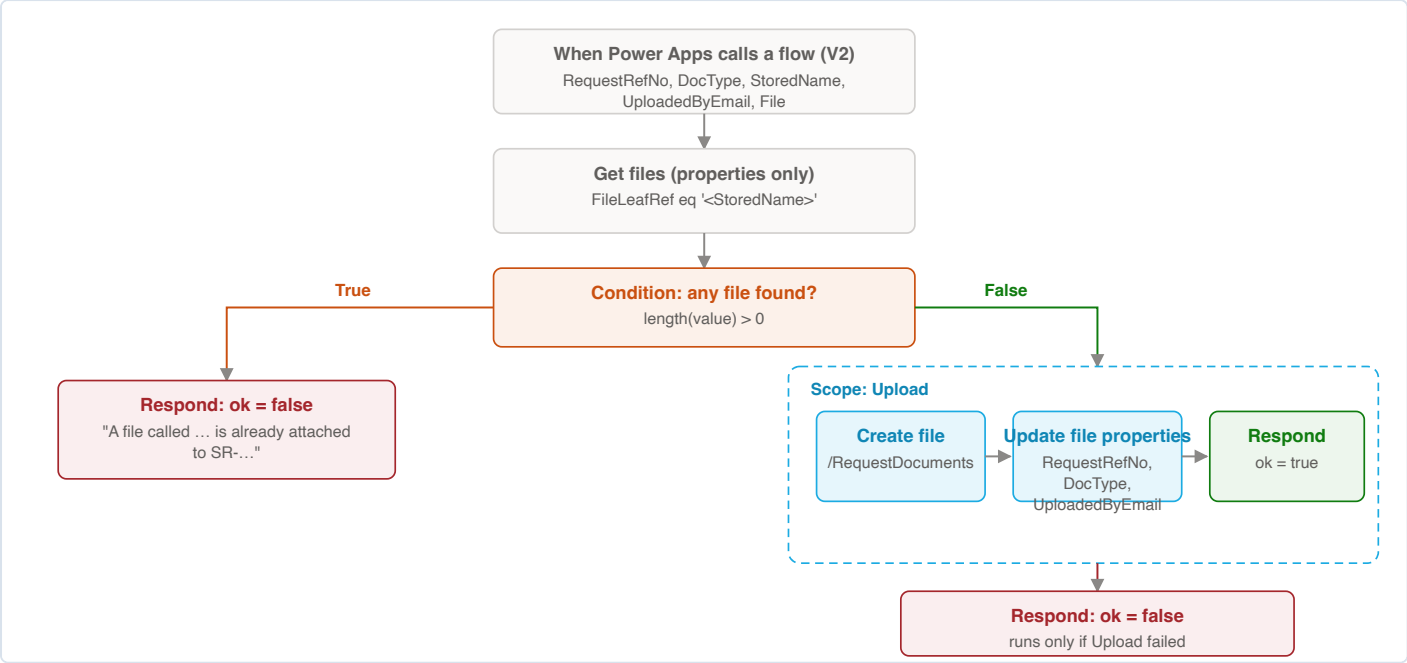


Figure 22. The UploadRequestDocument flow

Build it

1. Go to **make.powerautomate.com**, in the same environment as the app, and select **Create › Instant cloud flow**.
2. Name it `UploadRequestDocument`, choose the trigger **When Power Apps calls a flow (V2)**, and select **Create**.
3. In the trigger, add these inputs **in this order**. The app passes its values in the same order. Make all of them required.

#	Type	Name
1	Text	`RequestRefNo`
2	Text	`DocType`
3	Text	`StoredName`
4	Text	`UploadedByEmail`

#	Type	Name
5	File	`File`

4. Add **SharePoint › Get files (properties only)**:
   - **Site Address:** the Office Supplies site. **Library Name:** `RequestDocuments`.
   - **Filter Query:** `FileLeafRef eq '` `StoredName` `'` (the stored name between single quotes).
5. Add a **Condition**: `length(body('Get_files_(properties_only)')?['value'])` *is greater than* `0`.
6. In **True** (the name is taken), add **Respond to a PowerApp or flow** with two text outputs:
   - `ok` = `false`
   - `message` = `A file called` `StoredName` `is already attached to` `RequestRefNo` `. Rename it and try again.`
7. In **False**, add a **Scope** named `Upload`, and inside it:
   - **SharePoint › Create file**: **Site Address** the Office Supplies site, **Folder Path** `/RequestDocuments`, **File Name** `StoredName`, **File Content** the trigger's `File` content.
   - **SharePoint › Update file properties**: **Library Name** `RequestDocuments`, **Id** `ItemId` from **Create file**, then **RequestRefNo** = `RequestRefNo`, **DocType Value** = `DocType`, **UploadedByEmail** = `UploadedByEmail`.
   - **Respond to a PowerApp or flow**: `ok` = `true`, `message` = (leave empty).
8. Still in **False**, *after* the scope, add **Respond to a PowerApp or flow** with `ok` = `false` and `message` = `The document was not attached. Try again, or ask the site owner to check your access.`. Open its ⋯ › **Settings** › **Run after**, untick *is successful* and tick **`Upload`** **has failed** and **has timed out**. It only runs if something inside the scope failed.
9. Select **Save**, then go back to the app.

Connect it to the app

1. In Studio, select **Power Automate** in the left rail (the flow icon), then **Add flow**, and pick `UploadRequestDocument`.
2. The app can now call `UploadRequestDocument.Run(RequestRefNo, DocType, StoredName, UploadedByEmail, { contentBytes: …, name: … })`, which returns a record with `ok` and `message`. §14 shows the full button formula.

> ✓ **You should now see**
>
> Attach a small PDF to a request. It appears in the `RequestDocuments` library as `SR–…_yourfile.pdf` with `RequestRefNo`, `DocType` and `UploadedByEmail` filled in, and on the request's Documents tab and timeline. Attach the same file again and the app says it is already attached.

Notes

- The flow runs as the person using the app, with their SharePoint permission. They need Contribute on the site, as for everything else.
- The app also checks the name against the documents it has already loaded before calling the flow, so the usual duplicate never reaches it. The flow's own check catches a file that someone else attached after the app loaded its list.
- Power Apps passes files up to about 50 MB this way; very large scans should be uploaded straight to the library.

15. Test, run UAT and release

Step 1 — the delegation check, at a row limit of 1

Do this when every screen shows data and **before** any button that saves has been pressed (chapter 2 explains why).

1. Check **Settings** › **General** › **Data row limit** is still **1**.
2. Open each screen in Preview, as the persona who sees most on it, and check the lists below still show all their rows. A list that shows one row, or none, has a formula SharePoint can't run.

Screen	The list to check	Expected with the seeded data
Home	`galAwaiting`, `galMine`	Every request the persona holds / raised
Work queue	`galQueue`	Every request at that stage
New request	the product picker	Every active item of the office
Request detail	items, timeline, comments	Every line and every event
Review	`galReview`	Every line, with stock
Distribute	`galDist`	Every line still owed
Admin	`galCatalogue`	Every item of the office, active or not
Reports	the tiles	The same numbers as at 2,000

The four start-up collections (`colOffices`, `colMatrix`, `colSharing`, `colMyRoles`) hold one row each at this limit. That's expected: they are loaded whole, by design, and are checked at 2,000.

> ⚠ **If a list comes back with one row**
>
> Hover over the blue underline or the warning triangle on its formula: Studio names the part it can't send to SharePoint. The fix is nearly always in the formula's filter (use `=`, `<`, `>` on an indexed text, number or date column; never `<>`, `Not`, `IsBlank` or `Search` on a SharePoint list), not in the screen. Appendix A lists the indexed columns.

What this guide's queries rely on

Every query in this guide was checked against the index inventory in Appendix A. They fall into four groups:

Group	Examples	Why it is safe
Equality on an indexed text key	`Filter(RequestItems, RequestRefNo = varRef)`, `LookUp(Catalogue, OfficeItemKey = …)`, `LookUp(Requests, Title = varRef)`	SharePoint filters on the index, at any list size
Ranges on an indexed number or date	`StatusRank < 70`, `SubmittedOn >= varSince`, `OccurredOn >= varSince`, `MovedOn >= varSince`	Same; `Or` of two indexed equalities (Home's holder test) also delegates

Group	Examples	Why it is safe
An indexed column first, then a Yes/No	`Filter(Catalogue, OfficeCode = …, IsActive = true)`	The indexed `OfficeCode` narrows the rows before SharePoint looks at `IsActive`
Small lists, loaded whole	`Offices`, `OfficeSharing`, `ApprovalMatrix`, `AppRoles`, `RefCounter`	Tens of rows, far below any limit

Two places are known exceptions. Neither is a mistake, and neither should be "fixed" in the formula:

- **Reports** wraps its three reads in `FirstN(…, numReportCap)`. Studio underlines `FirstN` with a delegation warning. The filter inside it does delegate; `FirstN` only trims the result to 500 in the app, and the "Capped at 500" flag says so.
- **Renaming an office code** (§15.5) filters `Requests.SharingOfficeCode`, `RequestItems.OfficeCode` and `RequestItems.SourceOfficeCode`, which are not indexed. It works while those lists are under 5,000 rows; §15.5 says what to do after that.

Step 2 — raise the limit and test the buttons

1. Set **Data row limit** to **2000**.
2. Check `colMatrix` holds the seeded matrix rows and `colMyRoles` your roles (**Variables**, the *(x)* icon).
3. Wire up and test the buttons in the order of chapter 13.

> ✓ **The test that matters most**
>
> Submit one request and check SharePoint: exactly **one** new row in `Requests`, its lines in `RequestItems`, and two rows in `RequestTimeline`. Then submit two requests at the same moment from two browsers: both get references, and they differ.

Step 3 — user acceptance testing

Run the 31 cases the SPFx build passed, in the same order and with the same people. The full list, with where each is tested in the canvas app, is below. Each case's steps and expected results are in `pa.init.officesupplies/uat/scenarios/<case>.md`; they describe the same screens, so they apply to the canvas app unchanged, with one difference: instead of the web part's "impersonate email" setting, add `&as=<email>` to the app link.

To get the app link: in **make.powerapps.com** › **Apps**, select ⋯ › **Details** on Office Supplies and copy the **Web link**. It looks like `https://apps.powerapps.com/play/e/<environment>/a/<app id>?tenantId=…`. Add `&as=daniel.otieno@…` to the end to open it as that persona.

UAT cases

Run them in block order. Where a block impersonates a seeded person, open the app link with `&as=<their email from AppRoles>` (it needs `cfgAllowImpersonation = true`).

Block 1 Test account, self-routed office

No impersonation.

Case	Title	Where it is tested in the canvas app
UAT-001	Add a new office with a generated code	Catalogue & matrix › Offices & sharing: + **Add office**, the suggested code (ch. 11, §15.4)
UAT-002	A clashing office code gets a number	Office dialog: `lblCodeSuggestion` adds a number on a clash; `lblOfficeCodeError` (§15.4)
UAT-003	Build an approval chain for the test office	Approval matrix: + **Add step** four times, then **Validate chain** (§15.1, §15.2)
UAT-004	Add catalogue items and load stock	Catalogue & stock: + **Add catalogue item**, then **Adjust stock** (§15.3)
UAT-005	Let the country office lend stock to the test office	Offices & sharing: + **Add sharing link** (§15.4)
UAT-010	A request with no items cannot be submitted	New request: **Submit** with no quantity shows the error, writes nothing (§8)
UAT-011	Save a request as a draft	New request: **Save draft** (§8); Home shows it with **Edit**
UAT-012	Submit a request from the catalogue	New request: catalogue picker, **Submit** (§8); Request detail shows it at Review
UAT-013	Review a request and change a quantity	Review: change a quantity, note, **Proceed** (§10)
UAT-014	Return a request, then edit and resubmit it	Review: **Return** (§11); New request in edit mode, **Resubmit** (§9)
UAT-015	Reject a request	Review: **Reject** (§11); journey stops at its stage (DEF-141)
UAT-016	Confirm a reviewed request	Review at the Confirm stage: "Confirm SR-…", **Proceed** (§10)
UAT-017	Distribute part of a request	Distribute: less than approved, **Record distribution**; stays at Deliver (§12)
UAT-018	Distribute the rest of a request	Distribute again: the remainder; moves to Receive (§12)
UAT-019	Confirm receipt and close the request	Confirm receipt: **Confirm receipt** closes the request (§13)
UAT-020	Stock is offered from the lending office	Review: "⇄ From sharing office" and the lender stock (ch. 8 loader)
UAT-021	Comment on a request and attach a document	Request detail: **Post comment**, **Attach a file** and the flow (§14, ch. 14)

Block 2 Reviewer / requester persona (Daniel Otieno)

Impersonate Daniel Otieno.

Case	Title	Where it is tested in the canvas app
UAT-030	A reviewer sees stock hints on a request	Review as the seeded reviewer: availability badges (ch. 8)
UAT-031	A reviewer approves a request	Review as the seeded reviewer: **Proceed** (§10)
UAT-035	A requester fixes and resubmits a returned request	Home **Edit** on a Returned request; return reason shown; **Resubmit** (§9)

Case	Title	Where it is tested in the canvas app
UAT-040	Home shows my work and my requests	Home tiles and both lists as the reviewer/requester persona (ch. 4)

Block 3 Chief Field Office persona (Fatuma Noor)

Impersonate Fatuma Noor.

Case	Title	Where it is tested in the canvas app
UAT-032	The Chief Field Office confirms a request	Review at Confirm as the Chief Field Office persona: **Proceed** (§10)

Block 4 Storekeeper persona (Samuel Kiptoo)

Impersonate Samuel Kiptoo.

Case	Title	Where it is tested in the canvas app
UAT-033	A storekeeper distributes and stock runs out	Distribute as the storekeeper persona: "flags out of stock" (ch. 9, §12)

Block 5 Requester persona again (Daniel Otieno)

Impersonate Daniel Otieno.

Case	Title	Where it is tested in the canvas app
UAT-034	A requester confirms receipt of a delivery	Confirm receipt as the requester persona (§13)

Block 6 Empty-account persona (Peter Mwangi)

Impersonate Peter Mwangi.

Case	Title	Where it is tested in the canvas app
UAT-041	Home explains when there is nothing to show	Home as a persona with nothing: empty messages (ch. 4)

Block 7 Test account, reports and rename

No impersonation.

Case	Title	Where it is tested in the canvas app
UAT-042	Reports show activity and respond to filters	Reports: both filters change every figure (ch. 12)
UAT-050	Rename an office code and keep its data	Edit office with a new code: the rename across seven lists (§15.5)

Block 8 End-to-end journeys

Test account (no impersonation).

Case	Title	Where it is tested in the canvas app
E2E-01	A catalogue request from set-up to a closed request	Admin set-up, then submit → review → confirm → distribute → receipt

Case	Title	Where it is tested in the canvas app
E2E-02	A partial delivery and then the remainder	Two distributions; the request stays at Deliver between them
E2E-03	A request is returned, discussed, corrected and approved	Return, comment, resubmit, approve
E2E-04	Short stock is met by another office	A short line filled from the lending office, distributed from it

Step 4 — release

1. **Turn off testing access.** In `App.Formulas`, set `cfgAllowImpersonation = false`.
2. **Save and publish. File › Save**, then **Publish › Publish this version**.
3. **Share.** Select **Share** on the app, add the users (a Microsoft Entra security group is easiest), and leave **Co-owner** unticked for everyone but makers. Each user also needs **Contribute** on the SharePoint site, and must exist in `AppRoles` to see their work.
4. **Share the flow.** Users run `UploadRequestDocument` as themselves, so nothing more is needed. If the flow's owner leaves, add a co-owner to the flow so it keeps working.
5. **Tell users the first launch asks for permission** to use SharePoint: they select **Allow**.

Moving between environments

If you build in a test environment and release to production, put the app and the flow in a **solution**:

1. In **make.powerapps.com › Solutions**, create a solution called `Office Supplies` with your publisher.
2. Select **Add existing › App › Canvas app** and pick the app, then **Add existing › Automation › Cloud flow** and pick `UploadRequestDocument`.
3. Studio creates **connection references** for the SharePoint connection. When you import the solution into production, you pick the production connection for each.
4. If production uses a different SharePoint site, turn on the Studio setting **Automatically create environment variables when adding data sources** (under **Settings**) before adding the lists, so the site address becomes an environment variable you can change on import.
5. Export the solution as **managed** for production, **unmanaged** for keeping in source control.

Keeping this repository in step with the app

The app lives in Studio; the repository keeps a copy of each screen so changes can be reviewed.

1. In Studio, right-click a screen in the Tree view and select **View code**, then **Copy code**.
2. On macOS, in a terminal at the repository root: `pbpaste > powerapps/screens/<screen>.pa.yaml`. On Windows, paste into the file in a text editor.
3. Commit the change. Studio removes comments and sorts properties alphabetically, so the copy won't look like what you typed; that's expected (`oracle/NOTES.md` §19).

Keep `App.Formulas.fx`, `App.OnStart.fx` and `app-glue.md` §7–§15 in step the same way: when you change a formula in Studio, change it here too.

Appendix A. The lists and their columns

This is `docs/schema.md`, the definitive contract for the SharePoint site, from its first list to its stage table. Internal names are the names formulas use. The rules that hold across every list, and the index inventory, are at the end.

1. Offices

Base template `GenericList`. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Office code	Single line of text (built-in)	Yes	Yes	The office code. `JCO`, `BOR`, `MAL`, `BEN`, `WAU`. Reconfigured with `spo field set`.
`OfficeName`	Office name	Single line of text	Yes	No	`Juba Country Office`, `Bor Zonal Office`, `Malakal Zonal Office`, `Bentiu Zonal Office`, `Wau Zonal Office`
`OfficeType`	Office type	Choice	Yes	No	`Headquarters`, `Zonal office`, `Field office` — `Headquarters` and `Field office` verbatim from `admin.html` tab 3; `Zonal office` added for UNICEF South Sudan (see the judgement-call note below)
`Country`	Country	Single line of text	No	No	Not stated in the mock-up; seeded `South Sudan`
`IsActive`	Active	Yes/No	No	No	Default `1`

2. OfficeSharing

Base template `GenericList`. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Sharing link	Single line of text (built-in)	Yes	No	Human label, e.g. `JCO → BOR`
`FromOfficeCode`	From office code	Single line of text	Yes	No	The primary office that **lends**
`ToOfficeCode`	To office code	Single line of text	Yes	No	The secondary office that **borrows**
`SharePriority`	Share priority	Number (0 dp)	Yes	No	`Min="1"`. Lower number wins when several offices can lend.
`IsActive`	Active	Yes/No	No	No	Default `1`

3. Catalogue

Base template `GenericList`. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Item name	Single line of text (built-in)	Yes	No	Verbatim item name, e.g. `A4 photocopy paper (ream)`
`ItemCode`	Item code	Single line of text	Yes	No	Stable per-item code, unique **within** an office
`OfficeCode`	Office code	Single line of text	Yes	**Yes**	
`OfficeItemKey`	Office item key	Single line of text	Yes	**Yes — unique**	`EnforceUniqueValues="TRUE"`. Composed `<OfficeCode>|<ItemCode>`, e.g. `BOR|A4PAPER`
`Category`	Category	Single line of text	No	No	The mock-up states no categories — free text rather than an invented Choice
`UnitOfMeasure`	Unit of measure	Single line of text	No	No	`Ream`, `Box of 10`, `Pack of 5`, `Each`, `Pad`, `Cartridge`, `Box of 50`
`StockQty`	Stock quantity	Number (0 dp)	Yes	No	`Min="0"`. A **cache** — the ledger is truth.
`ReorderLevel`	Low-stock threshold	Number (0 dp)	No	No	`Min="0"`. Blank = no threshold (the mock-up shows `–` for Fax toner).
`UnitCost`	Unit cost	Currency (2 dp)	No	No	Not stated in the mock-up; left blank
`RowVersion`	Row version	Number (0 dp)	No	No	`Min="0"`. Bumped on every stock write for optimistic concurrency.
`IsActive`	Active	Yes/No	No	No	Default `1`. `FALSE` = the mock-up's **Discontinued** badge.

Derived, never stored: the mock-up's four catalogue badges. `Discontinued` when `IsActive = false`; `Out of stock` when `StockQty = 0`; `Low stock` when `StockQty <= ReorderLevel`; otherwise `Active`.

Not modelled: the mock-up's "Added by" column maps to SharePoint's built-in `Author` (Created By), which cannot be set at seed time and will show the account that runs `03-seed.sh`.

4. StockLedger

Base template `GenericList`. Append-only by convention (nothing in the app updates or deletes a row). **This list will exceed 5000 items** — every index below matters. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Entry	Single line of text (built-in)	Yes	No	Label, e.g. `BOR|A4PAPER 2026-07-13`
`OfficeItemKey`	Office item key	Single line of text	Yes	**Yes**	
`OfficeCode`	Office code	Single line of text	Yes	**Yes**	
`ItemCode`	Item code	Single line of text	Yes	No	

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`MovementType`	Movement	Choice	Yes	**Yes**	`Receipt`, `Distribution`, `Adjustment`, `Transfer in`, `Transfer out`, `Return`
`QtyDelta`	Quantity delta	Number (0 dp)	Yes	No	**Signed** — no `Min`. Negative for a distribution.
`BalanceAfter`	Balance after	Number (0 dp)	Yes	No	`Min="0"`. Computed before the `Catalogue` patch.
`RequestRefNo`	Request reference	Single line of text	No	**Yes**	Blank for a manual adjustment
`MovedOn`	Moved on	Date and time	Yes	**Yes**	
`MovedByEmail`	Moved by (email)	Single line of text	Yes	No	Shadow of `MovedBy`
`MovedBy`	Moved by	Person	No	No	
`Notes`	Notes	Multiple lines of text	No	No	Plain text, 4 lines

5. ApprovalMatrix

Base template `GenericList`. Versioning on (matrix changes are audited). Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Step	Single line of text (built-in)	Yes	No	Label, e.g. `BOR · Review`
`OfficeCode`	Office code	Single line of text	Yes	No	
`Section`	Section scope	Single line of text	Yes	No	A section name, or the literal `All sections` for an office-wide row. Text, not Choice, precisely so `All sections` is expressible.
`MatrixKey`	Matrix key	Single line of text	Yes	**Yes**	`<OfficeCode>|<Section>`, e.g. `BOR|All sections`
`StageCode`	Stage code	Choice	Yes	No	`Review`, `Approval`, `Confirm`, `Deliver`, `Receive`
`StageOrder`	Step	Number (0 dp)	Yes	No	`Min="1"`. The step number the admin screen shows.
`ActorRoleCode`	Acts as	Choice	Yes	No	`Requester`, `Reviewer`, `Chief Field Office`, `Storekeeper`, `Administrator`
`ApproverEmail`	Approver (email)	Single line of text	No	No	Shadow of `Approver`. Blank on the `Receive` row, which routes to the originator.
`Approver`	Approver	Person	No	No	
`OICEmail`	OIC (email)	Single line of text	No	No	Shadow of `OIC`

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`OIC`	Officer in charge	Person	No	No	
`IsActive`	Active	Yes/No	No	No	Default `1`

Resolution at runtime. The app computes `MatrixKey = OfficeCode & "|" & Section` and falls back to `OfficeCode & "|All sections"` — both are `MatrixKey = x`, so both delegate. A section-scoped row overrides an office-wide row for the same stage.

6. Requests

Base template `GenericList`. Versioning on. Attachments off (documents live in `RequestDocuments`).

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Reference	Single line of text (built-in)	Yes	**Yes — unique**	`EnforceUniqueValues="TRUE"`. `SR-YYYYMM-NNNN`. SharePoint enforces uniqueness; the app only retries.
`OfficeCode`	Office code	Single line of text	Yes	**Yes**	
`Section`	Section	Choice	Yes	No	`Programme`, `Child Protection`, `Health & Nutrition`, `Operations`, `Supply & Logistics`
`RequesterEmail`	Requester (email)	Single line of text	Yes	**Yes**	Shadow of `Requester`. Drives "My requests".
`Requester`	Requester	Person	No	No	
`CurrentlyWithEmail`	Currently with (email)	Single line of text	No	**Yes**	Shadow of `CurrentlyWith`. Drives "Awaiting my action".
`CurrentlyWith`	Currently with	Person	No	No	
`CurrentlyWithOICEmail`	Currently with OIC (email)	Single line of text	No	**Yes**	The OIC standing in for `CurrentlyWith`. No Person twin — it is a filter key only.
`Status`	Status	Choice	Yes	No	`Draft`, `Returned`, `Submitted`, `Reviewed`, `Partial delivery`, `Fully delivered`, `Fully received`, `Closed`, `Rejected` — verbatim, in this order
`StatusRank`	Status rank	Number (0 dp)	Yes	**Yes**	`Min="0"`. See the mapping table below. Written in the same `Patch` as `Status`.
`CurrentStageCode`	Current stage	Choice	No	No	`Review`, `Approval`, `Confirm`, `Deliver`, `Receive`

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Purpose`	Purpose	Choice	Yes	No	`Internal`, `External` — verbatim from `new-request.html`
`SubmittedOn`	Submitted on	Date and time	No	**Yes**	Blank while `Draft`
`ClosedOn`	Closed on	Date and time	No	No	Stamped by the auto-close path
`LineCount`	Line count	Number (0 dp)	Yes	No	`Min="0"`
`TotalQtyRequested`	Total requested	Number (0 dp)	No	No	`Min="0"`
`TotalQtyApproved`	Total approved	Number (0 dp)	No	No	`Min="0"`
`TotalQtyDistributed`	Total distributed	Number (0 dp)	No	No	`Min="0"`
`TotalQtyReceived`	Total received	Number (0 dp)	No	No	`Min="0"`
`SharingOfficeCode`	Sharing office code	Single line of text	No	No	Stamped when any line was sourced from a lending office
`ReturnReason`	Return reason	Multiple lines of text	No	No	Plain text. Return is blocked on an empty reason.
`RejectReason`	Reject reason	Multiple lines of text	No	No	Plain text. Reject is blocked on an empty reason.

`LineCount` and the four `TotalQty*` columns are **load-bearing, not convenience**. Without them the Home gallery needs a `CountRows` / `Sum` per row, which is both non-delegable and N+1.

7. RequestItems

Base template `GenericList`. The largest list in the design. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Line	Single line of text (built-in)	Yes	No	Label, e.g. `SR-202607-0038-01`
`RequestRefNo`	Request reference	Single line of text	Yes	**Yes**	
`OfficeCode`	Office code	Single line of text	Yes	No	The requesting office
`LineNo`	Line no.	Number (0 dp)	Yes	No	`Min="1"`
`ItemCode`	Item code	Single line of text	No	No	Blank on a free-text line
`ItemName`	Item name	Single line of text	Yes	No	**Snapshot** at line creation

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`IsCatalogueItem`	Catalogue item	Yes/No	No	No	Default `1`. `FALSE` for the "Other (type it in)…" line.
`OfficeItemKey`	Office item key	Single line of text	No	No	Blank on a free-text line
`UnitOfMeasure`	Unit of measure	Single line of text	No	No	**Snapshot** at line creation
`UnitCost`	Unit cost	Currency (2 dp)	No	No	**Snapshot** at line creation
`LinePurpose`	Purpose	Single line of text	No	No	`Training workshop`, `Section office`, `Monitoring visits`
`QtyRequested`	Requested	Number (0 dp)	Yes	No	`Min="0"`
`QtyApproved`	Approved	Number (0 dp)	No	No	`Min="0"`. Set on the Review screen.
`QtyDistributed`	Distributed	Number (0 dp)	No	No	`Min="0"`
`QtyReceived`	Received	Number (0 dp)	No	No	`Min="0"`
`SourceOfficeCode`	Source office code	Single line of text	No	No	Stamped when the line is filled from a lending office (`JCO` for the flipchart pads)
`LineStatus`	Line status	Choice	No	No	`Pending`, `Partially distributed`, `Fully distributed`, `Partially received`, `Fully received`, `Cancelled`
`ReceiptCondition`	Receipt condition	Choice	No	No	`Complete`, `Short`, `Damaged`, `Not received`
`LineNote`	Line note	Multiple lines of text	No	No	Plain text

Why the snapshot. A requisition must record what was requested *at the time*, not what the catalogue says today after a rename or reprice. It also makes the free-text line a normal row rather than a special case.

8. RequestComments

Base template `GenericList`. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Comment	Single line of text (built-in)	Yes	No	Label, e.g. `SR-202607-0038`
`RequestRefNo`	Request reference	Single line of text	Yes	**Yes**	
`CommentText`	Comment	Multiple lines of text	Yes	No	Plain text, 4 lines
`AuthorEmail`	Author (email)	Single line of text	Yes	No	Shadow of `CommentAuthor`

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`CommentAuthor`	Author	Person	No	No	**Named `CommentAuthor`, not `Author`** — `Author` is the reserved internal name of SharePoint's built-in Created By field and cannot be created.
`PostedOn`	Posted on	Date and time	Yes	**Yes**	

9. RequestTimeline

Base template `GenericList`. Append-only by convention. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Event	Single line of text (built-in)	Yes	No	The headline the timeline renders, e.g. `Request submitted`
`RequestRefNo`	Request reference	Single line of text	Yes	**Yes**	
`EventType`	Event type	Choice	Yes	No	`Draft created`, `Submitted`, `Routed`, `Reviewed`, `Returned`, `Rejected`, `Confirmed`, `Distributed`, `Received`, `Closed`, `Comment added`, `Reassigned`, `Stock adjusted`
`FromStatus`	From status	Single line of text	No	No	Text, not Choice — blank at draft creation and never filtered on
`ToStatus`	To status	Single line of text	No	No	
`StageCode`	Stage code	Choice	No	No	`Review`, `Approval`, `Confirm`, `Deliver`, `Receive`
`ActorEmail`	Actor (email)	Single line of text	No	No	Shadow of `Actor`. Blank for a system event.
`Actor`	Actor	Person	No	No	
`OccurredOn`	Occurred on	Date and time	Yes	**Yes**	
`Detail`	Detail	Multiple lines of text	No	No	Plain text, 4 lines

10. RequestDocuments

Base template `DocumentLibrary` — a library, not a list with attachments, because attachments cannot be queried across items. Versioning on, major versions only.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Title	Single line of text (built-in)	No	No	Optional in a library; `FileLeafRef` is the real name

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`RequestRefNo`	Request reference	Single line of text	Yes	**Yes**	
`DocType`	Document type	Choice	Yes	No	`Request PDF`, `Delivery note`, `Receipt note`, `Supporting document`
`UploadedByEmail`	Uploaded by (email)	Single line of text	No	No	

11. AppRoles

Base template `GenericList`. Versioning on. Attachments off.

Not in the mock-up's implied model, but required: the screens gate on role, and one person holds several — Deng Machar is both **Reviewer** and **Requester**.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Person	Single line of text (built-in)	Yes	No	Display name, e.g. `Deng Machar`
`UserEmail`	User email	Single line of text	Yes	**Yes**	
`OfficeCode`	Office code	Single line of text	Yes	No	
`RoleCode`	Role	Choice	Yes	No	`Requester`, `Reviewer`, `Chief Field Office`, `Storekeeper`, `Administrator`
`IsActive`	Active	Yes/No	No	No	Default `1`

12. RefCounter

Base template `GenericList`. Versioning on. Attachments off.

Internal name	Display name	SharePoint type	Required	Indexed	Notes / verbatim choice values
`Title`	Period	Single line of text (built-in)	Yes	Yes	`YYYYMM`, e.g. `202607`. Indexed so the per-mint lookup is deterministic.
`LastSeq`	Last sequence	Number (0 dp)	Yes	No	`Min="0"`. The highest `NNNN` minted in that month.

Reference numbering restarts each month, as the mock-up shows (`SR-202606-0114` in June, then `SR-202607-0029` in July). Implementation: `Patch` the `RefCounter` row for `YYYYMM`, read back `LastSeq + 1`, mint the reference, and rely on `EnforceUniqueValues` on `Requests.Title` plus an `IfError` retry loop capped at 5 attempts.

StatusRank

`<>` on Text does not delegate to SharePoint and `Not` never delegates, so `Status <> "Closed"` is poison. Every workflow query is a *range* query, so status carries a shadow number written in the same `Patch`.

Status (verbatim, Choice)	StatusRank
Draft	10
Returned	15
Submitted	20
Reviewed	30
Partial delivery	40
Fully delivered	50
Fully received	60
Closed	70
Rejected	90

Open = `StatusRank < 70`. Never `Sort` by `Status` — sorting a Choice column is not delegable under any circumstances.

StageCode

Five choice values, used by `ApprovalMatrix.StageCode`, `Requests.CurrentStageCode` and `RequestTimeline.StageCode`:

StageCode	The actor stage it names
`Review`	Reviewer
`Approval`	Finance approval
`Confirm`	Chief Field Office
`Deliver`	Storekeeper
`Receive`	Requester (originator)

The six journey legs the rail renders are, verbatim from `request-detail.html`:

`Submitted` → `Review` → `Approval` → `Confirm` → `Deliver` → `Receive & close`

`Submitted` is the requester's own act and needs no matrix row, which is why there are five `StageCode` values for six legs. So **`ApprovalMatrix` holds up to five rows per office + section**.

The skip rule is the absence of a row. A stage is skipped iff no active matrix row exists for that `MatrixKey` + `StageCode`. No sentinel rows, no `IsSkipped` flag. Bor has four rows and no `Approval` row — which is exactly why `admin.html` shows four steps and the rail strikes `Approval` through. The rail renders by left-joining the six canonical stages against the collected matrix rows; a skipped leg uses the Label `Strikethrough` property, not a separate control.

The matrix's displayed `Stage` string is derived, never stored. `admin.html` shows `SUBMIT → Review`, `REVIEW → Confirm`, `CONFIRM → Distribute`, `DISTRIBUTE → Receipt`. That label is built at render time from `<previous active stage> → <this stage>`. That is precisely what makes the chain reconfigurable: add an `Approval` row and step 2's label becomes `REVIEW → Approval` with no code change. Storing it would freeze the chain.

Rules that hold across every list

1. **Internal names are clean PascalCase, no spaces.** Every column is created through `m365 spo field add --options AddFieldInternalNameHint --xml '<Field …/>'` with `Name` and `StaticName` both set. Creating a column called "Office code" in the web UI would give it the internal name `Office_x0020_Code`; almost every "delegation randomly stopped working" report traces to exactly that. Display names may contain spaces; internal names never do.
2. **Zero Lookup columns anywhere in the design.** Relationships are carried by shadow text keys — `RequestRefNo`, `OfficeCode`, `OfficeItemKey`, `MatrixKey`. Lookups make a column Complex (killing `Sort` and `StartsWith` on it), hit the per-view lookup cap, throw on over-threshold lists, and — decisively — a Lookup to `Catalogue` would make the "Other (type it in)…" free-text line impossible.
3. **Every Person column has a shadow `…Email` single-line-of-text column**, written in the same `Patch`. Filter only on the shadow (deterministic index use above 5000 items); the Person column earns its keep for the list UI and future flows.
4. **Indexes are declared inline at column-creation time** (`Indexed="TRUE"` in the CAML). SharePoint refuses to add an index to a list already over 5000 items, so the provisioning order is strictly `01-lists.sh` → `02-fields.sh` → `03-seed.sh`. Getting this backwards is unrecoverable without recreating the list.
5. **Uniqueness** is `EnforceUniqueValues="TRUE"`, which requires `Indexed="TRUE"` on the same column. Two columns carry it: `Requests.Title` and `Catalogue.OfficeItemKey`.
6. **Multiple-lines-of-text columns are plain text**: `RichText="FALSE"`, `AppendOnly="FALSE"`. Rich text and append-only versioning both break Power Fx round-tripping.
7. **Number columns are `Decimals="0"`** unless they are currency, which is `Type="Currency" Decimals="2"`.
8. **`RequestDocuments` is a document library** (`--baseTemplate DocumentLibrary`), not a list with attachments — attachments cannot be queried across items. Attachments are disabled on every other list.
9. **`Title`** is the SharePoint built-in and cannot be created with `spo field add`. Where a list uses it as a real key it is reconfigured afterwards with `m365 spo field set --internalName Title …` (display name, `Required`, `Indexed`, `EnforceUniqueValues`). Where a list has no natural use for it, it still exists and is required by SharePoint, so it carries a human-readable label.

Delegation contract

Five queries, verified against the SharePoint connector's delegation table. The app-build phase copies these verbatim.

Query	Formula
Awaiting my action	`Filter(Requests, StatusRank < 70, Or(CurrentlyWithEmail = User().Email, CurrentlyWithOICEmail = User().Email))`
My requests	`SortByColumns(Filter(Requests, RequesterEmail = User().Email), "SubmittedOn", SortOrder.Descending)`
Lines for a request	`Filter(RequestItems, RequestRefNo = varRef)`
Catalogue for an office	`Filter(Catalogue, OfficeCode = varOffice, IsActive = true)`
Item picker search	`Filter(Catalogue, OfficeCode = varOffice, StartsWith(ItemName, cbo.SearchText))` — use `StartsWith`, **never** `Search()`

The rules those queries rest on

- `Person.Email = X` **is** delegable (only `Email` and `DisplayName` are).
- `Choice.Value = X` **is** delegable; `Sort` on Choice is **not**.
- `<>` on Text, `Not`, `IsBlank`, `Sum`, `CountRows` — **not** delegable.
- SharePoint `ID` delegates `=` only; there is no `ID >` paging.
- `And` / `Or` of delegable expressions **are** delegable.

Reports

Reports cannot aggregate server-side. Bound the set with a delegable filter (office + date range) into a collection, aggregate locally with `GroupBy` / `Sum`, and display the row count so a truncated aggregate is never presented as a KPI.

Index inventory

Every column below carries `Indexed="TRUE"` at creation time. SharePoint allows 20 indexes per list; the busiest list here uses seven.

List	Indexed columns
Offices	`Title`
OfficeSharing	—
Catalogue	`OfficeCode`, `OfficeItemKey` (unique)
StockLedger	`OfficeItemKey`, `OfficeCode`, `MovementType`, `RequestRefNo`, `MovedOn`
ApprovalMatrix	`MatrixKey`
Requests	`Title` (unique), `OfficeCode`, `RequesterEmail`, `CurrentlyWithEmail`, `CurrentlyWithOICEmail`, `StatusRank`, `SubmittedOn`
RequestItems	`RequestRefNo`
RequestComments	`RequestRefNo`, `PostedOn`
RequestTimeline	`RequestRefNo`, `OccurredOn`
RequestDocuments	`RequestRefNo`
AppRoles	`UserEmail`
RefCounter	`Title`

Appendix B. Defect checklist

Every defect from UAT round 2, and where this guide handles it. Generated with `powerapps/uat-mapping.md`.

Defect checklist

All 60 defects from UAT round 2. **Fixed** ones are behaviour the canvas app must reproduce; **By design** ones are deliberate and stay as they are unless the product owner decides otherwise (Appendix C).

DEF	Severity	What went wrong in the SPFx build	Status	Where the canvas build handles it
DEF-01	High	Catalogue product dropdown never appears on New request	Fixed	ch. 6 `cboProduct`, `btnAddLine`
DEF-02	Medium	Item notes are not saved	Fixed	§8/§9 `LineNote`; ch. 7 Notes column
DEF-03	Medium	Home's Review button ignores the stage	Fixed	ch. 4 `btnAwaitingAction` via `tblStageAction`
DEF-04	Medium	Return and Reject are not limited to the current holder	Fixed	§10/§11 `DisplayMode`
DEF-05	High	Remainder of a partial delivery cannot be distributed	Fixed	§12 `varDeliverWith`: a partial stays at Deliver
DEF-06	Medium	Failed saves are mostly silent	Fixed	Every `IfError` in §8–§15; §12 line summary
DEF-07	Medium	Documents with the same file name overwrite each other	Fixed	§14 stored name and duplicate check; the flow
DEF-08	Low	No segregation of duties	By design	Not changed: one person may hold several stages
DEF-09	Low	Email casing is compared exactly	Fixed	App.OnStart `Lower()`; Recipe R3
DEF-10	Low	Three left-nav items go back to Home	Fixed	ch. 3 menu `OnSelect`; ch. 5
DEF-11	Low	A deactivated catalogue item cannot be reactivated in the app	Fixed	ch. 11 `colAdminCat` keeps inactive items; `tglEditItemActive`
DEF-12	Low	Status says Reviewed after the Confirm step	By design	Not changed: Proceed writes Reviewed at every stage
DEF-13	Low	Reports: Requests by office ignores the office filter	Fixed	ch. 12 `colByOffice` from `colScopeReq`
DEF-14	Low	The Active flag on an office does nothing	Fixed	`IsActive <> false` in every office picker; ch. 11 lists all
DEF-101	Medium	Posting a comment or attaching a document adds no event to the Timeline	Fixed	§14 timeline rows
DEF-102	Medium	Home counts a person's own Returned and Fully delivered requests as awaiting their action	By design	Not changed: Home counts them

DEF	Severity	What went wrong in the SPFx build	Status	Where the canvas build handles it
DEF-103	Medium	Confirm and Deliver steps are routed to the officer in charge, not the matrix approver	Fixed	Recipe R2: approver holds, OIC beside
DEF-104	Low	Confirm step writes the timeline title "Request confirm confirmed"	Fixed	§10 `Switch(varFromStage, …)` titles
DEF-105	Medium	Add dialogs keep the previous values when reopened	Fixed	ch. 11 `Reset()` before each dialog
DEF-106	Medium	A closed request shows 'Unknown' as the actor of the last journey step	Fixed	ch. 7 `colJourney.Actor`
DEF-107	Low	The Confirm step's screen is titled 'Review'	Fixed	ch. 8 `ScreenTitle` in `varPageTitle`
DEF-108	Low	A new line's Tracking badge says 'Unknown'	Fixed	ch. 7 Tracking `Coalesce(…, "Pending")`
DEF-109	Low	The request reference wraps over three lines in a narrow header	Fixed	ch. 3 reference label `Wrap: false`
DEF-110	Low	The Edit request banner promises a new reference number	Fixed	ch. 6 `lblRouting`
DEF-111	Low	Currently with says Unknown for a draft	Fixed	`txtNoHolder` in "Currently with"
DEF-112	Medium	Distribute does not say which office the stock comes from	Fixed	ch. 9 "Available now · from …"
DEF-113	Low	'1 items' on the Review and Distribute headers	Fixed	"1 item" / "N items" formulas
DEF-114	Medium	The request reference is clipped in the Review and Confirm headers	Fixed	ch. 3 reference label; top bar sizing
DEF-115	Low	New request says Defaulted from your profile after the office is changed	Fixed	ch. 6 `lblOfficeHint.Visible`
DEF-116	Low	A bare ellipsis stands in for the office name, and the persona chip is clipped	Fixed	`varMyOfficeName` falls back to "No default office"
DEF-117	Medium	Remaining on the request screen counts requested minus received, not what is still to be delivered	Fixed	ch. 7 Remaining column
DEF-118	Low	Stock after says below threshold when the stock after equals the threshold	Fixed	ch. 9 `lblAfter`
DEF-119	Low	The Distributed timeline row prints the running total as if it were this step	Fixed	§12 `Detail`
DEF-120	Low	Home's action button says Review for a request waiting at Confirm	Fixed	ch. 4 and 5 `RowLabel`
DEF-121	Low	The item note typed on New request is shown nowhere	Fixed	ch. 7 Notes column
DEF-122	Low	No Routed to Receive row after the last distribution	Fixed	§12 "Routed to Receive"
DEF-123	Low	The Receipt confirmed row is blank	Fixed	§13 `Detail`
DEF-124	Medium	A Returned request offers Review, and the Review screen lets a reviewer act on it	Fixed	Recipe R3 `varIsEditable`; ch. 4, 7, 8 buttons

DEF	Severity	What went wrong in the SPFx build	Status	Where the canvas build handles it
DEF-125	Low	The Edit request screen does not show why the request was returned	Fixed	ch. 6 `lblReturnReason`
DEF-126	Low	Distribute says 'above threshold' for an item with no threshold	Fixed	ch. 9 `lblAfter` wording
DEF-127	Low	A Returned request still shows the Reviewer holding it at the Review step	Fixed	ch. 7 `varJourneyStage`
DEF-128	Low	Review & distribution offers Review for a Returned request	Fixed	ch. 5 `galQueue.Items`
DEF-129	Low	The Edit request screen lights New request in the nav	Fixed	ch. 6 `varNavActive`
DEF-130	Low	Timeline rows from the same minute swap places between loads	Fixed	ch. 7 sort by OccurredOn then ID
DEF-131	Low	The brand line under Supply Requests is blank on New and Edit request	Fixed	ch. 3 `lblBrandOffice`
DEF-132	Medium	'Held for' counts from submission, not from the last hand-off	Fixed	Recipe R5 from `Modified`
DEF-133	Low	The Edit request title does not show the reference	Fixed	ch. 6 `varPageTitle`
DEF-134	Medium	Home's Confirm receipt button in Awaiting my action opens the request screen, not the receipt form	Fixed	ch. 4 button `OnSelect` is not `Select(Parent)`
DEF-135	Medium	Any role can open Catalogue & matrix and its edit controls	By design	Not changed; ch. 11 shows the optional gate
DEF-136	Low	A line cut by the reviewer shows Requested 4 and Fully received 2 with no explanation	Fixed	ch. 7 Approved column
DEF-137	Low	'This quarter' on Reports is a rolling 91 days	Fixed	ch. 12 `varSince`
DEF-138	Medium	Median time held per stage shows only Review	Fixed	Routed rows carry `StageCode` (§8, §10, §12)
DEF-139	Low	The Active hint on Edit office is stale	Fixed	ch. 11 office dialog resets from `varEditOffice`
DEF-140	Low	Median delivery tile reads 0.0 days	Fixed	ch. 12 `lblKpiMedian`
DEF-141	Low	A rejected request shows its later journey legs as complete	Fixed	ch. 7 `varJourneyStage`, `varJourneyDone`
DEF-142	Low	The 'This quarter' range chip starts on 30 Sep	Fixed	ch. 12 `DateTime(…, 1, 0, 0, 0)`
DEF-143	Medium	Median time held per stage draws no bars	Fixed	ch. 12 `varStageUnit`
DEF-144	Medium	Distribute screen shows zero stock for a few seconds after it opens	Fixed	ch. 9 `varDistReady`
DEF-145	Low	Reports stage chart and delivery tile ignore the office filter	Fixed	ch. 12 `colScopeTL`
DEF-146	Low	The stage chart caption always names Review as the slowest stage	Fixed	ch. 12 `lblStageCaption`

Appendix C. Decisions to confirm with the product owner

The current app made these choices during UAT. Each one is reproduced in this guide, because the canvas app should behave the same way. Each is also easy to change, and this appendix says where.

#	Decision	What the app does	Where to change it
1	**The officer in charge holds a stage beside the approver** (DEF-103)	When a matrix row names both, either person can act, and both see the request in "Awaiting my action".	Recipe R2 (§7): to make the OIC *replace* the approver while set, use `WithEmail: Coalesce(next.OIC, next.Approver, R)` and `OICEmail: ""`.
2	**A partial delivery stays with the storekeeper** (DEF-05)	The requester can only confirm receipt once everything approved has gone out.	§12, `CurrentStageCode` / `CurrentlyWithEmail` in the header patch.
3	**A short receipt still closes the request**	If everything approved was delivered, confirming receipt closes it; a shortfall is recorded on the line only.	§13, `varCloseNow`.
4	**Proceed always writes "Reviewed"** (DEF-12)	The status says Reviewed at Approval and Confirm too; the stage column says where it is.	§10, the header patch's `Status`.
5	**Held for counts from the last change** (DEF-132)	Any change to the request, not just a hand-off, restarts the clock.	Recipe R5.
6	**No segregation of duties** (DEF-08)	One person may review, confirm and distribute the same request if the matrix names them.	Add a check on the Proceed and Record distribution buttons against `colTimeline`'s actors.
7	**Everyone can open Catalogue & matrix and Reports** (DEF-135)	No role gate.	Chapter 11, "Who can open this screen".
8	**Home counts my own Returned and Fully delivered requests** (DEF-102)	They are mine to act on (edit, confirm receipt), so they are "awaiting my action".	Home's `colAwaiting` filter.
9	**The stage order is fixed**	Requests always move Review → Approval → Confirm → Deliver → Receive, skipping stages with no row. `StageOrder` on the matrix only orders the admin list.	`tblJourney` in `App.Formulas`.
10	**Reports filter on the office of the request** (DEF-13)	Every figure follows the office filter.	Chapter 12, `colScopeReq`.

Small differences from the current app

These are deliberate. Each corrects something the current app did by accident.

Difference	Why
Return, Reject and Close **clear** `CurrentStageCode` (`Blank()`).	The SPFx build sent an empty value that SharePoint ignored, so the old stage code stayed; four defects (DEF-124, 127, 128, 141) were screen-side workarounds for that. The workarounds stay too, so older rows still show correctly.
Return and Reject timeline rows carry the stage they left.	Reports counts a return as the end of a stage, as `src/domain/timeline.ts` intends; the SPFx rows had no stage code, so returns were never counted.
The distribution timeline row's "from" status is the request's real status.	The SPFx build always wrote "Reviewed", which is wrong for a second partial delivery.
Edit mode disables Submit for anyone but the requester, and once the request is no longer a Draft or Returned.	The SPFx build relied on the Edit buttons being hidden.
Reference minting writes five attempts out in full.	The first canvas plan's loop (`app-glue.md` §6) can't be built: Power Apps doesn't allow `Set` inside `ForAll`.

Not reproduced

Item	Note
Reconciliation (`app-glue.md` §5.2)	The first canvas plan specified a "Reconcile" button that repairs `Catalogue.StockQty` from the ledger. The SPFx build never shipped it (its `findDrift` is unused), so this guide leaves it out. It can be added to Catalogue & stock from §5.2 as written.
Reassign	The Request detail mock-up has a Reassign button; neither build implements it.
Open in Power BI	Disabled in both builds.

Appendix D. Glossary

Term	Meaning
Canvas app	A Power Apps app you lay out screen by screen, like a slide deck, and wire up with formulas.
Studio	The Power Apps editor at make.powerapps.com, where you build the app.
Power Fx	The formula language of canvas apps, close to Excel formulas.
Control	Anything on a screen: a label, button, gallery, text box, dropdown.
Property	A setting of a control, such as `Text`, `Fill`, `Visible` or `OnSelect`. A property holds a formula.
Behaviour property	A property that runs when something happens (`OnSelect`, `OnChange`, `OnVisible`). Only these can save data or set variables.
Gallery	A control that repeats a template for each row of a table. Every list and table in this app is a gallery.
Collection	A table held in the app's memory, named `col…`. Filled with `ClearCollect`.
Variable	A single value held by the app, named `var…`. Set with `Set`.
Named formula	A value defined once in `App.Formulas` and used by name everywhere (`clrBrand`, `tblStatus`).
Data source	A SharePoint list the app is connected to.
Delegation	Asking SharePoint to do the filtering, so the app gets only the rows it needs. A formula that can't be delegated quietly works on the first rows only.
Data row limit	The most rows the app fetches when a formula can't be delegated. 1 while checking, 2,000 in use.
Choice column	A SharePoint column with a fixed list of values. Read as `Status.Value`, written as `{ Value: "Submitted" }`.
Shadow column	A plain column kept beside another so it can be filtered quickly: `StatusRank` beside `Status`, `RequesterEmail` beside `Requester`.
Approval matrix	The `ApprovalMatrix` list: one row per office, section and stage, naming who acts.
Stage	One step of a request's journey: Review, Approval, Confirm, Deliver, Receive.
Holder	The person a request is currently with (`CurrentlyWithEmail`), and the officer in charge beside them (`CurrentlyWithOICEmail`).
OIC	Officer in charge: a stand-in named on a matrix row, who can act alongside the approver.
Lending office	An office that can supply another's shortfall (`OfficeSharing`), lower `SharePriority` first.
Ledger	`StockLedger`: one row for every stock movement. The stock figure on `Catalogue` must always match its newest row.
Reference	A request's number, `SR-YYYYMM-NNNN`, unique and counting from 1 each month (`RefCounter`).
Recipe	A block of Power Fx this guide uses on several screens, defined in §7 (R1–R6).
Impersonation	Opening the app as someone else with `&as=<email>`, for testing only.
Flow	A Power Automate process. This app has one, `UploadRequestDocument`.
Solution	A package that carries the app and the flow between environments.
DEF-nnn	A defect number from the UAT defect log (Appendix B).
UAT	User acceptance testing: the 31 recorded cases the app must pass.